



Faculty of Computing and AI, Air University, Islamabad
Department of Creative Technologies
Software Engineering



FRACTAL

By

Ahmad Hassan	FA22-BSSE-221775
Faraz Ashraf	FA22-BSSE-221847

Supervisor

Dr. Iqbal Murtza

Bachelor of Science in Software Engineering (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



Faculty of Computing and AI, Air University, Islamabad
Department of Creative Technologies
Software Engineering



FRACTAL

**A project presented to
AIR University, Islamabad**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Software Engineering (2022-2026)

By

Ahmad Hassan	221775
Faraz Ashraf	221847

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Student Name 1

Ahmad Hassan

FA22-BSE-221775/ISB

Student Name 2

Faraz Ashraf

FA22-BSE-2218475/ISB

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “**FRACTAL**” was developed by **Ahmad Hassan (CIIT/FA22-BSE/TN-221775)** and **Faraz Ashraf (CIIT/FA22-BSE/TN-221847)** under the supervision of “Dr. Iqbal Murtza” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.



Supervisor

External Examiner

Head of Department
(Department of Computer Science)

Executive Summary

The increase in innovation demands an increase in computation resources like GPUs, which are prohibitively expensive and are not suitable for an average independent user or a small company. Existing solutions are centralized to specific devices that are fewer in number, have a static location, and have high battery consumption. Meanwhile, the billions of handheld devices remain idle, wasting an unfathomable amount of combined computation power every second. The billions of these handheld devices remain idle, wasting an unfathomable amount of combined computation power every second. The data centers used for computation require water for cooling and increase of water use for data centers of Microsoft increases from 6.4 million cubic meters to 7.8 from 2022 to 2023. There is an urgent need for innovative strategies for energy-efficient infrastructure to mitigate the data centers' impact on the environment “FRACTAL” a decentralized distributed computation harvesting platform proposed address the gap by developing computing platform that utilize the computation power of mobile devices. The system consists of two modules, one is client side module that involves the participation of mobile devices that provide computation for model training, preprocessing and resource allocation, and other server-side modules that handles the management, database, model aggregation, and task allocation to the devices. By shifting computation paradigms to mobile devices, we reduce the reliance on cost-effective data centers either entirely or partially, contribute to global sustainability efforts by reducing the reliance on huge data centers that lead to carbon emissions, high resource consumption, etc., and utilize computational resources at every possible opportunity. The key benefit of the product is that it provides access to high computation resources, making it accessible for an average independent user or a small company, promoting global sustainability by reducing high infrastructure dependency.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Dr. Iqbal Murtza”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Student Name 1

Ahmad Hassan

Student Name 2

Faraz Ashraf

Abbreviations

zkVM	Zero knowledge virtual machine
BOINC	Berkeley open infrastructure for network computing
SRS	Software Requirement Specification
SDD	Software design document
ML	Machine learning
OS	Operating system
CKPT	Checkpoint
MVC	Model View Controller

Table of Contents

1	Introduction.....	13
1.1	Vision Statement	13
1.2	Related System Analysis/Literature Review	13
1.3	Project Deliverables.....	15
1.4	System Limitations/Constraints.....	16
1.5	Tools and Technologies.....	16
1.6	Relevance to Course Modules	17
2	Problem Definition.....	18
2.1	Problem Statement.....	18
2.2	Problem Solution	19
2.3	Objectives of the Proposed System	19
2.4	Scope	20
2.5	Modules	21
3	Requirement Analysis	23
3.1	User classes and characteristics	23
3.2	Requirement Identifying Technique	24
3.3	Functional Requirements.....	25
3.3.1	Functional Requirement 1	25
3.3.2	Functional Requirement 2	26
3.3.3	Functional Requirement 3	26
3.3.4	Functional Requirement 4.....	27
3.3.5	Functional Requirement 5	27
3.3.6	Functional Requirement 6.....	28
3.3.7	Functional Requirement 7	28
3.3.8	Functional Requirement 8.....	29
3.3.9	Functional Requirement 9.....	29
3.3.10	Functional Requirement 10.....	30
3.3.11	Functional Requirement 11	30
3.3.12	Functional Requirement 12	31
3.3.13	Functional Requirement 13	31
3.3.14	Functional Requirement 14.....	32
3.3.15	Functional Requirement 15	32
3.3.16	Functional Requirement 16.....	33
3.3.17	Functional Requirement 17	33
3.3.18	Functional Requirement 18.....	34
3.3.19	Functional Requirement 19	34
3.3.20	Functional Requirement 20.....	35
3.3.21	Functional Requirement 21	35
3.3.22	Functional Requirement 22	36
3.3.23	Functional Requirement 23	36
3.4	Non-Functional Requirements.....	37
3.4.1	Reliability	37
3.4.2	Usability	37
3.4.3	Performance	37
3.4.4	Security	38
3.5	External Interface Requirements	38
3.5.1	User Interfaces Requirements	38
3.5.2	Software interfaces	39

3.5.3	Communications interfaces	40
4	Design and Architecture	41
4.1	Architectural Design.....	41
4.2	Design Models.....	1
4.3	Data Design	34
4.3.1	Data Dictionary	35
4.4	Human Interface Design.....	37
4.4.1	Screen Objects and Actions.....	42
4.4.2	Objects.....	42
4.4.3	Actions	42
5	Implementation	43
5.1	Algorithm	43
5.2	User Interface	45
6	Testing and Evaluation	52
6.1	Unit Testing.....	53
6.2	Functional Testing.....	60
6.3	Business Rules Testing.....	63
6.4	Integration Testing.....	66
7	Conclusion and Future Work	68
7.1	Conclusion.....	68
7.2	Future Work.....	68
8.	References.....	70

List of Tables & Figures

Table 1: Related System Analysis with Proposed Project Solution	14
Table 2: Tools and Technologies for Proposed Project	17
Table 3: User classes and characteristics	23
Table 4: Event response table	24
Table 5: FR1	25
Table 6: FR2	26
Table 7: FR3	26
Table 8: FR4	27
Table 9: FR5	27
Table 10: FR6	28
Table 11: FR7	28
Table 12: FR8	29
Table 13: FR9	29
Table 14: FR10	30
Table 15: FR11	30
Table 16: FR12	31
Table 17: FR13	31
Table 18: FR14	32
Table 19: FR15	32
Table 20: FR16	33
Table 21: FR17	33
Table 22: FR18	34
Table 23: FR19	34
Table 24: FR20	35
Table 25: FR21	35
Table 26: FR22	36
Table 27: FR23	36
Table 28: Event response table	9
Table 29: Algorithm.....	44

Figure 1: Box and line diagram	41
Figure 2: Architecture diagram	42
Figure 3: APP Interface 1	1
Figure 4: APP Interface 2	2
Figure 5: APP Interface part3	2
Figure 6: APP Interface 3	2
Figure 7: APP Interface 5	3
Figure 8: APP Interface 4	3
Figure 9: APP Interface complete.....	4
Figure 10: APP Interface	5
Figure 11: Resource management.....	5
Figure 12: Interface.....	6
Figure 13: App Global	7
Figure 14: ER1-Activity diagram	10
Figure 15: ER1 system sequence diagram	11
Figure 16: ER1 State transition diagram.....	11
Figure 17: ER2 Activity diagram.....	12
Figure 18: ER2 System sequence diagram	13
Figure 19: ER2 State machine diagram	14
Figure 20: ER3 Sequence diagram	14
Figure 21: ER3 Activity diagram.....	15
Figure 22: ER3 State machine diagram	16
Figure 23: ER4 State machine diagram	16
Figure 24: ER4 Activity diagram.....	17
Figure 25: ER4 System sequence diagram	18
Figure 26: ER5 State machine diagram	18
Figure 27: ER5 Activity diagram.....	19
Figure 28: ER5 System sequence diagram	20
Figure 29: ER6 Activity diagram.....	21
Figure 30: ER6 System sequence diagram	22
Figure 31: ER7 Activity diagram.....	23
Figure 32: ER7 Sequence diagram	24
Figure 33: ER7 State machine diagram	24
Figure 34: ER8 Activity diagram.....	25
Figure 35: ER8 Sequence diagram	26
Figure 36: ER9 Activity diagram.....	27
Figure 37: ER9 System sequence diagram	28
Figure 38: ER9 State machine diagram	29
Figure 39: ER10 Activity diagram.....	30
Figure 40: ER10 System sequence diagram	31
Figure 41: ER10 State machine diagram	31
Figure 42: ER11 Activity diagram.....	32
Figure 43: ER11 Sequence diagram	33
Figure 44: ER11 State machine diagram	34
Figure 45: ERD diagram.....	35
Figure 46: Home	38

Figure 47: Setting.....	38
Figure 48: Active	39
Figure 49: Model training	39
Figure 50: Device insight screen.....	40
Figure 51: Usage insight screen.....	40
Figure 52: Status screen	41
Figure 53: Registration screen	41
Figure 54: About US UI.....	46
Figure 55: Device Insight UI	46
Figure 56: Device registration UI	47
Figure 57: Forget password UI	47
Figure 58: Active UI	48
Figure 59: Inactive UI.....	48
Figure 60: Model training UI.....	49
Figure 61: Registered Info UI.....	49
Figure 62: Unregister UI.....	50
Figure 63: Setting UI	50
Figure 64: Heartbeat UI	51

1 Introduction

Fractal is a decentralized distributed computing platform that will utilize the computational power of android mobile devices. Based on current progress and completed modules, project successfully implemented the development of a mobile side application package, application package load, loading of model, in device training, initialization of weight and bias, interpreter loading, data array creation and training of fixed tasks. All the progress reflects the core innovation of the project enabling training on mobile-on-mobile devices

1.1 Vision Statement

For AI/ML companies, research institutions, independent developers, tech startups, and small, medium, and large enterprises that require extensive computing needs for their high computation intensive tasks/projects, the decentralized distributed computation harvesting platform is a mobile based application that utilizes the computation power of mobile devices (primarily Android devices) to provide cloud computation services. Unlike traditional computation solutions such as CPU/GPU farms, which are prohibitively expensive and are not suitable for an average independent user or a small company, our software will enable low-cost, scalable, yet powerful computing services as a unified computation unit. This will eliminate the need for multimillion CPU/GPU farms, reducing the cost of high-performance computations significantly. Utilizing the decentralized approach will result in lower infrastructure dependency, fault tolerance, sustainability, and an eco-friendly computation harvesting environment.

1.2 Related System Analysis/Literature Review

Systems related to our proposed projects are provided below

- In Yokohama, Japan, a conference took place in which some researchers published a paper in which they propose a distributed machine learning system, i.e., virtual data centers with electric vehicles, which includes electric cars parked in parking lots, hybrid LAN, and central servers(Bai, Jia, Deng, & Dong, 2024).This distributed

machine learning system aims to utilize the computational power of electric cars parked in parking lots for model training.

- BOINC is an open-source software platform for computing using volunteer resources (Anderson., n.d.). BOINC is designed especially for computer owners. Computer owners participate in the BOINC projects and monitor how their resources are allocated
- The Nexus network is a verifiable supercomputer that combines the computational power of worldwide devices into single computers (Marin, n.d.). With Nexus zkVM, a zero-knowledge virtual machine instance, each node supplies computation. In the zkVM 3.0 update, users earn nex points by providing computation.

Table 1: Related System Analysis with Proposed Project Solution

Application Name	Weakness	Proposed Project Solution
BOINC	Volunteering computing is only for computer owners No federated learning	Our proposed project utilizes mobile devices and designs for federated learning, allowing model training with the exchange of model updates, offering privacy and efficiency

NEXUS	Require high-power devices for computation High battery consumption due to zk proof (verification process)	Our proposed project with TensorFlow offers better support for Androids also it offers adaptive model updates and selective participation to weak devices and federated learning, allowing model training with exchange of model
		updates instead of raw which leads to less battery consumption.
Distributed machine learning system (DMLS)	Distributed machine learning systems are limited to electric vehicles only, with limited scalability due to only 40 million vehicles, of which 17.1 million are smart cars. Static location for computation	Our proposed project utilizes mobile devices that number more than 6.8 billion now, so there are no scalability constraints, limitations, or easy accessibility. Dynamic network and independent of location constraint

1.3 Project Deliverables

The project deliverables are provided below:

- Client side application
- Software requirement specification document (SRS)
- Software design document (SDD)
- Final report
- Final presentation

1.4 System Limitations/Constraints

- **Network Latency**

The system depends on communication between devices and servers for task allocation, model aggregation, etc. Any kind of lag or delay in communication impacts the model training efficiency.

- **Device availability and computation fluctuations**

Device and computation power availability or consistency is unpredictable; computation power is not guaranteed at any time; it may fluctuate, so the availability of the device may be turned off by the user or disconnected due to network issues.

- **Software and compatibility issues**

The system working depends on software such as TensorFlow Lite, Kotlin libraries, etc., which may not be available for all devices. These may not be backward or forward compatible, thus leading to complicated version issues.

- **Dependency on device-specific optimization**

Devices differ in their capabilities, such as CPU, RAM, battery, etc., which results in inconsistent training power. For smooth work, the system must dynamically adapt to the device's capabilities, which is also a challenge on firmware, legacy, and hardware levels.

- **Platform Fragmentation**

The diversity in devices' hardware capabilities and operating systems makes it difficult for single-fit work mechanisms. For smooth work, the system is required to dynamically accommodate differences.

1.5 Tools and Technologies

The tools and technologies for the implementation of the proposed project are given below:

Table 2 Tools and Technologies for Proposed Project

Tools And Technologies	Tools	Version	Rationale
	Android Studio	latest	IDE for Android App Development
	Firebase	latest	Backend Services
	Figma	latest or 125.2.3	Design Work
	Technology	Version	Rationale
	Python	3.11 or 3.12 ++	Backend Processing
	Kotlin	2.0.20	App Development
	TensorFlow	2.18	Create ML models
	TensorFlow Lite	latest	Deploy ML models on devices
	Firebase Fire store	latest	Database
	Firebase Authentication	latest	Authentication
	Firebase Cloud Functions	latest	Server logic

1.6 Relevance to Course Modules

Fractal a decentralized distributed computation harvesting system is the combination of all the courses we studied during bachelors (BSSE).

- Programming fundamentals & OOP helps us in structuring and reusable of our code with concepts of encapsulation, class, inheritance, polymorphism etc.
- SDA and SCD help with architectural design of the system that uses such as MVC.
- Software project management, software quality management and software metrics ensure the quality and timely delivery of FYP 1, FYP2, FYP proposal and FYP3.
- Human computer interaction helps us in designing our UI to make it more efficient, accessible and usable.
- Mobile application development helps us to be aware of the Firebase fire store, firebase authentication and Android studio as these are the key tools and technologies
- So as do many other courses like Probability and statistics, OS and computer network etc.

2 Problem Definition

This section discusses the increase in computation needs within the past years, and the effect of the ways used to get this computation. Also, this section discussed the solution to these problems and the objectives of the proposed solution

2.1 Problem Statement

With the linearly increasing innovation, the computation needs are increasing exponentially. The computational power needed to train AI is now rising seven times faster than ever (Adnan Mehonic, 2021). From 1959 to 2012, the amount of computation to train AI models doubled every two years, which after 2012 shortened to every 3.4 months. This means computation to train AI models increases by a factor of 2 every 3.4 months (Thomas Gabor, 2020). Now, the number of phone users has reached 6.8 billion, and this shows that 86% of the world's population has at least one cell phone (Aslan, 2023). The billions of these handheld devices remain idle, wasting an unfathomable amount of combined computation power every second. The rapid increase in data centers due to their high energy consumption and excessive thermal footprints leads to significant environmental strain (Katherine Saltzman, 2025). The cooling requirement leads to an increase in carbon emission leading to heat dissipation impacting climate and sustainability goals. The blame on

ChatGPT for the Los Angeles fire is also due to the use of water as a cooling strategy for data centers (Jane Theirs, 2025). The increase of water use for data centers of Microsoft increases from 6.4 million cubic meters to 7.8 from 2022 to 2023. There is an urgent need for innovative strategies for energy-efficient infrastructure to mitigate the data centers' impact on the environment. Thus, our proposed software system is being developed to meet the exponential computational needs of any industry that requires extensive computation services.

Our software will enable low-cost, scalable, yet powerful computing services as a unified computation unit. This will eliminate the need for multimillion CPU/GPU farms, reducing the cost of high-performance computations significantly. The utilization of a decentralized approach results in lower infrastructure dependency, fault tolerance, sustainability, and an eco-friendly computation harvesting environment.

2.2 Problem Solution

A decentralized distributed computing platform that will utilize the computation power of mobile devices (primarily Android devices). The project will constitute a client-side mobile application installed on the user's device, which will be backed by a server alongside a device registration database. For the operational process: The computationally extensive task (such as AI model training) will be divided into smaller chunks and distributed among the active idle mobile phone devices. Each device will perform the assigned task locally for a fixed duration. After the duration, the processed data from the devices will be retrieved, aggregated, and federated into a single unit. This approach to harvesting computational resources from mobile devices will leverage the ability to unite the decentralized computational units into one single vast hyper-scale computational grid, thus providing an efficient, low-cost, and energy-efficient approach for providing resources to computation-hungry processes

2.3 Objectives of the Proposed System

- **Incorporating Explode in Modern Computation Needs**

While modern computation requires extensive scaling, by shifting the computation paradigm to mobile devices, the combined resources will provide a single supercomputing unit that will present services head-to-head and compete toe-to-toe to replace existing solutions.

- **Mitigating centralized computation bottlenecks**

By using a decentralized computation approach, training models parallelly on mobile devices will provide a solution where any single point of failure will be tolerated, by assigning the failed task to other available mobile devices

- **Utilization of idle time patches**

Accompanying 100% of the devices idle, the computational resources will be utilized at every possible time frame. When they are being used under idle times like at night, or charging times, the devices will be open to contribute fractionally even at their busiest times

- **Global sustainability and energy efficiency**

The computation utilization of the ever-increasing global market of mobile phones will auto incorporate the need for specialized computation units composed of data centers

that lead to carbon emissions, high resource consumption, etc. The distribution of load on a metropolitan or global scale will spread the thermal foot equally and reduce the thermal throttling, which leads to higher loads and temperatures, thus creating a vicious cycle. Our solution aims to break this vicious cycle and contribute to the effort to global sustainability by shifting the paradigm to individual mobile devices

2.4 Scope

The decentralized distributed computing system aims to utilize the computational power of mobile devices for model training. The goal of the system is to eliminate the need for multimillion CPU/GPU farms, reducing the cost of high-performance computations significantly.

The system will comprise two components: a client-side mobile application and a server-side backend. While online, a client-side mobile application is installed on the user's device, which will be backed by a server-side. While online, client-side mobile applications will operate independently. The client-side application will involve the participation of mobile devices and handling device registration, task requests, model training, and analyzing resources. The server side will handle management, databases, model aggregation, and task allocation to the devices.

A NoSQL database will be used in server-side storage, e.g., device registrations, untrained models, pre-trained models, etc. The main server will be utilized for providing backend services such as discovering the details of active registered devices, logging task assignments, cloud functioning, and other necessary backend logic.

The computationally intensive task will be divided into smaller chunks and distributed among the active idle mobile phone devices. Each device will perform the assigned task locally for a fixed duration. After the duration, the processed data from the devices will be retrieved, aggregated, and federated into a single unit.

By providing an efficient, low-cost, and energy-efficient approach to providing resources than traditional ones, this platform will be a solution for independent researchers and AI/ML companies requiring large-scale computation.

2.5 Modules

The project, “Decentralized distributed computation harvesting from mobile devices to provide cloud computation services,” is divided into two main modules: mobile-side modules and server-side modules. These modules work according to a queue-based asynchronous task system, which will handle requests, tasks, and model training in a non-blocking manner. These modules are further divided into sub-modules, which are provided below:

Module 1: Mobile-Side Module

Mobile-Side Module also called as Client Application module involves the participation of mobile devices that provide computation for model training. Mobile-Side Module further divided into sub modules which are provided below

Submodule 1: Network Module

FE-1: Each device is registered with the server. Registration using device ID, network status, RAM, battery health, etc.

FE-2: Exchange ping to devices to check the status with the return of ping from devices too to change or update

FE-3: Task requests from devices to servers in case of availability

Submodule 2: Data Reception & Preprocessing Module

FE-1: Receive data from local device or from the server

FE-2: Formatting the data into required structured format before model training

Submodule 3: Local Model Training Module

FE-1: With TensorFlow lite, model training execution is performed on devices

FE-2: Logs and models weigh stores asynchronously

Submodule 4: Weight & Bias Transmission Module

FE-1: After local model training, model weight and bias transmitted to the server

Submodule 5: Resource Management

FE-1: With resource statistics of devices like device status (online/offline), battery, memory, etc., analyze the availability of the device for model training.

FE-2: Based on device statistics and integrated preferences, dynamically start or stop the training of the device

Module 2: Server-Side Modules

The server-side module handles management, databases, model aggregation, and task allocation to the devices. The server-side module is further divided into sub-modules, which are provided below

Submodule 1: Database Module

FE-1: The Server database stores all the details of registered devices with availability

FE-2: Maintain the global model which results from the combination of all model updates and after each update, update the global model too

FE-3: Global Data-to-be-Trained (pre-processed)

FE-4: Stores all the task assignment logs of the devices, including current progress and completion

Submodule 2: Task Splitting and Segmentation Module

FE-1: Analyze global models and check for improvement

FE-2: Tasks are divided into small chunks, which further distributed among devices

Submodule 3: Network & Session Management Module

FE-1: Device availability monitoring and identifying faults by checking network and device status over time

FE-2: Load balancing by allocating tasks efficiently to the devices according to their memory, battery, network capability

Submodule 4: Task Allocator

FE-1: Task assignment and allocation to the devices according to their memory and battery capability

FE-2: Task reallocation to other devices in case of delay or no response

Submodule 5: Model Aggregation & Update Module

FE-1: Collecting trained models update from the registered devices

FE-2: Assign weight to trained model data

FE-3: With aggregation weights, update the global model.

FE-4: Preparing a model update and redistributing among the devices for training

3 Requirement Analysis

This section contains information about users' classes and characteristics, the technique used to gather the requirements and list of all requirements of the Fractal

3.1 User classes and characteristics

The user classes and their characteristics that the system will support are provided below

Table 3 User classes and characteristics

User Class	Description
User/Contributor	These individuals own mobile devices (Android) and agree to contribute their mobile computational resources. They install the application on their devices and contribute their resources during idle time. The application performs tasks securely in the background
Admin	Admin is responsible for managing the system updates, maintenance, allocation, splitting, and aggregation modules. Admin also oversees the performance and security of the system and reviews logs.
AI/ML companies, enterprises, and research institutes	AI/ML companies, enterprises, and research institutes provide computational jobs to be executed on a decentralized distributed computation system. These individuals submit model training tasks

3.2 Requirement Identifying Technique

The system is based on a queue-based asynchronous system where most of the functionalities are performed without user involvement. Due to this reason, the requirement identification technique used for the Decentralized distributed computation harvesting system is the Event response table. The Event response table for the system is provided below:

Table 4 Event response table

Event	System State	System Response
The user signs up, powers on the device, and connects to a network	The device is not registered	The device registered with the server using stats like ID, RAM, battery, etc.
The server sends a ping to the device	The device is currently online	Respond to server ping with the status
Device available resources	The device is ready for training	Send task request
Task assigned	The device is idle	The device receives data, preprocess it, and formats it for training
Data has been preprocessed and is available	The device is idle and ready for training	With TensorFlow Lite/ PyTorch Mobile, start local training and continuously monitor performance
Local training complete	Waiting for the transfer weights to the server	Transmit weights and biases to the server
Receive weight and biases from other devices	Model aggregation is in waiting	Save weight, aggregate, and then update the global model
Global model updated	The device is idle or available for the new task	Redistribute the model to available devices
New training job for the task queue	Device available	Check devices in the database, and their resources, and allocate tasks

Task failed	Timeout	Recheck devices in the database, their resources, and allocate tasks
Device offline	Task incomplete	Terminate the training, or reassign the task to another device
Task assigned to all available	All devices are updated	Save logs in a database, and continuously monitor contributions
Device low battery	The task is in running	Pause training, reassign the task to another device
No response from the device	Task incomplete	Ping the device, terminate training, and reassign the task to another device

3.3 Functional Requirements

This section describes the functional requirements of the Decentralized distributed computational harvesting system. The functional requirements are provided below

3.3.1 Functional Requirement 1

Identifier	FR-1
Title	Queue-based/Message-based asynchronous job
Requirement	The system shall be able to perform queue-based/message-based asynchronous tasks
Source	scope, module
Rationale	Allow to perform the primary function without waiting for other tasks to complete
Dependencies	No
Priority	High

Table 5: FR1

3.3.2 Functional Requirement 2

Identifier	FR-2
Title	User Registration
Requirement	The user shall be able to register using a registered email and password to access the system
Source	context diagram, scope, network module
Rationale	Link devices to an authenticated user
Business Rule	Must use Firebase Authentication
Dependencies	No
Priority	High

Table 6 FR2

3.3.3 Functional Requirement 3

Identifier	FR-3
Title	User login
Requirement	The user shall be able to log in using a valid email and password to access the system
Source	context diagram, scope, network module
Rationale	No repeated registration
Dependencies	FR-2
Priority	High

Table 7: FR3

3.3.4 Functional Requirement 4

Identifier	FR-4
Title	Un-registration
Requirement	The user shall be able to unregister the device by providing a problem description and screenshots
Source	scope
Rationale	User control
Dependencies	FR-2
Priority	High

Table 8: FR4

3.3.5 Functional Requirement 5

Identifier	FR-5
Title	Active model training
Requirement	The user shall be able to activate model training using the button
Source	Mockups
Rationale	Allow user mobile resources participation in training
Business Rule	The device must be registered and online. The battery must be above the selected limit. FR- 23
Dependencies	FR-2
Priority	High

Table 9: FR5

3.3.6 Functional Requirement 6

Identifier	FR-6
Title	Inactive model training
Requirement	The user shall be able to inactivate model training using the button
Source	Mockups
Rationale	Allow the user to restrict mobile resources participation for training
Business Rule	The device must be registered and online
Dependencies	FR-5
Priority	High

Table 10: FR6

3.3.7 Functional Requirement 7

Identifier	FR-7
Title	Device status
Requirement	The system shall ping devices to check the availability status
Source	network module
Rationale	Maintain device availability
Business Rule	The device must be registered and online
Dependencies	Fr-2
Priority	High

Table 11: FR7

3.3.8 Functional Requirement 8

Identifier	FR-8
Title	Task allocation
Requirement	The system shall assign tasks to devices based on their resources and availability
Source	Server-side module
Rationale	Work distribution
Business Rule	The user device must meet the resource standards
Dependencies	FR-5
Priority	High

Table 12: FR8

3.3.9 Functional Requirement 9

Identifier	FR-9
Title	Task reallocation
Requirement	The system shall reallocate tasks to devices in case of no response or timeout
Source	Server-side module
Rationale	Reduce delay, ensure fault tolerance
Business Rule	Only in case of no response or delay The user device must meet the standards
Dependencies	FR-5, FR-8, FR-22
Priority	High

Table 13: FR9

3.3.10 Functional Requirement 10

Identifier	FR-10
Title	Data reception
Requirement	The system shall be able to receive data from the server
Source	Mobile-side module, event response table
Rationale	Device access to data
Dependencies	FR-5, FR-8
Priority	High

Table 14: FR10

3.3.11 Functional Requirement 11

Identifier	FR-11
Title	Format
Requirement	The system shall be able to format the received data into a compatible format
Source	Mobile-side module, event response table
Rationale	Ensure validity for the required format
Dependencies	FR-5, FR-8, FR-10
Priority	High

Table 15: FR11

3.3.12 Functional Requirement 12

Identifier	FR-12
Title	Model training
Requirement	The system shall be able to train a model using TensorFlow Lite/PyTorch Mobile, saving weight and logs
Source	Mobile-side module, event response table
Rationale	Scope distributed training
Business Rule	Meet training constraints like in FR-5
Dependencies	FR-11
Priority	High

Table 16: FR12

3.3.13 Functional Requirement 13

Identifier	FR-13
Title	Monitoring resources
Requirement	The system shall monitor device resources continuously for the task distribution
Source	Mobile-side module (Resource Management), event response table
Rationale	Ensure availability and task distribution
Dependencies	FR-5
Priority	High

Table 17: FR13

3.3.14 Functional Requirement 14

Identifier	FR-14
Title	Transmit updates
Requirement	The system shall be able to transmit weights and logs asynchronously to the server after local training
Source	event response table
Rationale	Model training needs
Dependencies	FR-12
Priority	High

Table 18: FR14

3.3.15 Functional Requirement 15

Identifier	FR-15
Title	Training preferences by user
Requirement	The user shall be able to set a preference for training, such as on charging, Wi-Fi, and cellular
Source	Mockups
Rationale	User control over device insights
Business Rule	Training doesn't start unless preferences are satisfied
Dependencies	FR-3
Priority	Medium

Table 19: FR15

3.3.16 Functional Requirement 16

Identifier	FR-16
Title	Ongoing training summary
Requirement	The system shall display an ongoing training summary, such as performance, time left, and temperature
Source	Mockups
Rationale	User understanding
Dependencies	FR-5
Priority	Medium

Table 20: FR16

3.3.17 Functional Requirement 17

Identifier	FR-17
Title	Heartbeat
Requirement	The system shall display a heartbeat, ensuring the device is responsive and online
Source	Mockups
Rationale	Ensure device online
Dependencies	FR-5
Priority	Medium

Table 21: FR17

3.3.18 Functional Requirement 18

Identifier	FR-18
Title	Usage Insights
Requirement	The system shall display usage insights such as RAM, ROM, and Temperature with the heartbeat
Source	Mockups
Rationale	User understanding
Dependencies	FR-5, FR-17
Priority	Medium

Table 22: FR18

3.3.19 Functional Requirement 19

Identifier	FR-19
Title	Model aggregation
Requirement	The system shall aggregate the received weighs and biases, update the global model
Source	event response table, Server-side module
Rationale	Consistent model updates
Business Rule	Aggregation after receiving updates exceeds the limit
Dependencies	FR-14, FR-5
Priority	High

Table 23: FR19

3.3.20 Functional Requirement 20

Identifier	FR-20
Title	Global update
Requirement	The system shall apply a version to the global update and redistribute it to devices for accuracy
Source	event response table, Server-side module
Rationale	Model accuracy
Dependencies	FR-19, FR-5
Priority	High

Table 24: FR20

3.3.21 Functional Requirement 21

Identifier	FR-21
Title	Optimize
Requirement	The system shall analyze and optimize the training load when the user clicks on the “Optimize” button
Source	Mockup
Rationale	Enhance model training
Business Rule	Optimization doesn’t work if it exceeds the constraints, such as “battery less than” is selected, etc.
Dependencies	FR-5, FR-13
Priority	High

Table 25: FR21

3.3.22 Functional Requirement 22

Identifier	FR-22
Title	Battery level for training
Requirement	The system shall allow a user to select a charging percentage such that the device starts training when charging is above the selected percentage
Source	Mockup
Rationale	Optimize battery health
Business Rule	Training doesn't start till the battery is below that selected percentage
Dependencies	FR-3
Priority	High

Table 26: FR22

3.3.23 Functional Requirement 23

Identifier	FR-23
Title	Registered info
Requirement	The system shall display registered information about the user, device, hardware, software, and network
Source	Mockup
Rationale	Ensure Details of the registered device
Dependencies	FR-3
Priority	Medium

Table 27: FR23

3.4 Non-Functional Requirements

This section describes the non-functional requirements of the Decentralized distributed computational harvesting system

3.4.1 Reliability

RE-1: The system shall detect the inactive, hung device, device constraints, or task failure within 180 seconds

RE-2: The system shall pause the local training if the battery is less than the selected limit within 45 seconds

RE-3: The system shall reattempt task reassignment to the device leads in the event of failure for up to 300 seconds

RE-4: The system shall reallocate 95% of tasks successfully to an alternate device within 180 seconds in case of an inactive, hung device or task failure

RE-5: The system shall achieve a Mean time between failures (MTBF) of 60 hours under normal load and conditions (device insights)

3.4.2 Usability

USE-1: The system shall allow a user to complete registration without prior training within 1 minute

USE-2: The system shall allow a user to active or inactive training within 2 taps

USE-3: The system shall provide self-explanatory and intuitive user experience and interaction

USE-4: The system shall provide an onboarding tutorial for the first user to go through the features within 140 seconds

USE-5: The system shall allow a user to set training preferences (Wi-Fi, cellular, on charging, overnight) within a single screen

3.4.3 Performance

PER-1: The system shall assign tasks to available device resources within 45 seconds after checking resources

PER-2: The system shall only consume RAM and ROM within 60%-100% of a mobile device

PER-3: The system shall initiate model training on devices within 60 seconds after task allocation

PER-4: The system shall transmit weights and logs to the server within 20 seconds after server handshake

PER-5: The system shall accommodate 15 updates within 3 minutes (model aggregation module)

3.4.4 Security

SEC-1: The system shall perform authentication by Firebase Authentication using email/password

SEC-2: The system shall allow participation only of registered users in model training

SEC-3: The system shall time out the session of inactive or unresponsive devices after 180 seconds

SEC-4: The system shall allow un-registration by requiring the terms & conditions consent of a user before proceeding

3.5 External Interface Requirements

This section provides information to ensure that the Decentralized Distributed Computation Harvesting System will communicate properly with users and with external hardware or software elements. External interface requirements are provided below

3.5.1 User Interfaces Requirements

UIR-1: The application shall conform to Google Material Design, W3C web content accessibility (WCAG 2.1), and IEEE Software engineering standards

UIR-2: The application shall follow the bottom navigation bar with three tabs: Device, Home, and Model

UIR-3: The application shall support two color themes: Black (with light text) and White (with black text) per WCAG 2.1 standard

UIR-4: The application shall use all content, headings, and user info centered-aligned or per design aesthetics

UIR-5: The application shall provide a real-time graph for usage insights (RAM, ROM, CPU, TEMP)

UIR-6: The application shall use consistent icons, padding, and fonts for all components

UIR-7: The application shall support devices with a minimum screen resolution of 360 x 640 density independent pixels

UIR-8: The application shall be optimized for portrait orientation

UIR-9: The application shall support a responsive layout, ranging in various Android screen sizes from 360 x 540 to 1440 x 3200

UIR-10: The application shall provide inline messaging for success/error states

UIR-11: The application icons shall be vector-based based consistent across both themes

UIR-12: The application button labels shall be aligned according to the form-based progression standards

3.5.2 Software interfaces

SI-1: Firebase Fire store

SI-1.1: The system shall store device registration, task assigned data using Firebase Fire store as a NoSQL database

SI-1.2: The system shall fetch data for training preferences using the Firebase Fire store NoSQL database

SI-2: Firebase Authentication

SI-2.1: The system shall authenticate a user by email/password rule using Firebase Authentication

SI-2.2: The system shall validate “Active” training using Firebase Authentication

SI-3: Operating system of Android

SI-3.1: The system shall use Android OS to monitor device insights while model training, such as RAM, ROM, CPU, temperature, etc.

SI-4: TensorFlow Lite/PyTorch Mobile

SI-4.1: The system shall execute local training on the device using TensorFlow Lite/PyTorch Mobile

SI-5: Firebase Cloud Function (Server logic, API)

SI-5.1: The system shall use Firebase Cloud Function to handle device ping, aggregation, transfer weights/biases, model updates, and session timeout

3.5.3 Communications interfaces

CI-1: The system shall transmit model updates to the server using the REST API

CI-2: The system shall send and receive pings from the devices using the Firebase cloud function

CI-3: The system shall use HTTPS to ensure secure data transmission between user devices and the server

CI-4: The system shall use Firebase Cloud Messaging for alerts and notifications

CI-5: The system shall use Firebase authentication to send a password reset email to the user when they click on “Forget Password”.

4 Design and Architecture

This section provides the architectural design and design model of the Fractal “a decentralized distributed computation harvesting system”

4.1 Architectural Design

Box and line diagram displays the business process concept of the application. It displays application components and their connection. In consideration of major subsystem diagram (server, UI, controlling (heartbeat, resource monitoring, local training), database) is too small so this diagram is relevant to components and connection. Fractal box and line diagram is provided below:

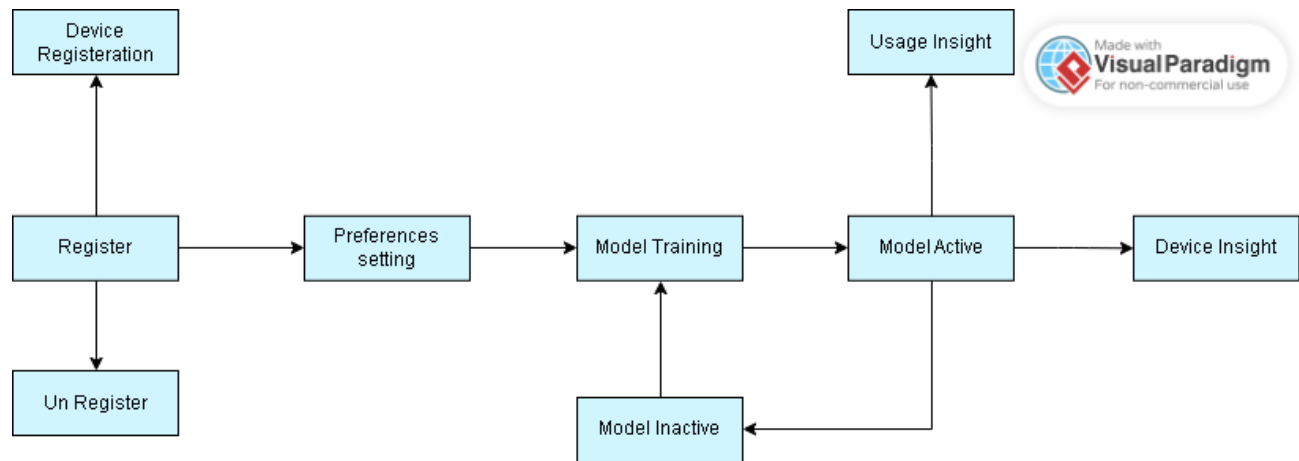
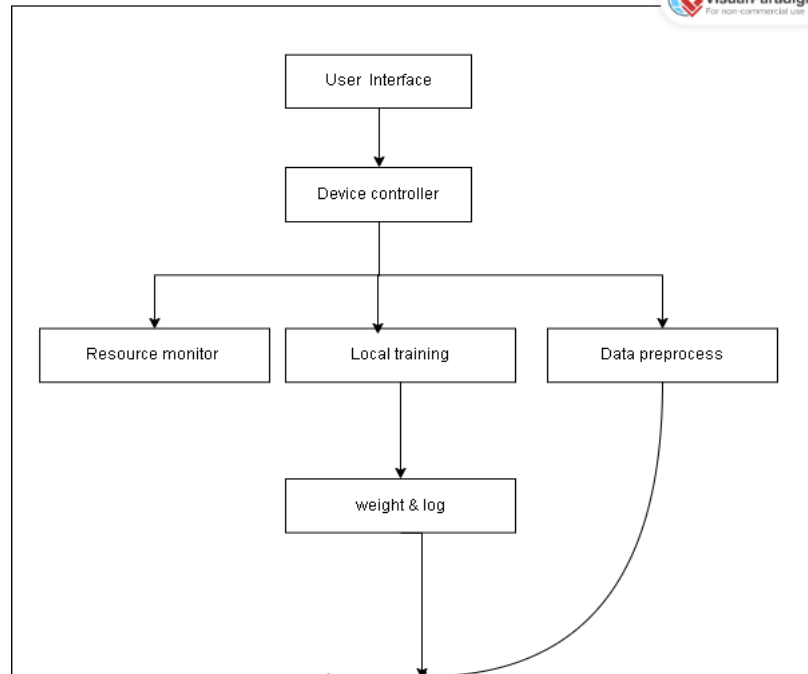


Figure 1: Box and line diagram

1- Client



Server

server to client

client to server

Communication

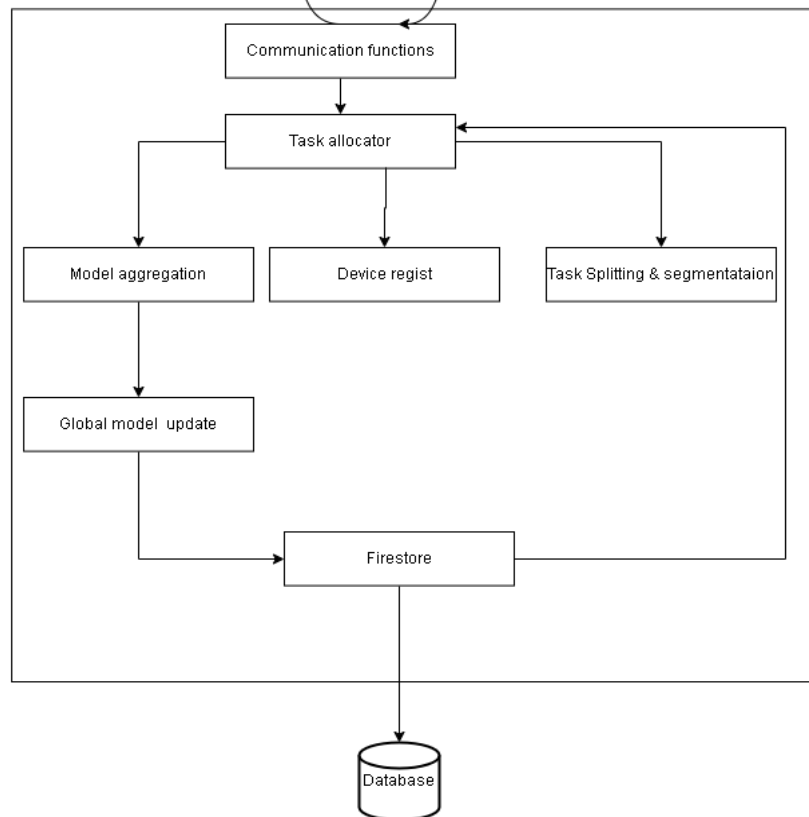


Figure 2: Architecture diagram

4.2 Design Models

The design methodology for the project “Fractal” is Object oriented programming methodology. OOP supports core of its concepts (polymorphism, encapsulation, polymorphism, modularity) those helps in reducing coupling (minimizing dependencies between entities) and high coupling entity focus on single responsibility. The Fractal project most of the functionalities are performed without user involvement. Due to this reason, the requirement identification technique used for the Decentralized distributed computation harvesting system is the Event response table. So instead of use-cases for diagrams we are according to each event and system response

Class Diagram (for whole system):

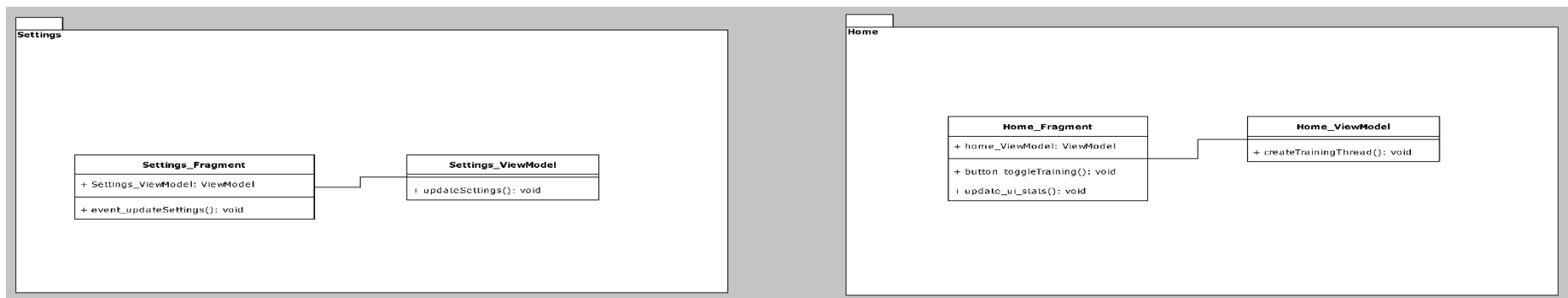


Figure 3 APP Interface 1

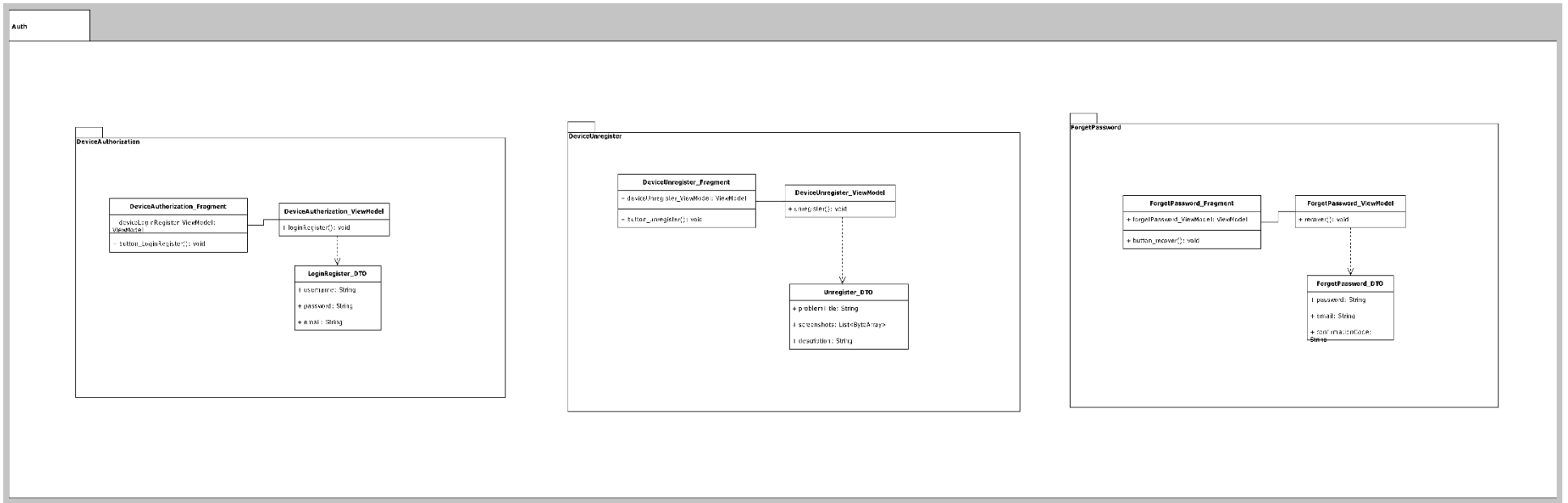


Figure 4 APP Interface 2

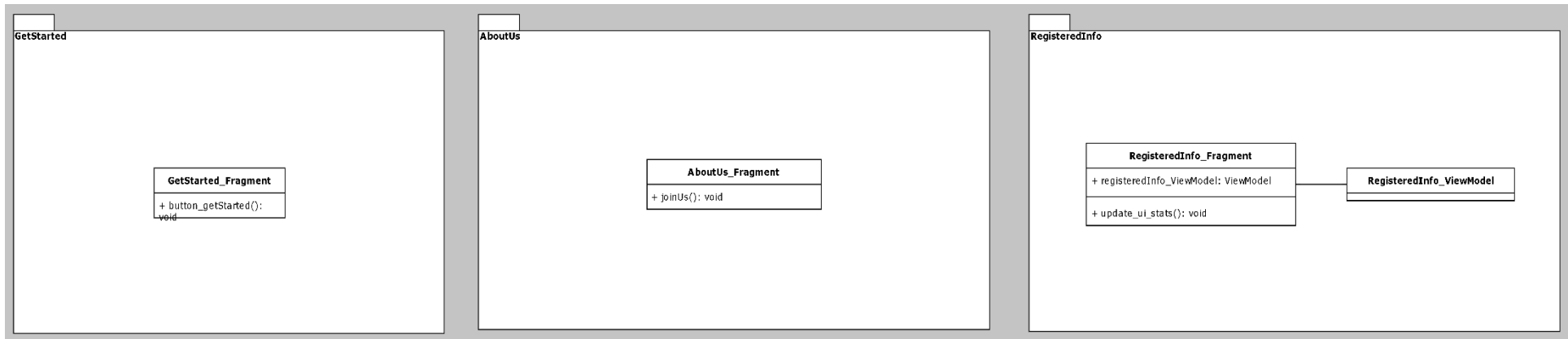


Figure 6 APP Interface 3

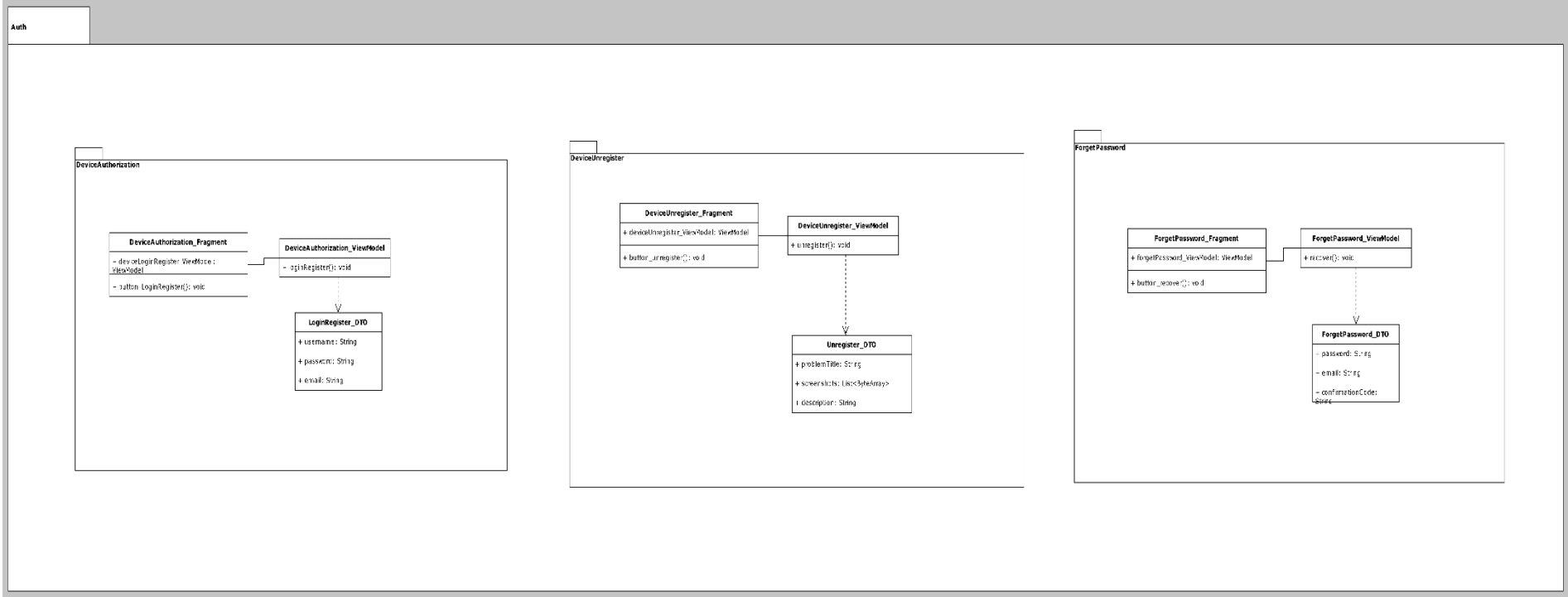


Figure 8 APP Interface 4

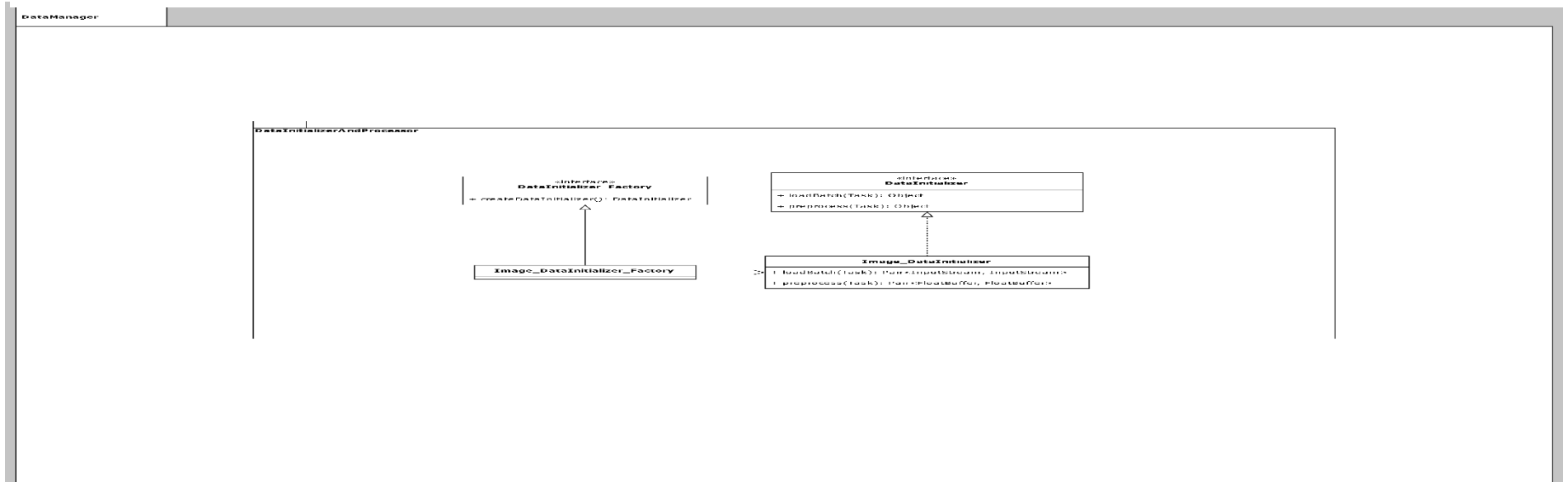


Figure 7 APP Interface 5

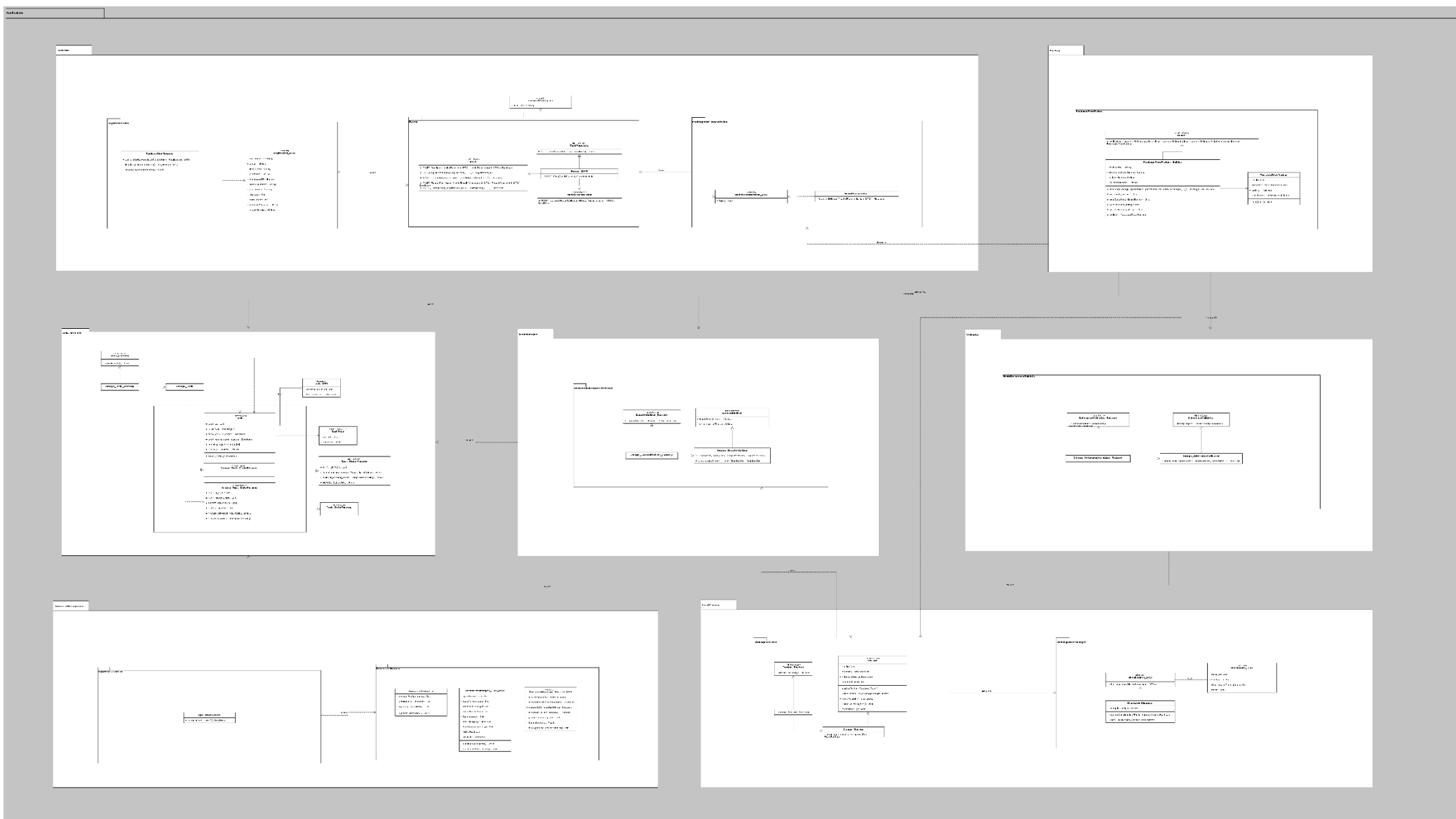


Figure 9 APP Interface complete

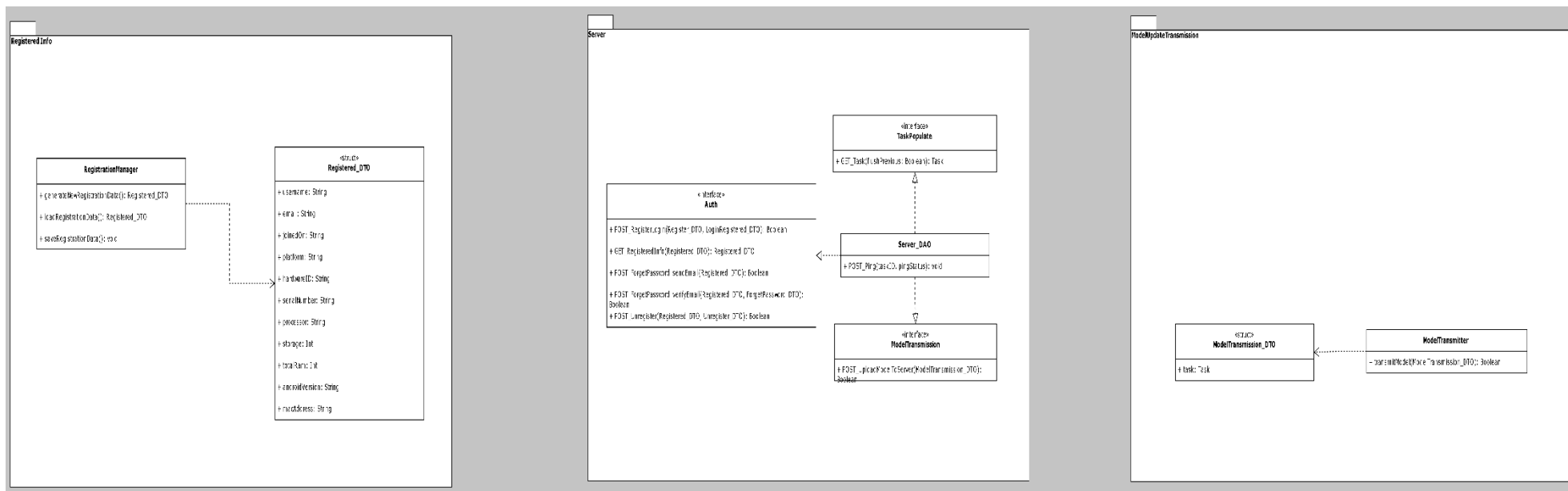


Figure 10 APP Interface

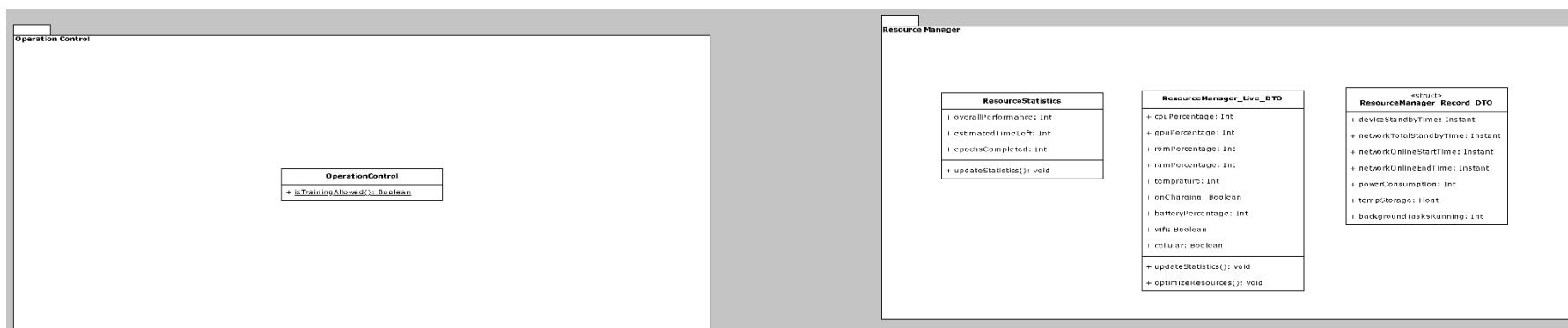


Figure 11 resource management

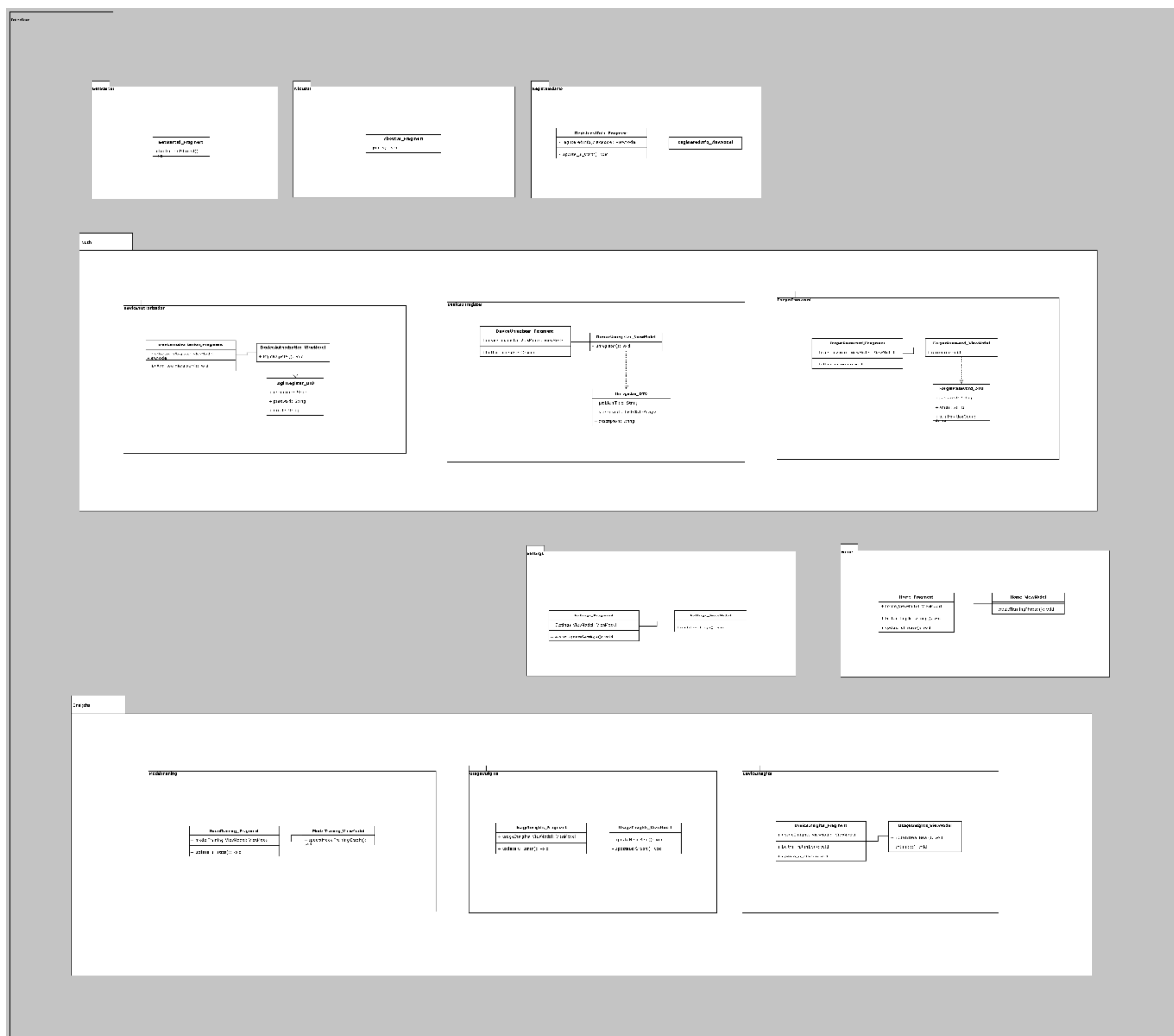


Figure 12 Interface

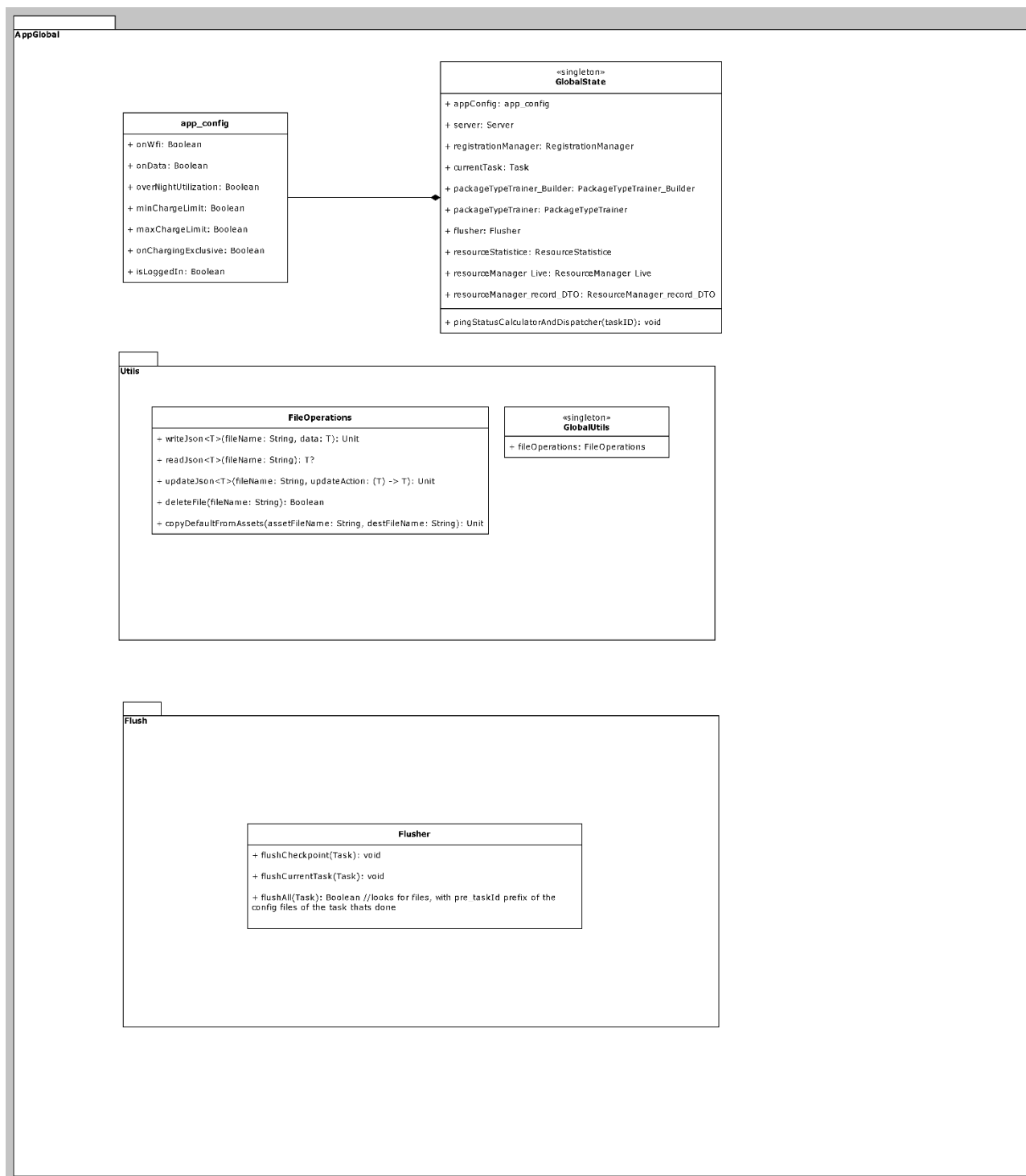


Figure 13 App Global

In this system most of the functionalities are performed without user involvement. Due to this reason, the requirement identification technique used for the Decentralized distributed computation harvesting system is the Event response table. The Event response table for the system is provided below:

Event	System State	System Response
The user signs up, powers on the device, and connects to a network	The device is not registered	The device registered with the server using stats like ID, RAM, battery, etc.
The server sends a ping to the device	The device is currently online	Respond to server ping with the status
Device available resources	The device is ready for training	Send task request
Task assigned	The device is idle	The device receives data, preprocess it, and formats it for training
Data has been preprocessed and is available	The device is idle and ready for training	With TensorFlow Lite/PyTorch Mobile, start local training and continuously monitor performance
Local training complete	Waiting for the transfer weights to the server	Transmit weights and biases to the server
Receive weight and biases from other devices	Model aggregation is in waiting	Save weight, aggregate, and then update the global model
Global model updated	The device is idle or available for the new task	Redistribute the model to available devices
New training job for the task	Device available	Check devices in the database, and their resources, and

queue		allocate tasks
Task failed	Timeout	Recheck devices in the database, their resources, and allocate tasks
Device offline	Task incomplete	Task failed and reassign the task to another device
Task assigned to all available	All devices are updated	Save logs in a database, and continuously monitor contributions
Device low battery	The task is in running	Pause training, reassign the task to another device

Table 28 Event response table

ER-1 Activity Diagram:

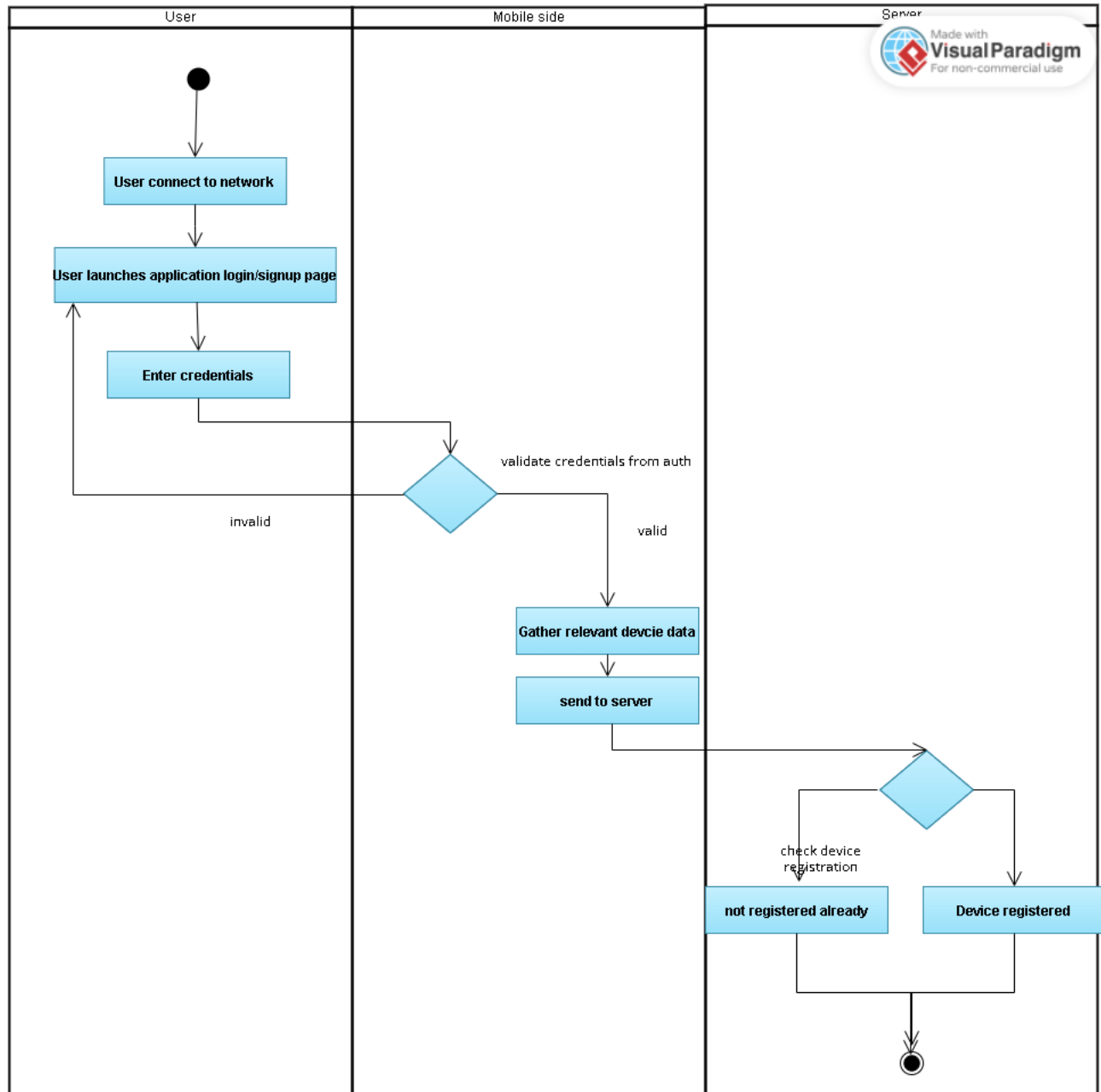


Figure 14- ER1-Activity diagram

ER-1 System Sequence diagram:

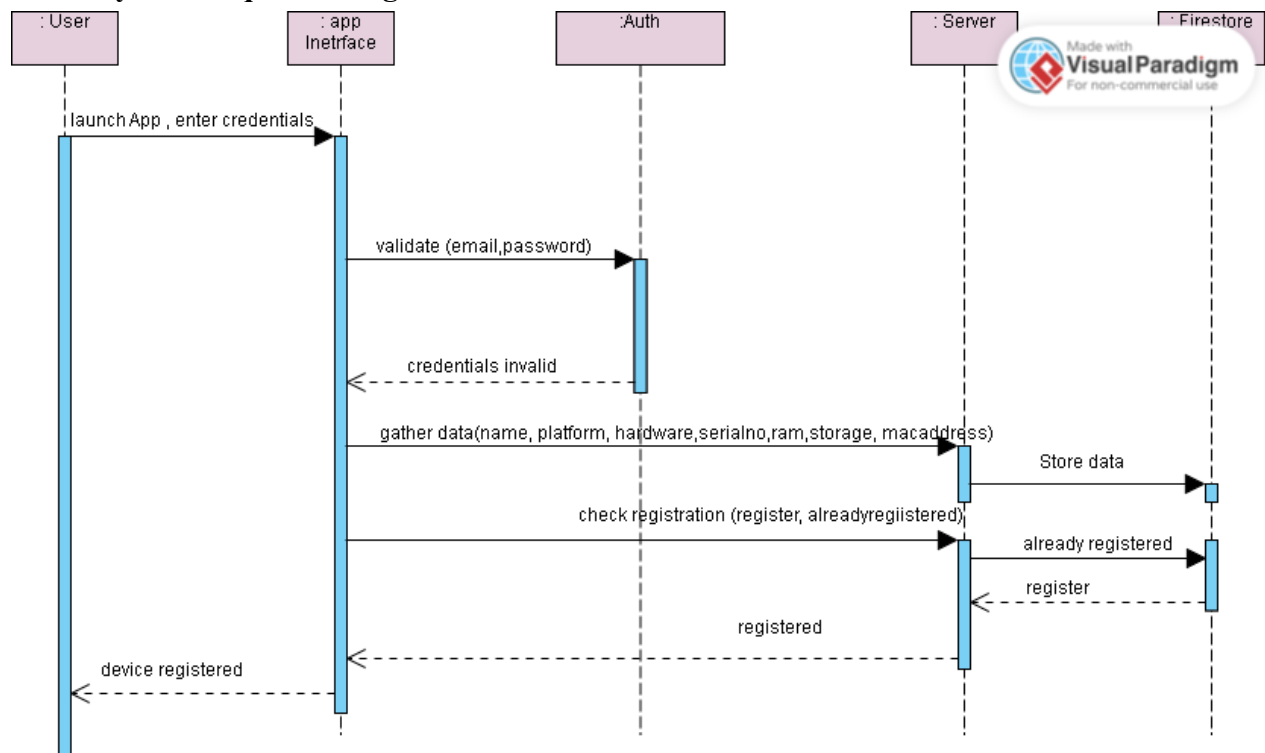


Figure 15-ER1 system sequence diagram

ER-1 State transition diagram:

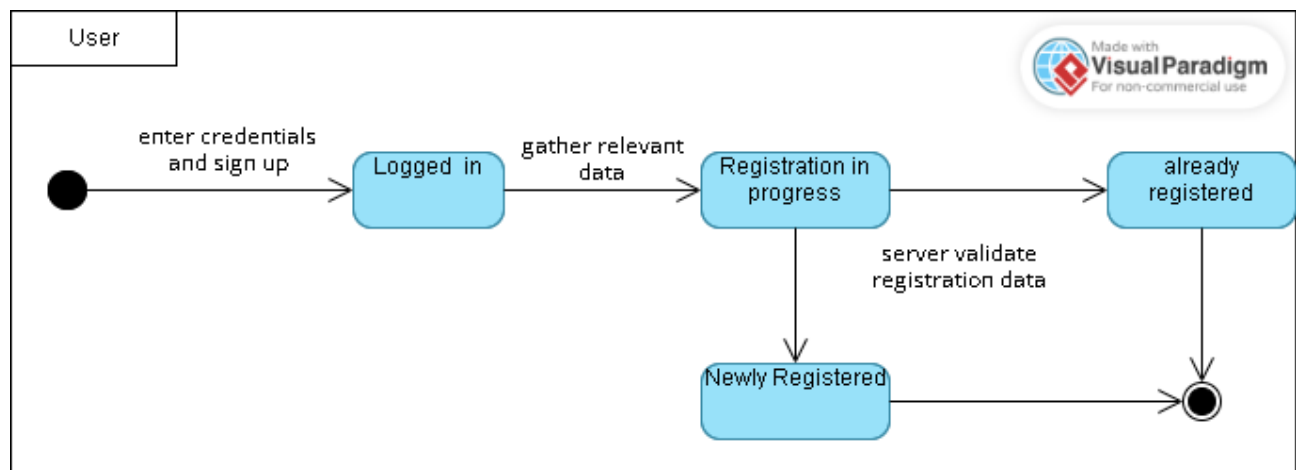


Figure 16-ER1 State transition diagram

ER-2 Activity diagram:

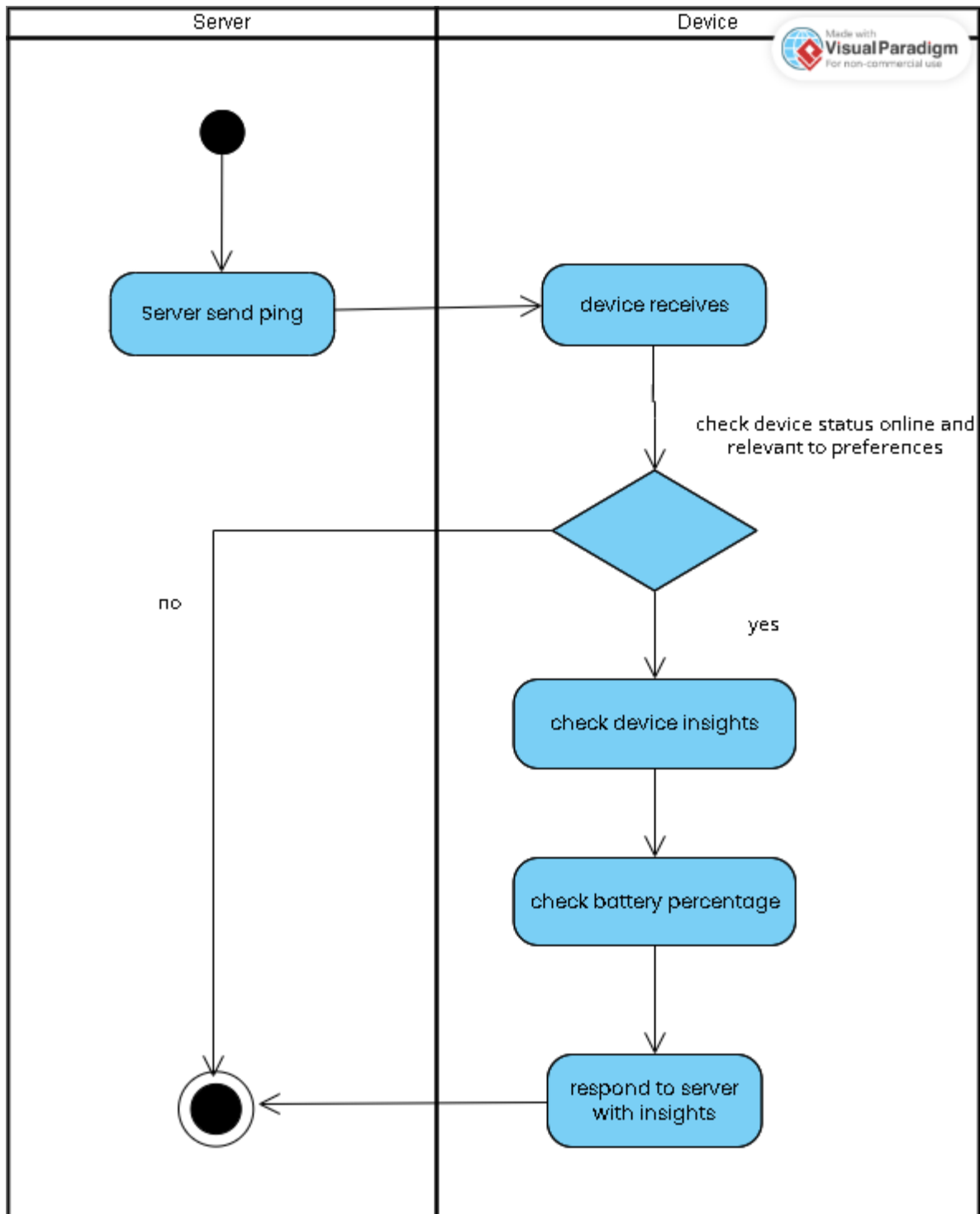


Figure 17- ER2 Activity diagram

ER-2 System Sequence diagram:

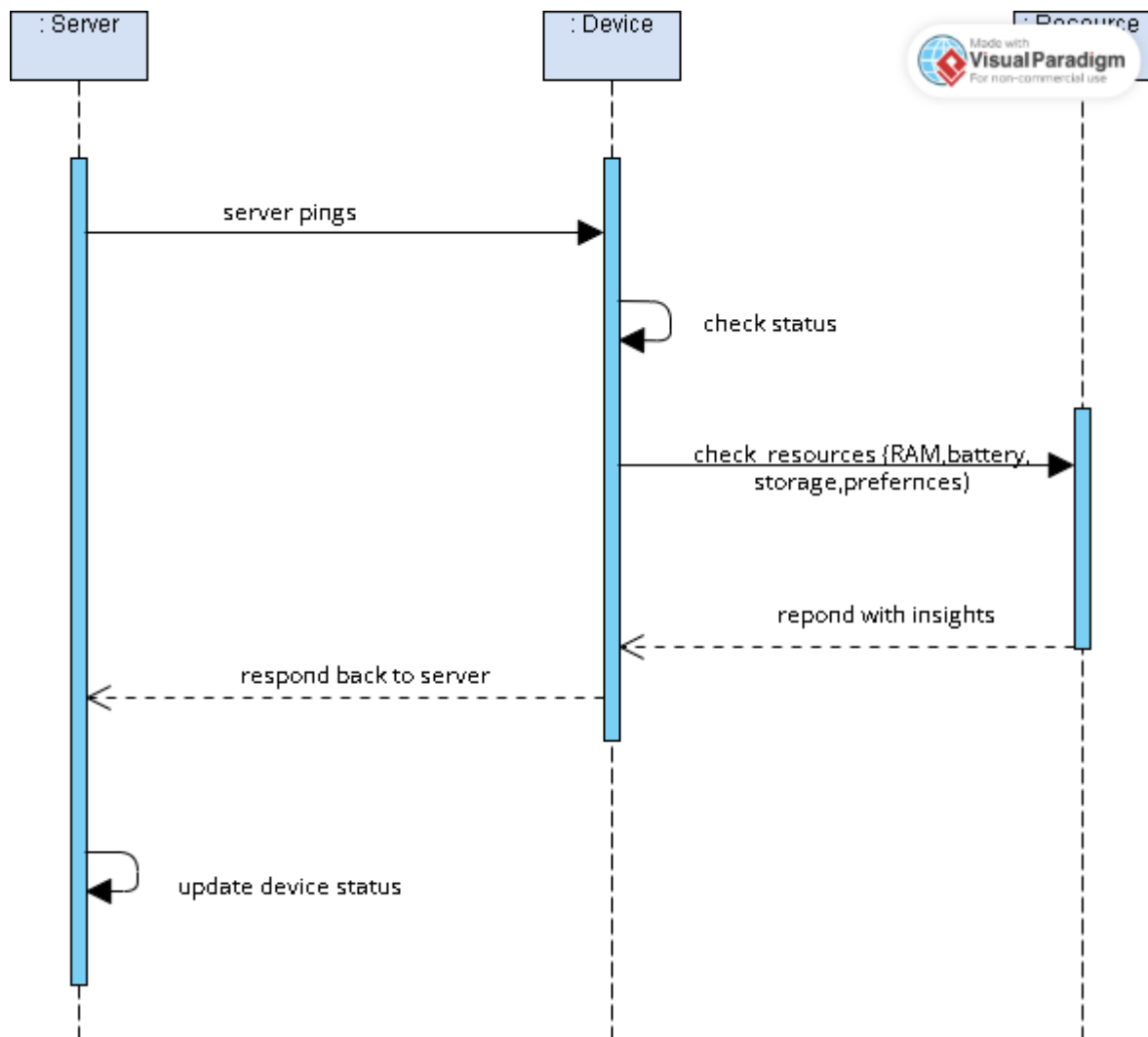


Figure 18 - ER2 System sequence diagram

ER-2 State transition diagram:

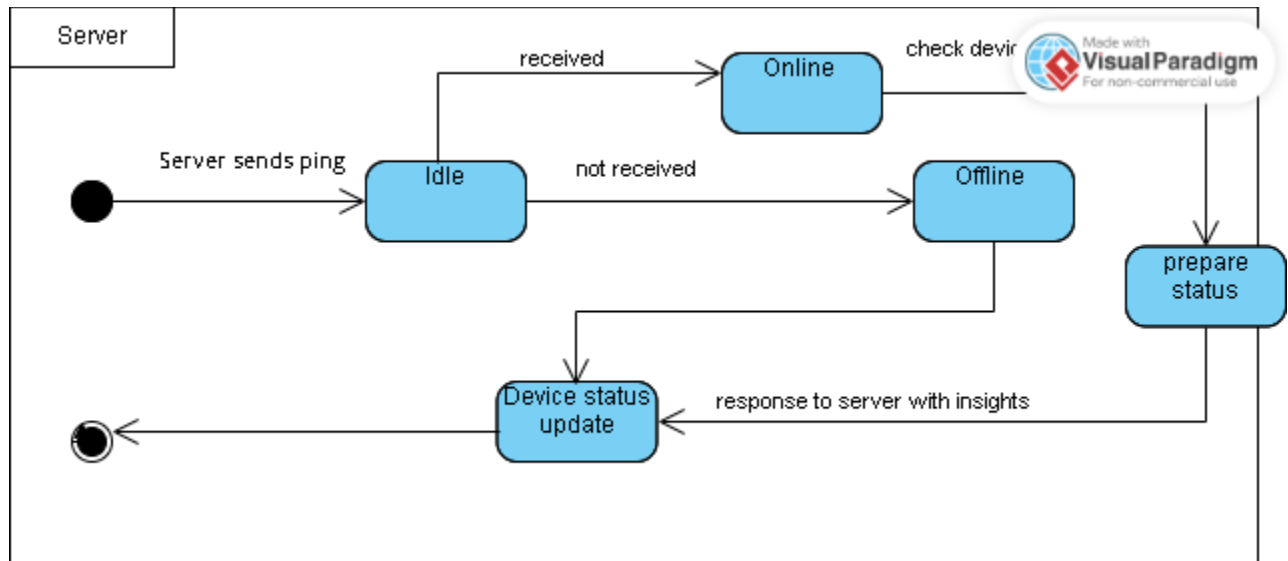


Figure 19 -ER2 State machine diagram

ER-3 Sequence diagram

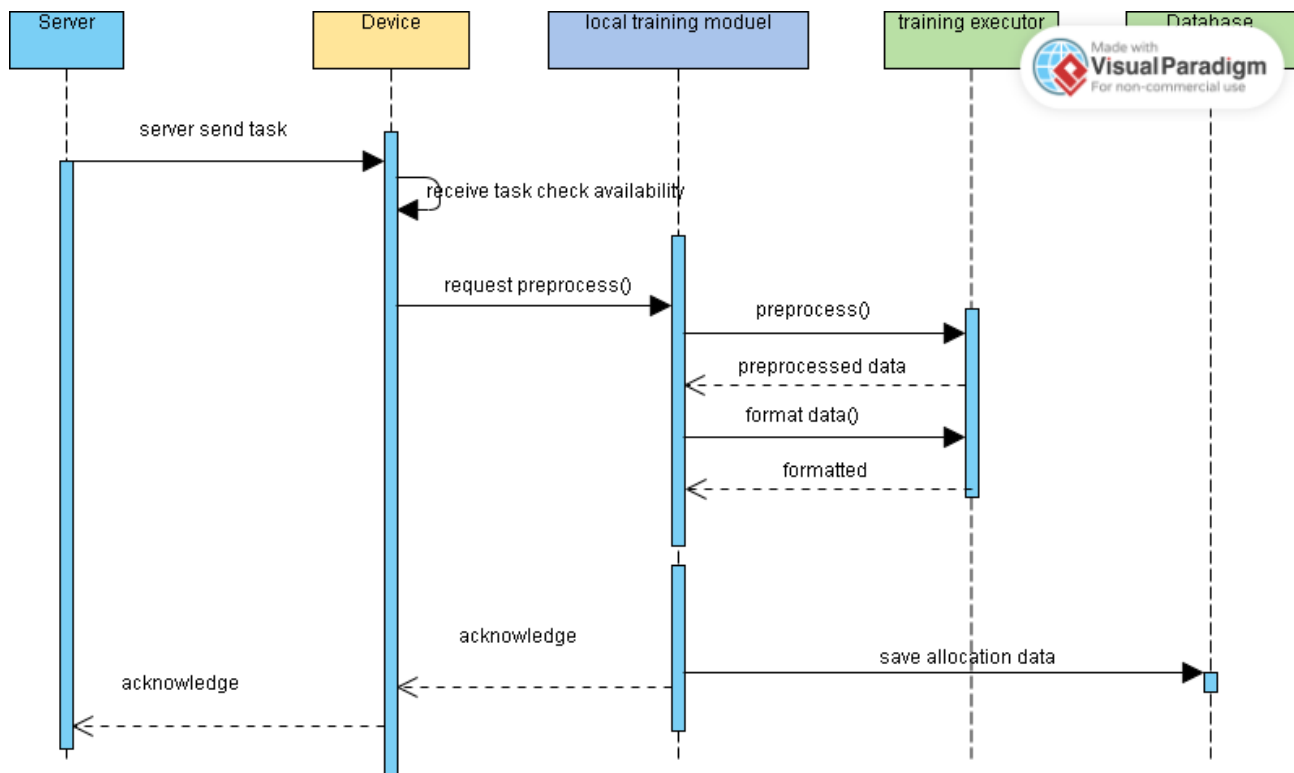


Figure 20 - ER3 Sequence diagram

ER-3 Activity diagram:

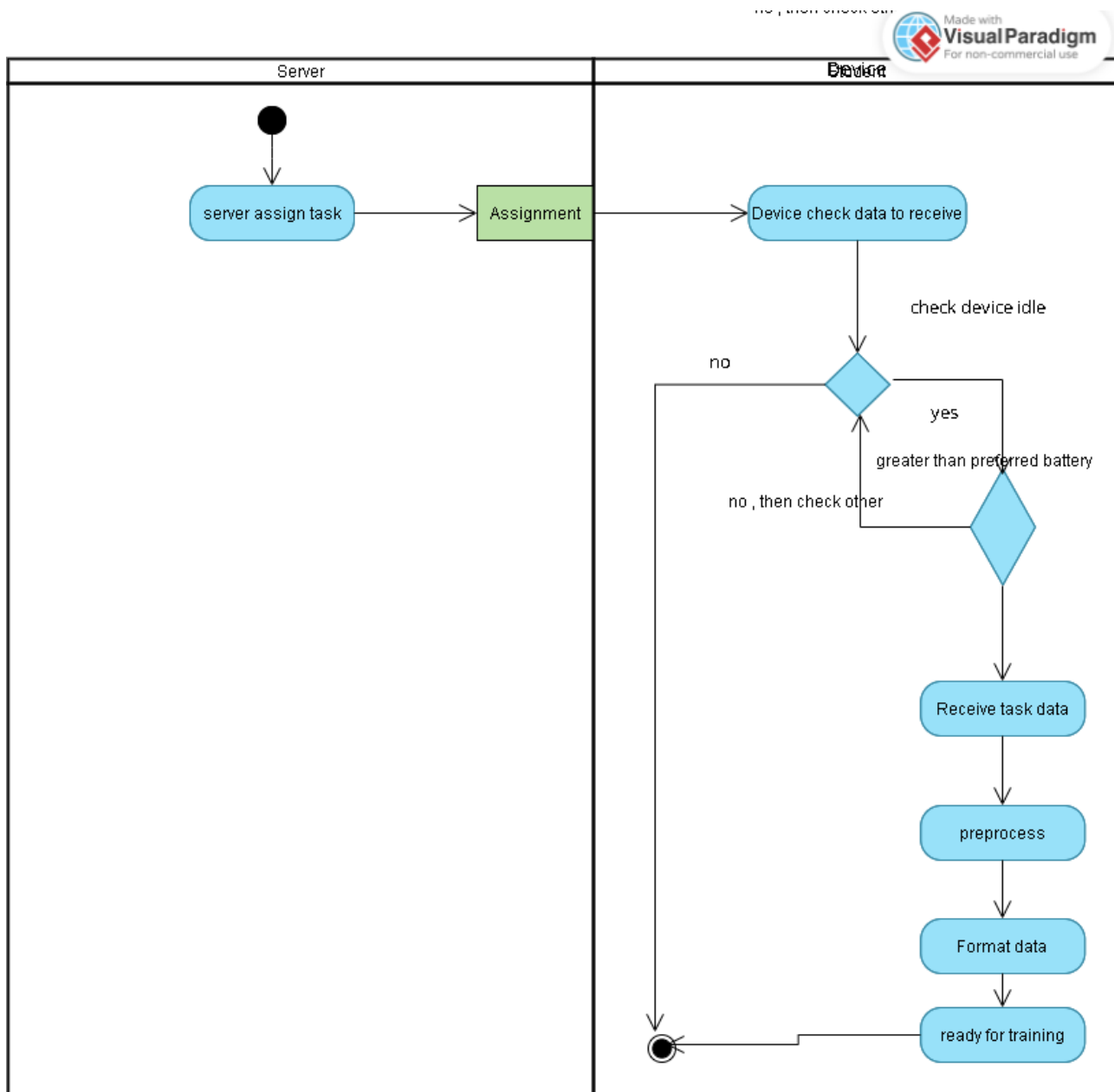


Figure 21 - ER3 Activity diagram

ER-3 State machine:

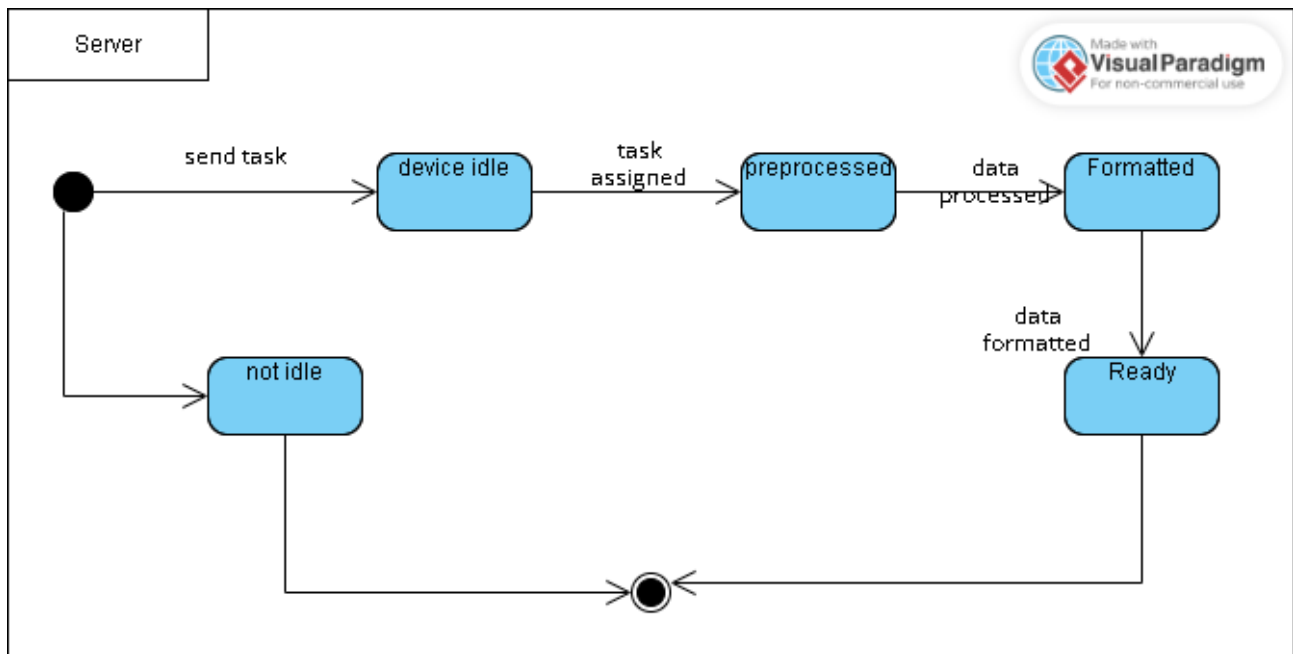


Figure 22 - ER3 State machine diagram

ER-4 State machine:

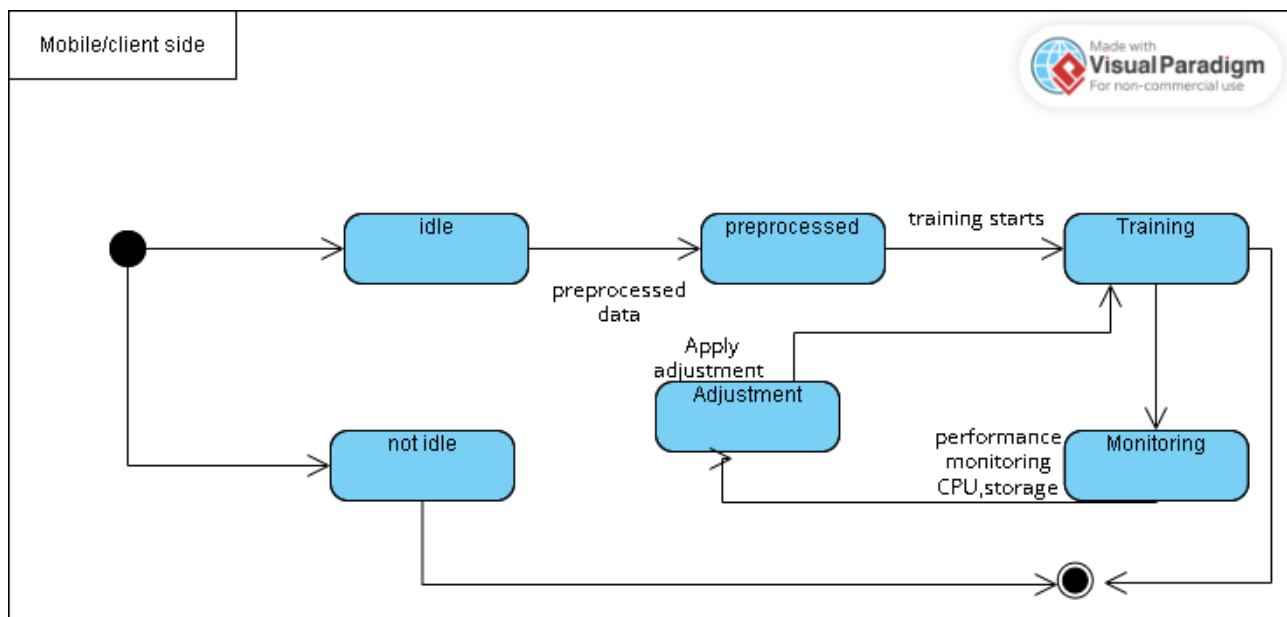


Figure 23- ER4 State machine diagram

ER-4 Activity Diagram:

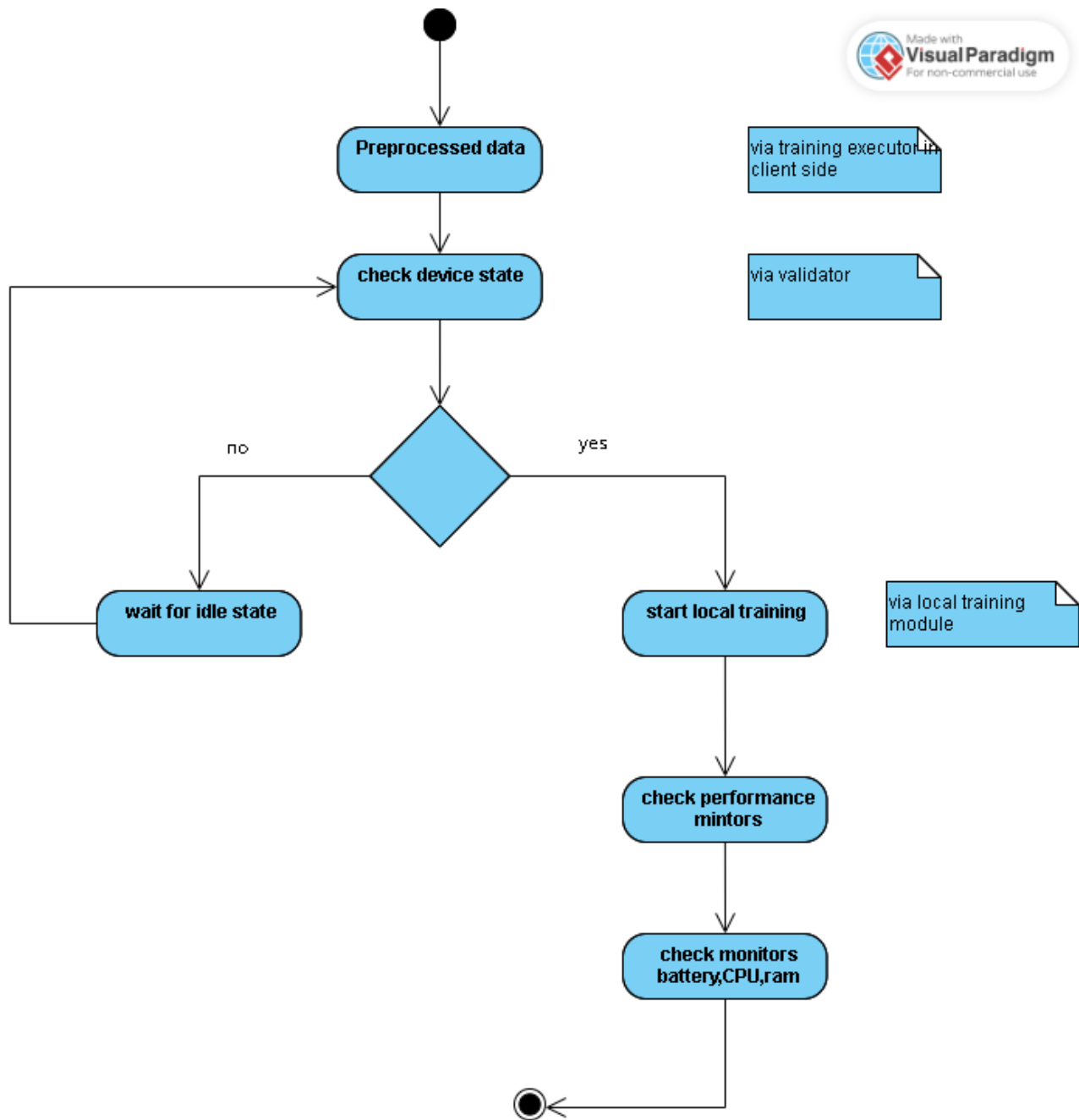


Figure 24- ER4 Activity diagram

ER-4 System Sequence:

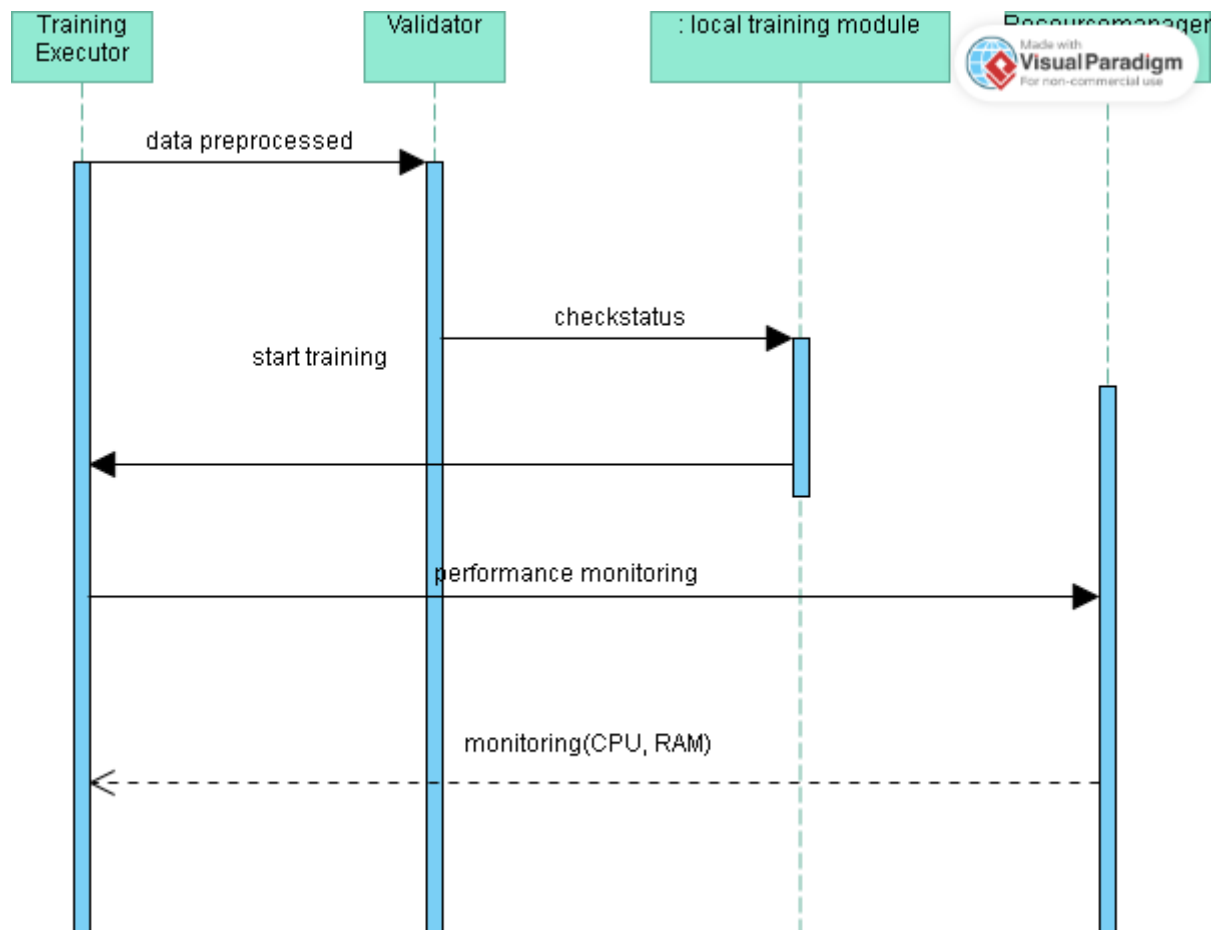


Figure 25- ER4 System sequence diagram

ER-5 State machine Diagram:

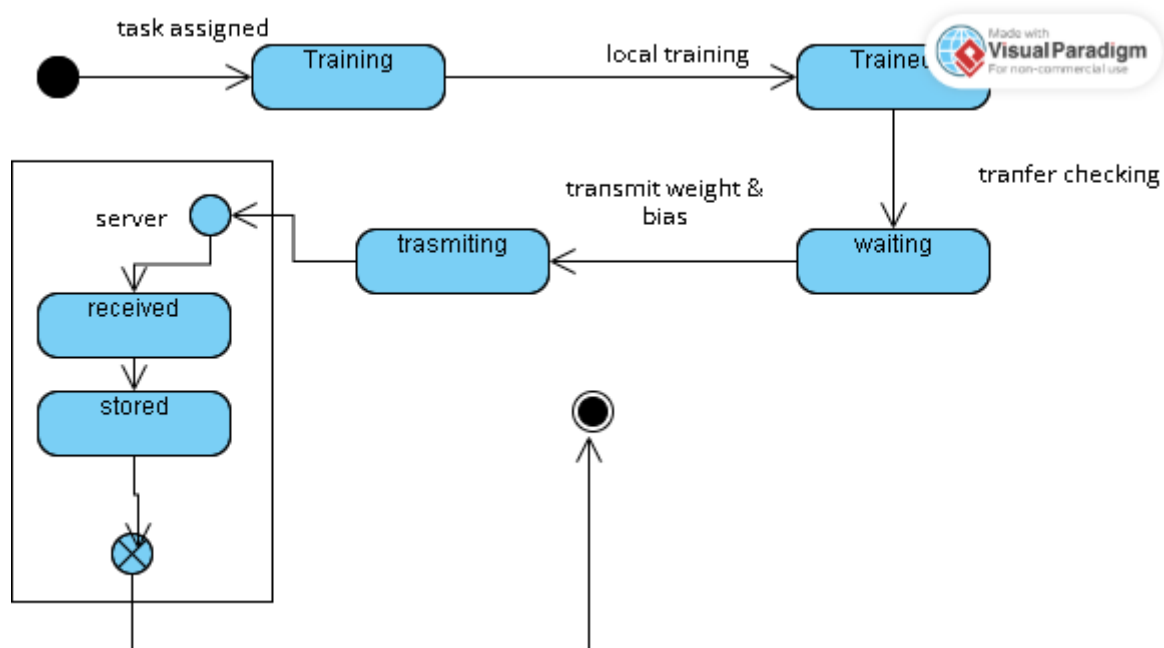


Figure 26- ER5 State machine diagram

ER-5 Activity Diagram:

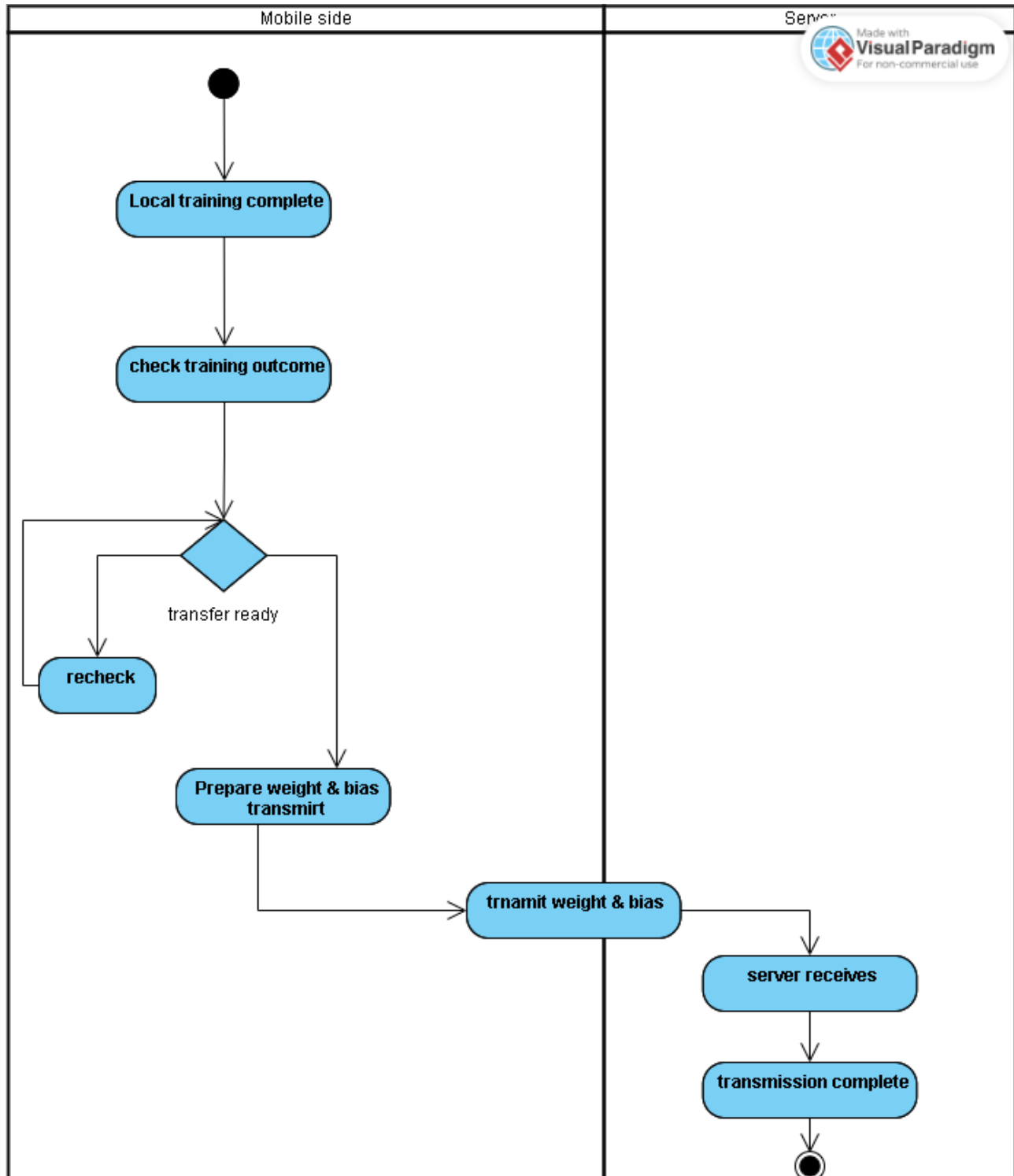


Figure 27- ER5 Activity diagram

ER-5 System Sequence diagram:

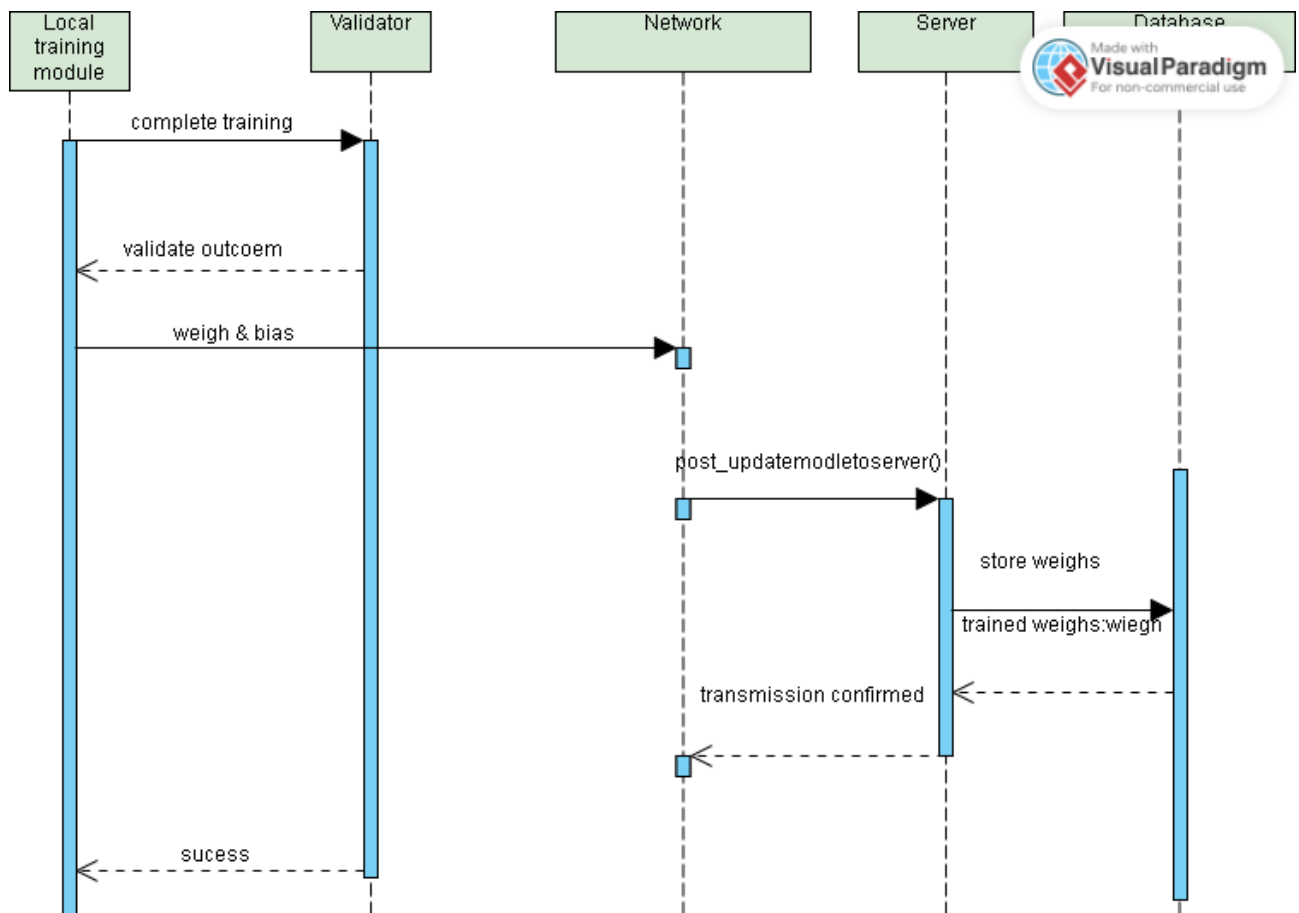


Figure 28- ER5 System sequence diagram

ER-6 Activity diagram:

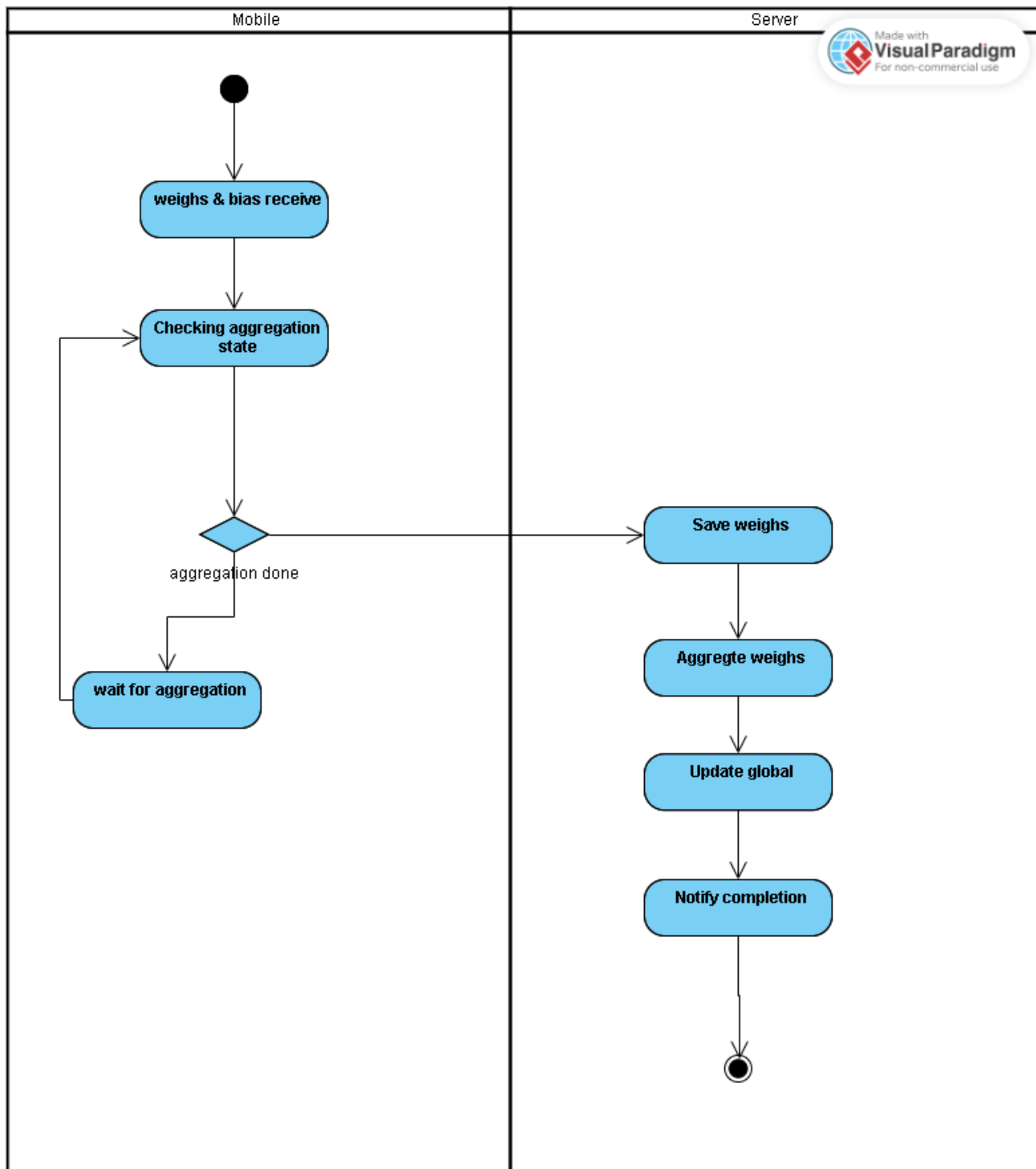


Figure 29- ER6 Activity diagram

ER-6 Sequence diagram:

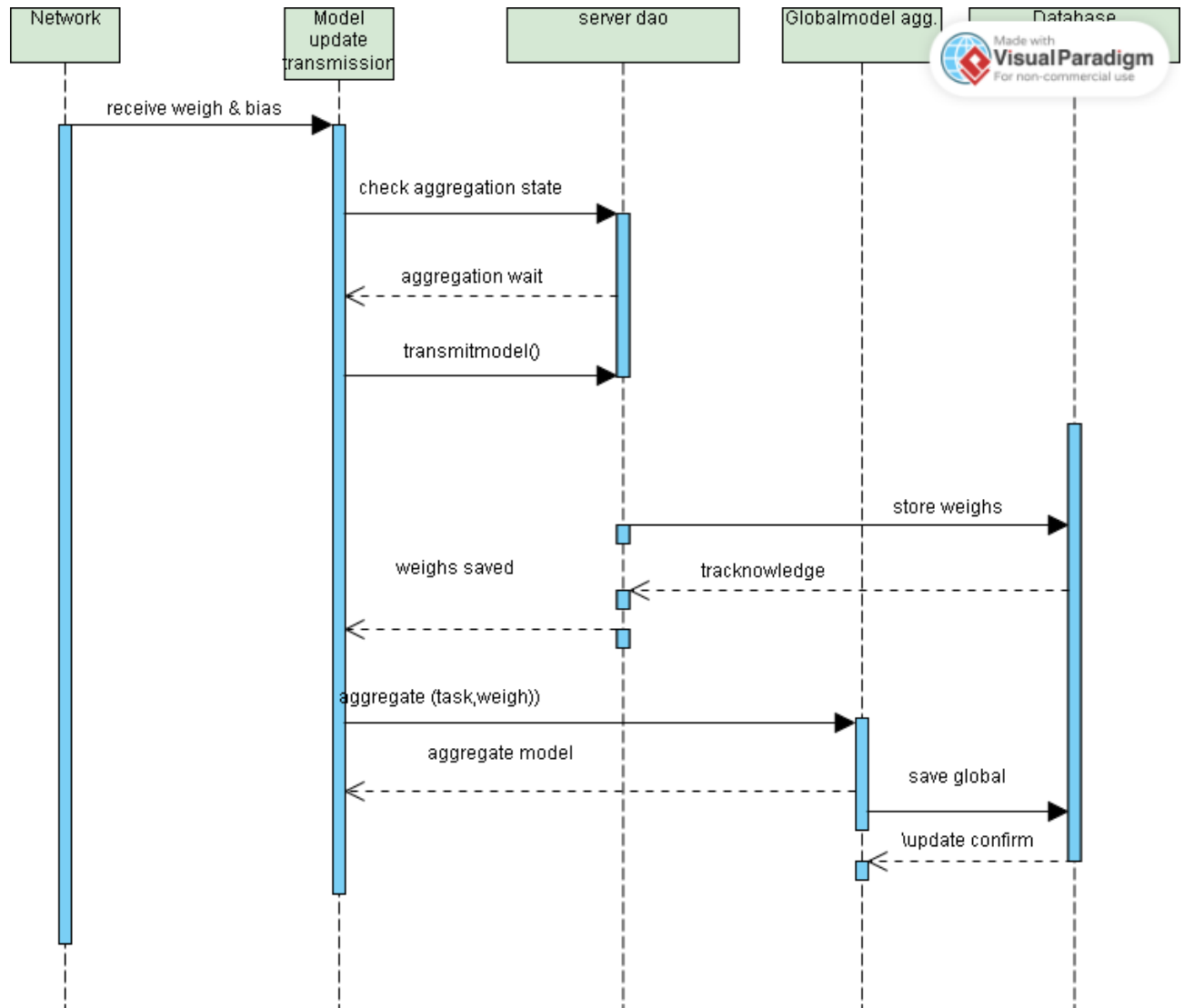


Figure 30- ER6 System sequence diagram

ER-7 Activity diagram:

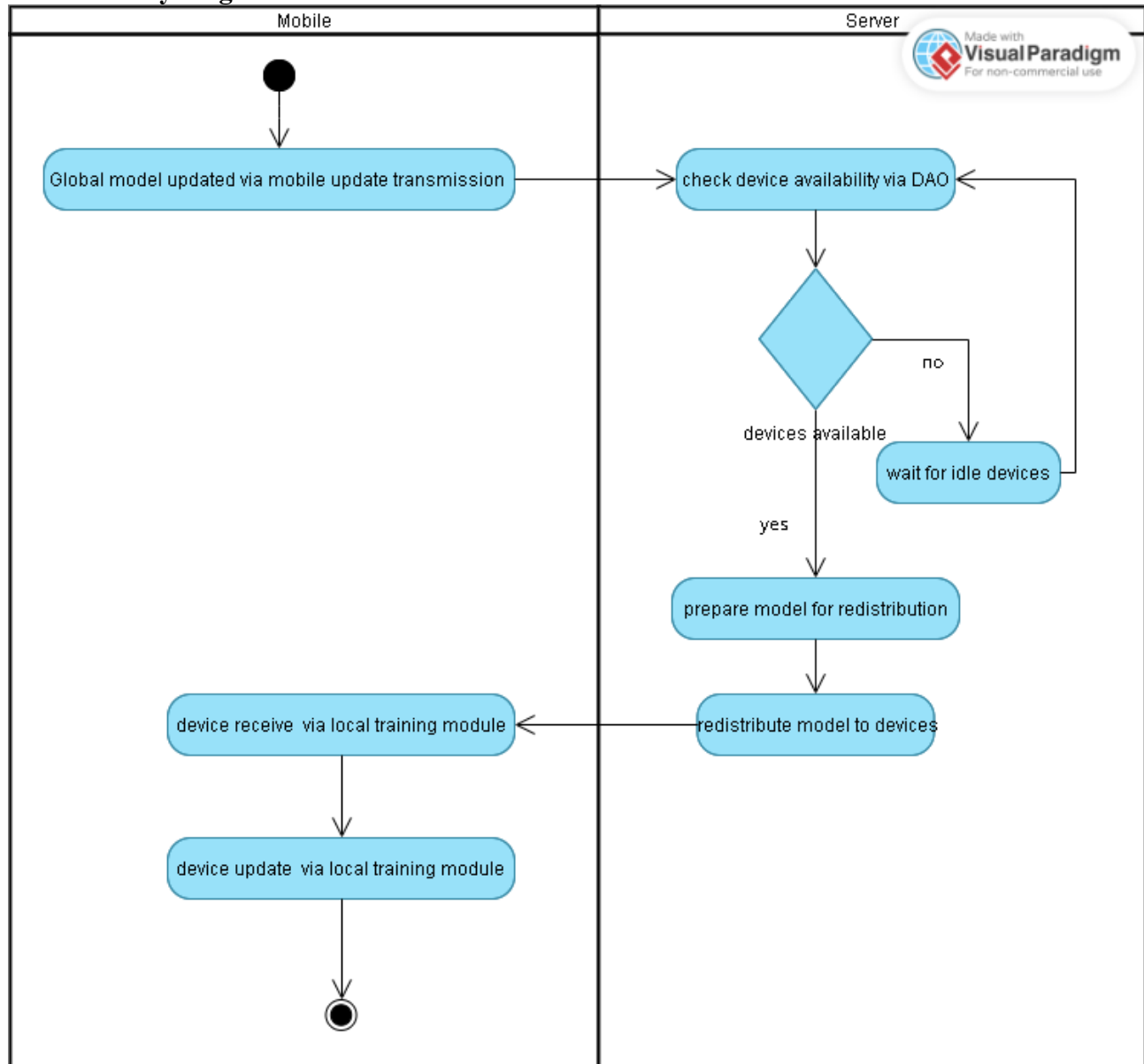


Figure 31- ER7 Activity diagram

ER-7 Sequence diagram:

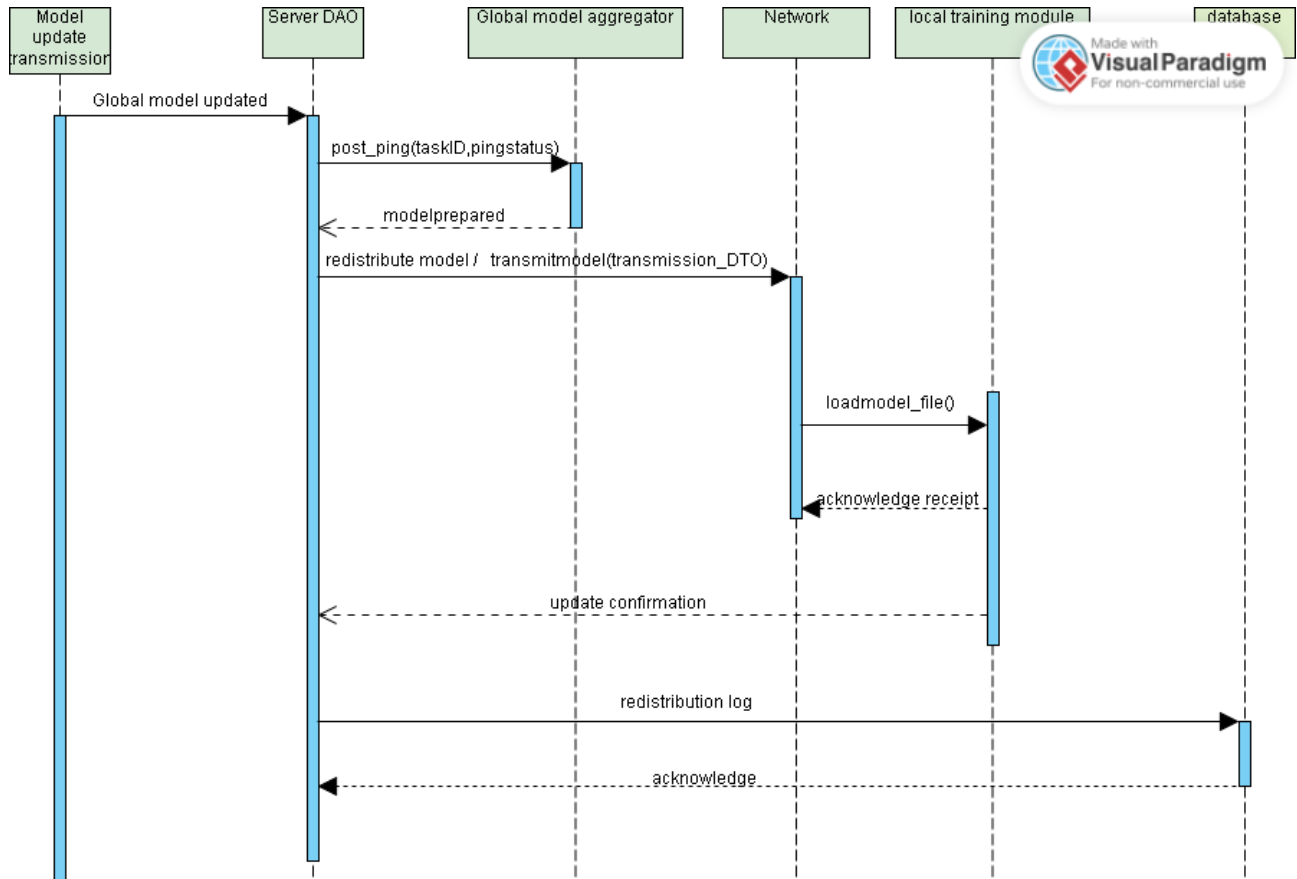


Figure 32- ER7 Sequence diagram

ER-7 State machine diagram:

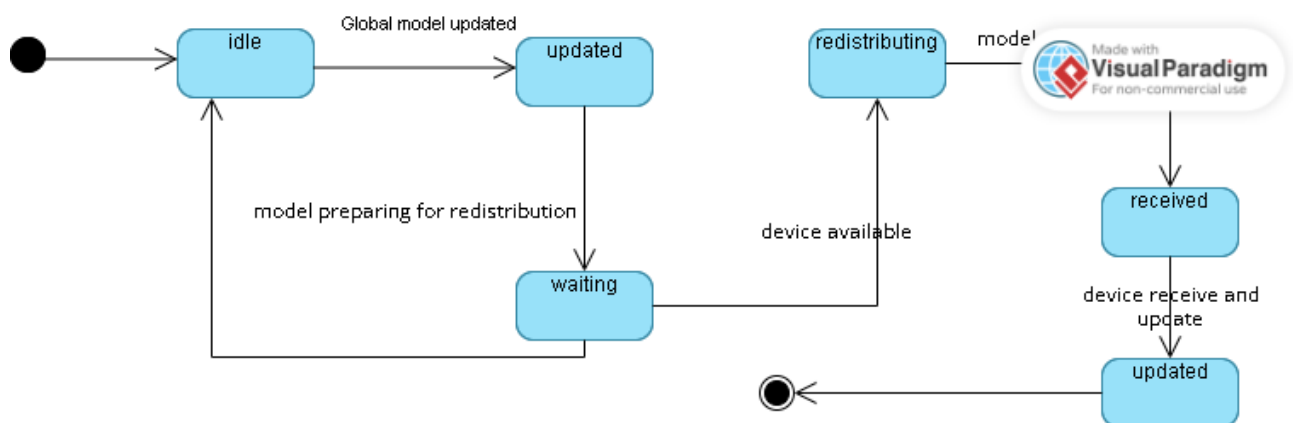


Figure 33- ER7 State machine diagram

ER-8 Activity diagram:

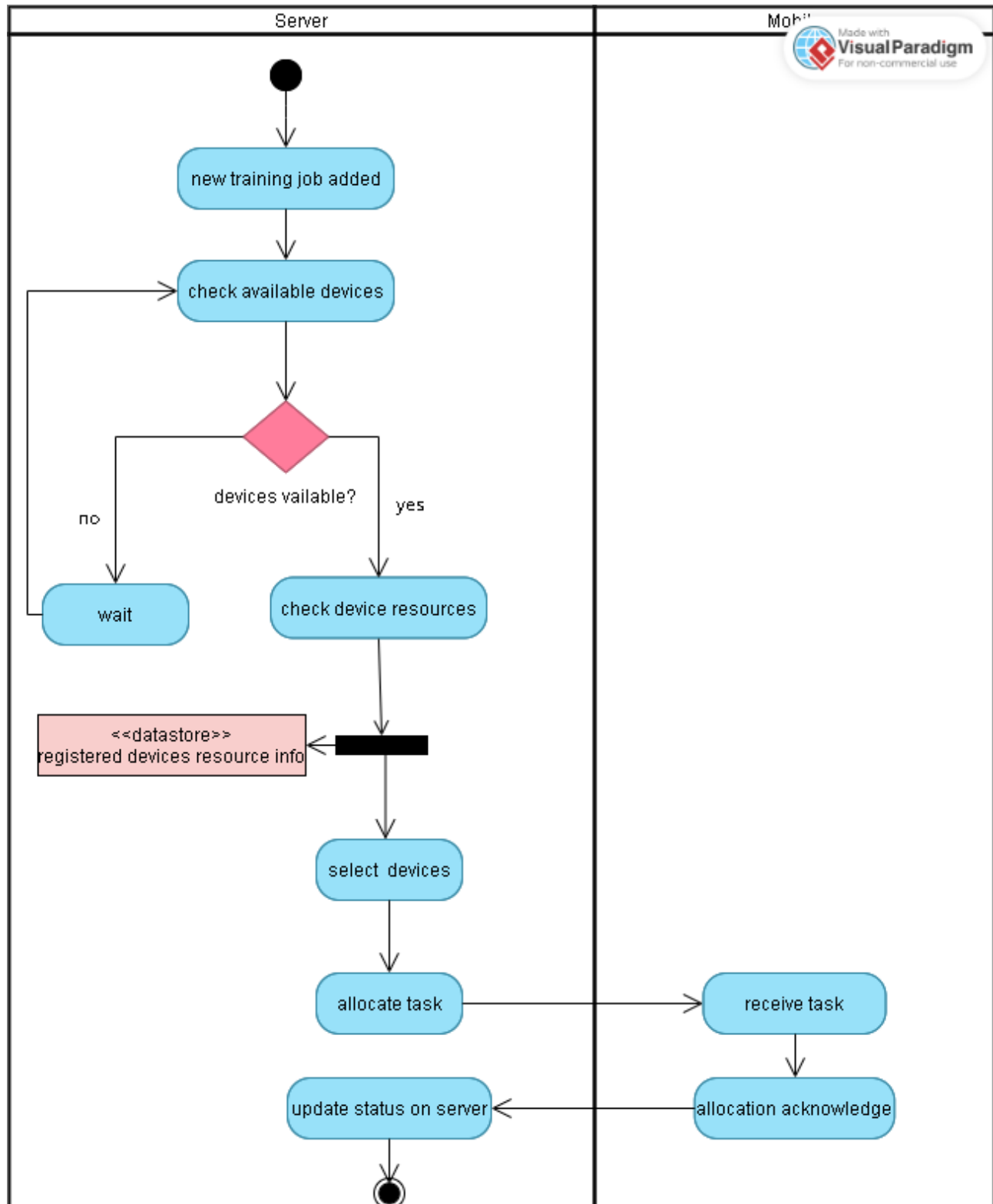


Figure 34- ER8 Activity diagram

ER-8 Sequence diagram:

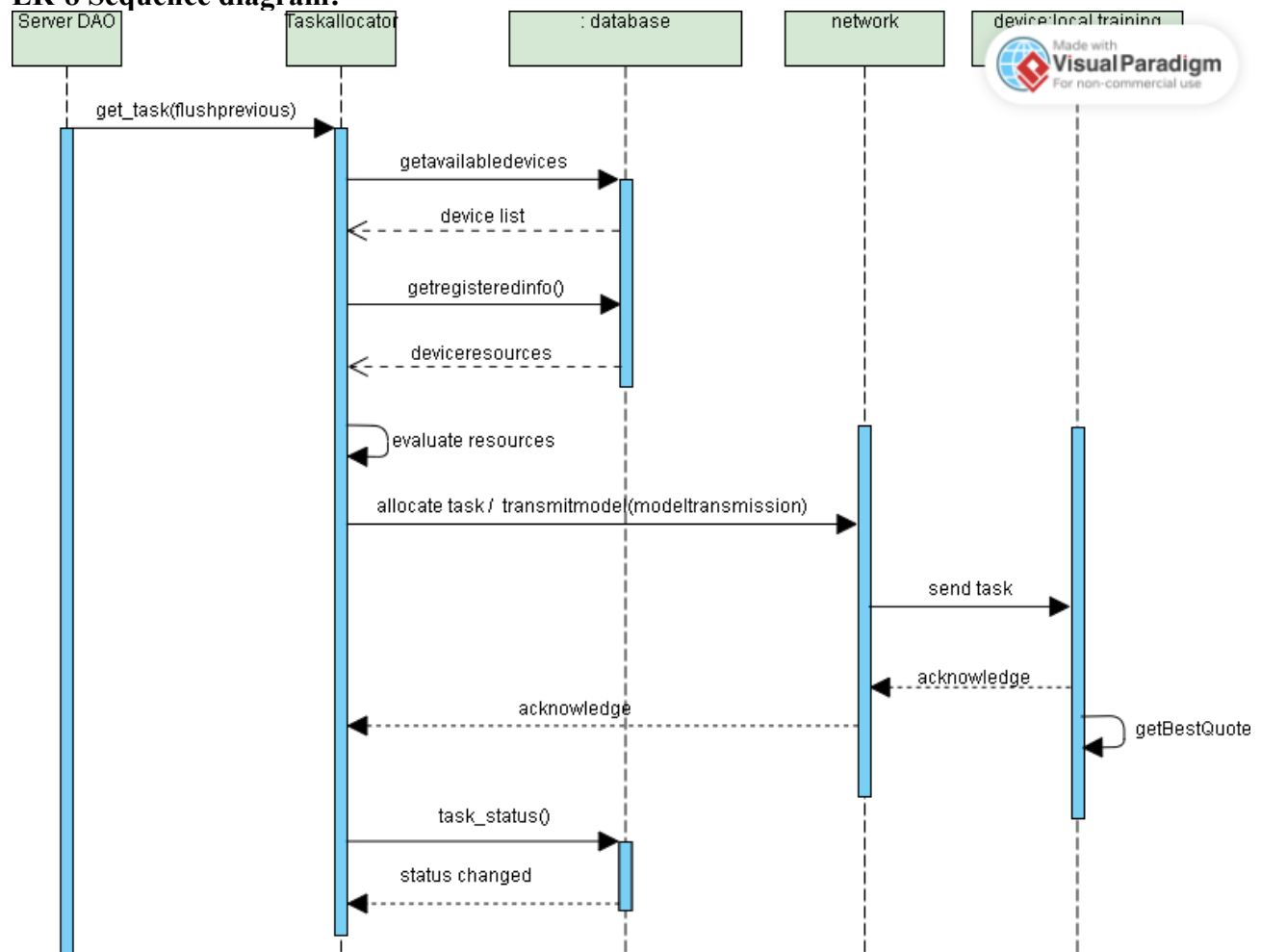
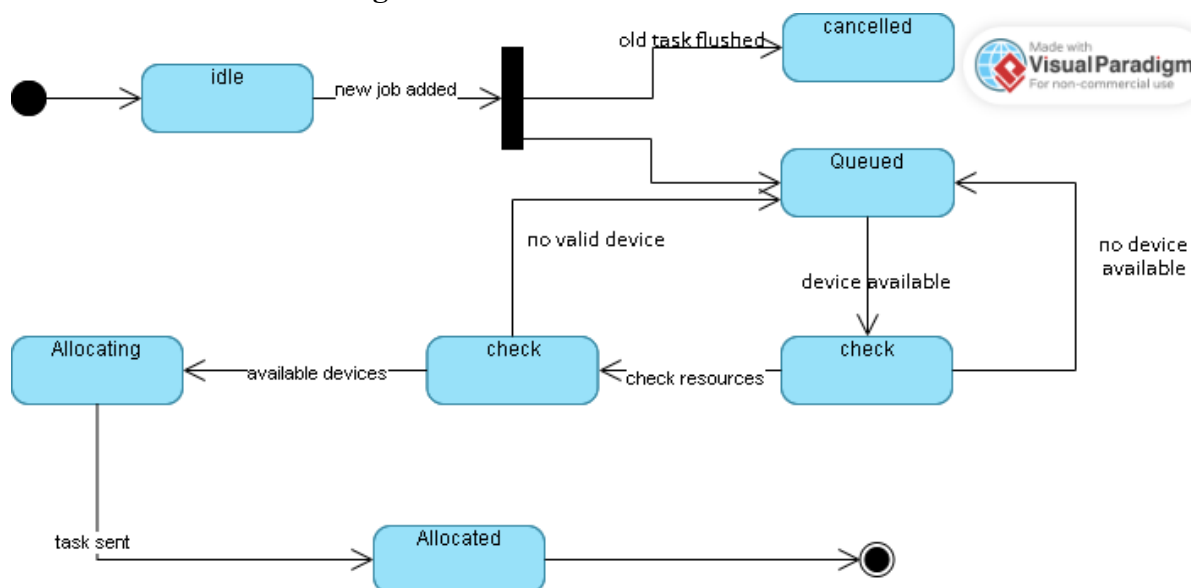


Figure 35- ER8 Sequence diagram

ER-8 State machine diagram:



ER-

9 Activity diagram:

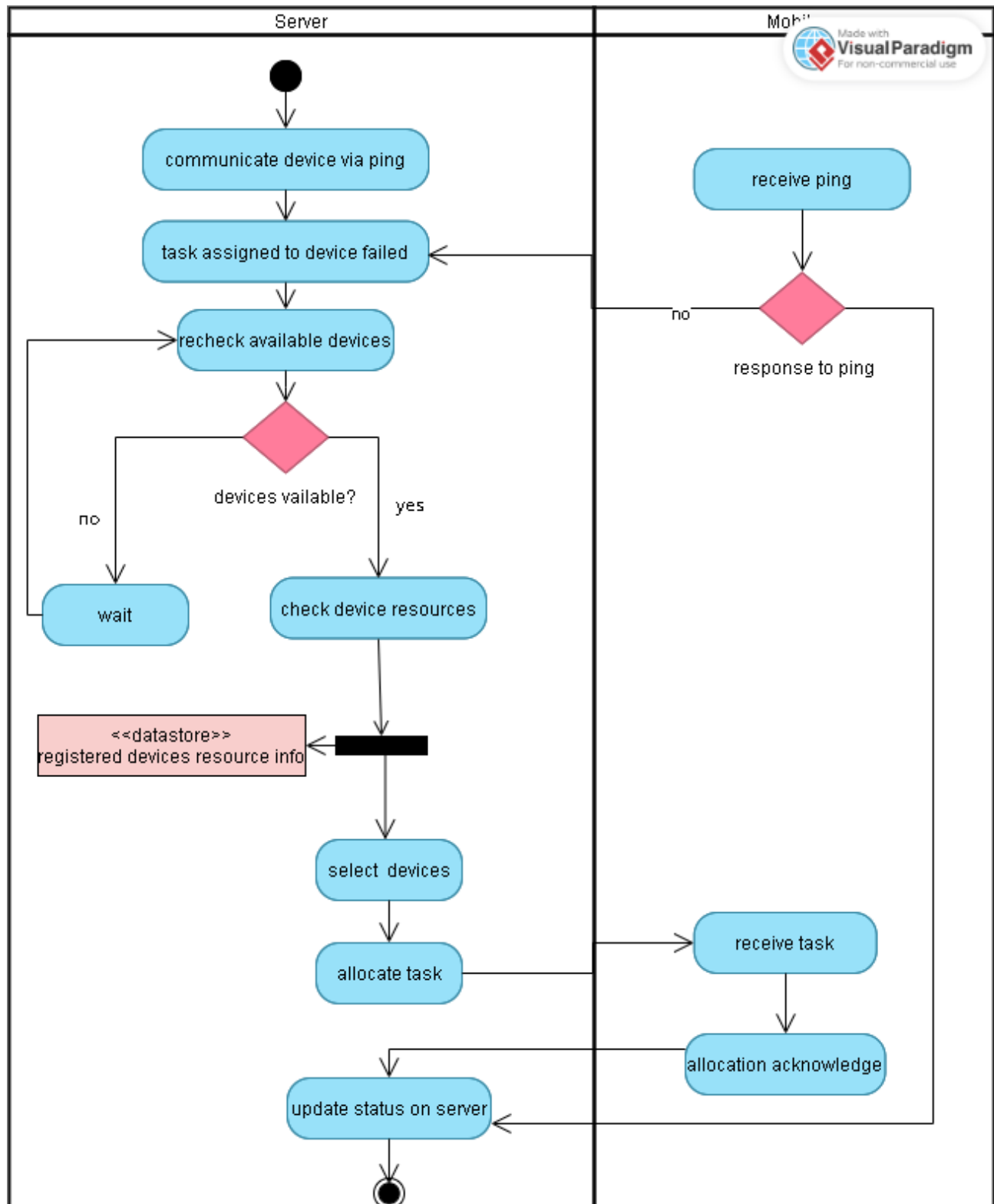


Figure 36- ER9 Activity diagram

ER-9 Sequence diagram:

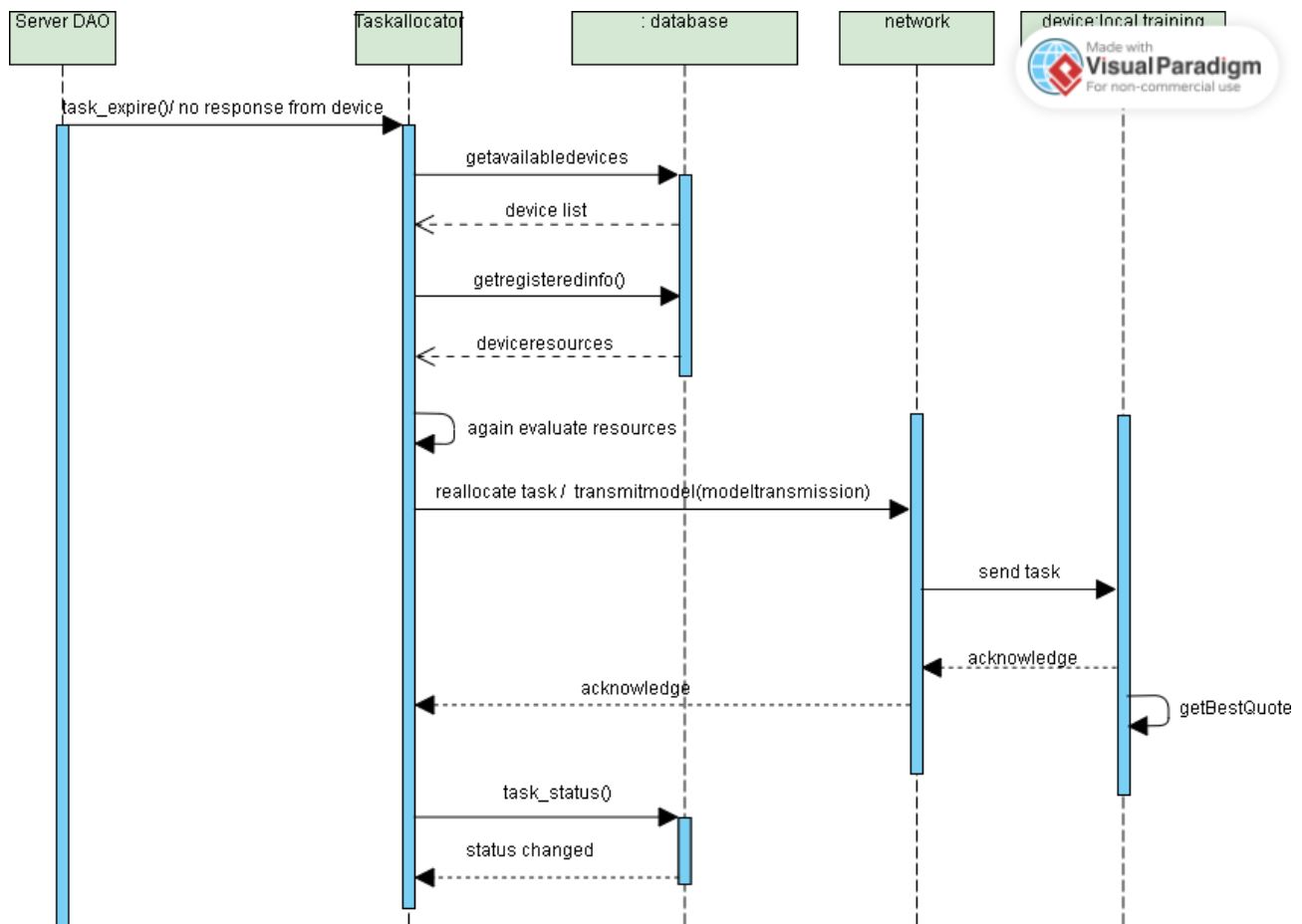


Figure 37- ER9 System sequence diagram

ER-9 State machine diagram:

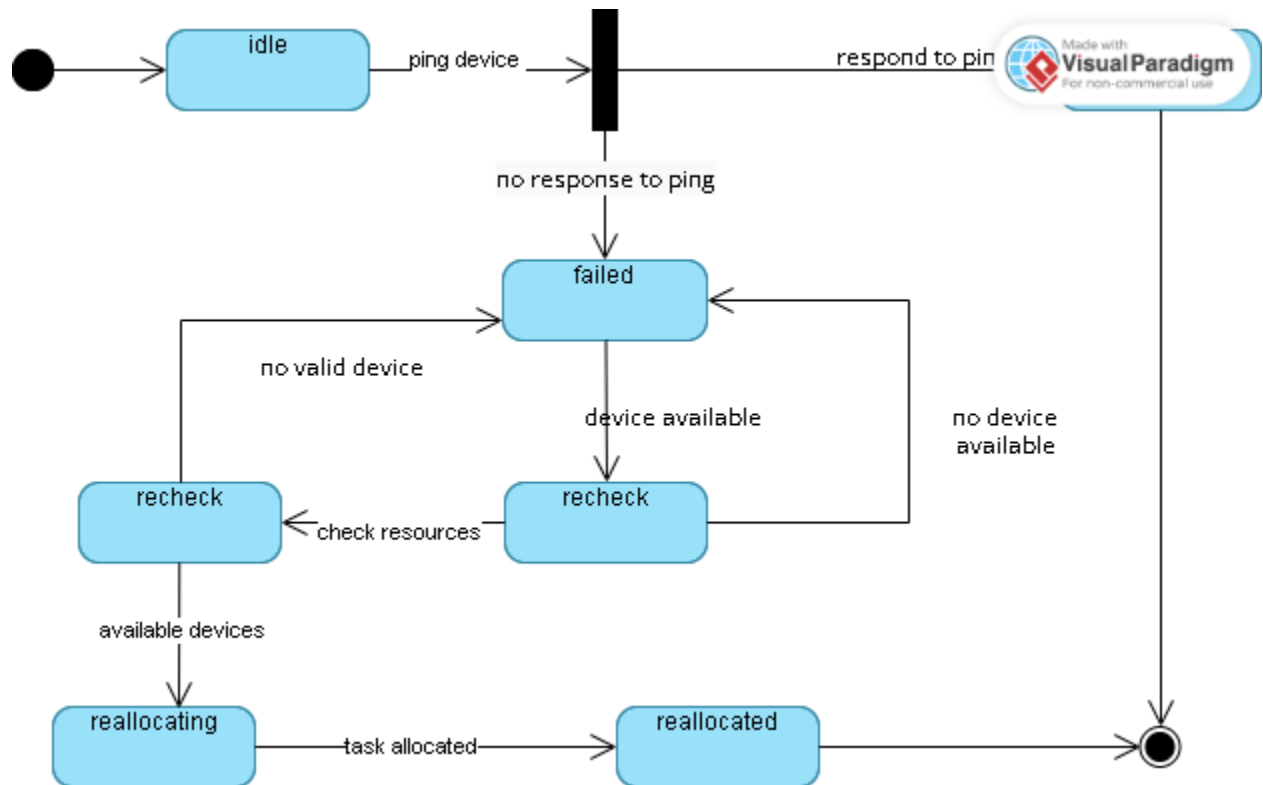


Figure 38- ER9 State machine diagram

ER-10 Activity diagram:

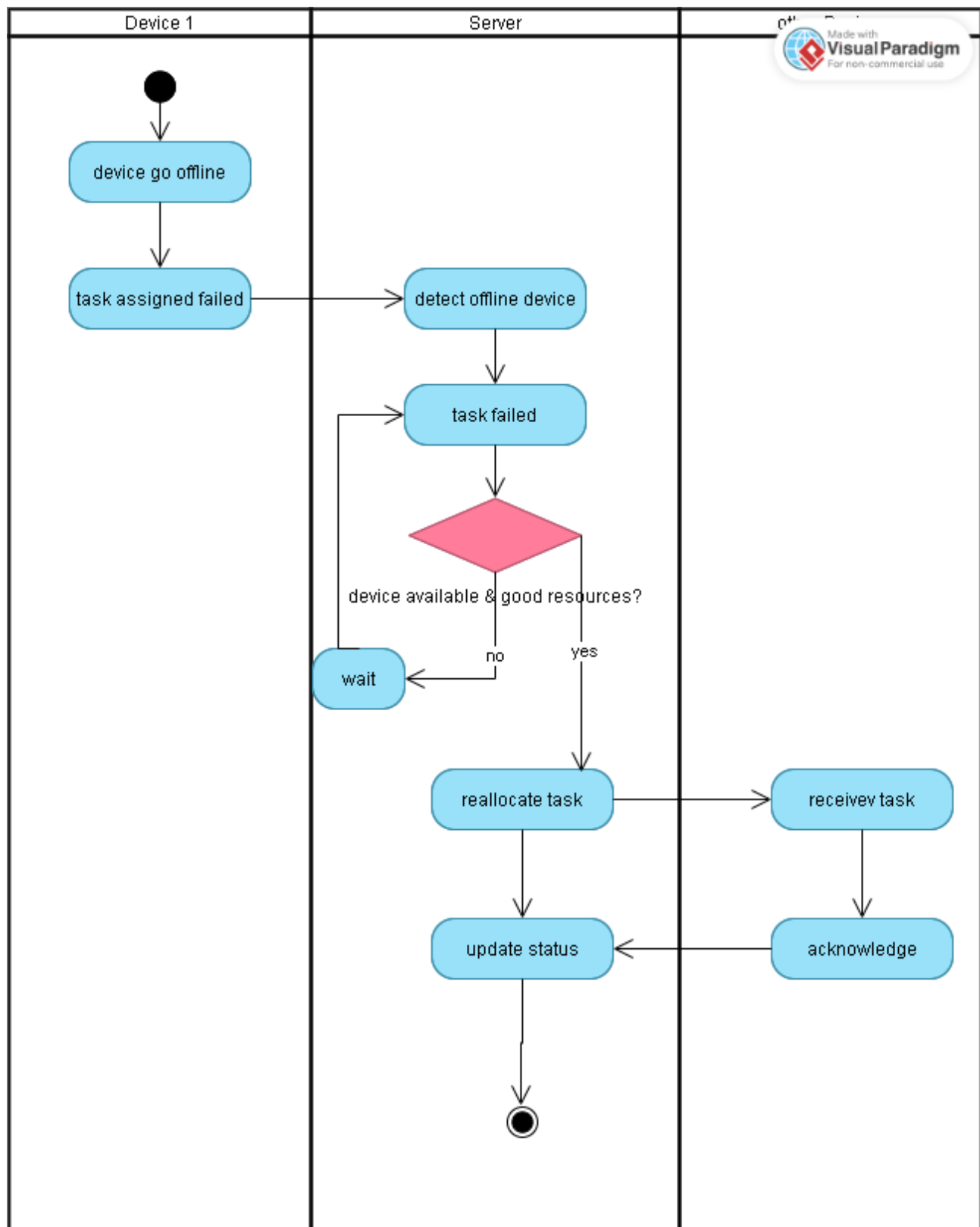


Figure 39- ER10 Activity diagram

ER-10 Sequence diagram:

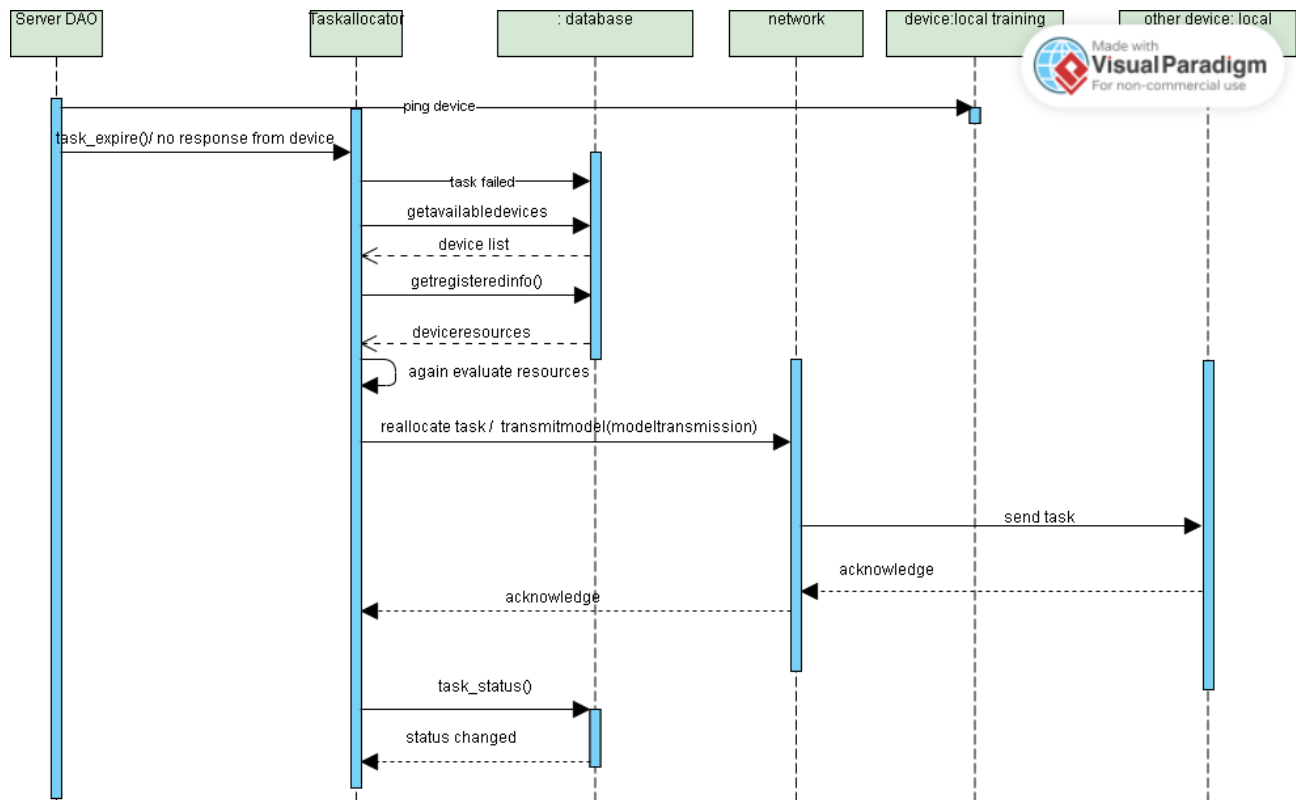


Figure 40- ER10 System sequence diagram

ER-10 State machine diagram:

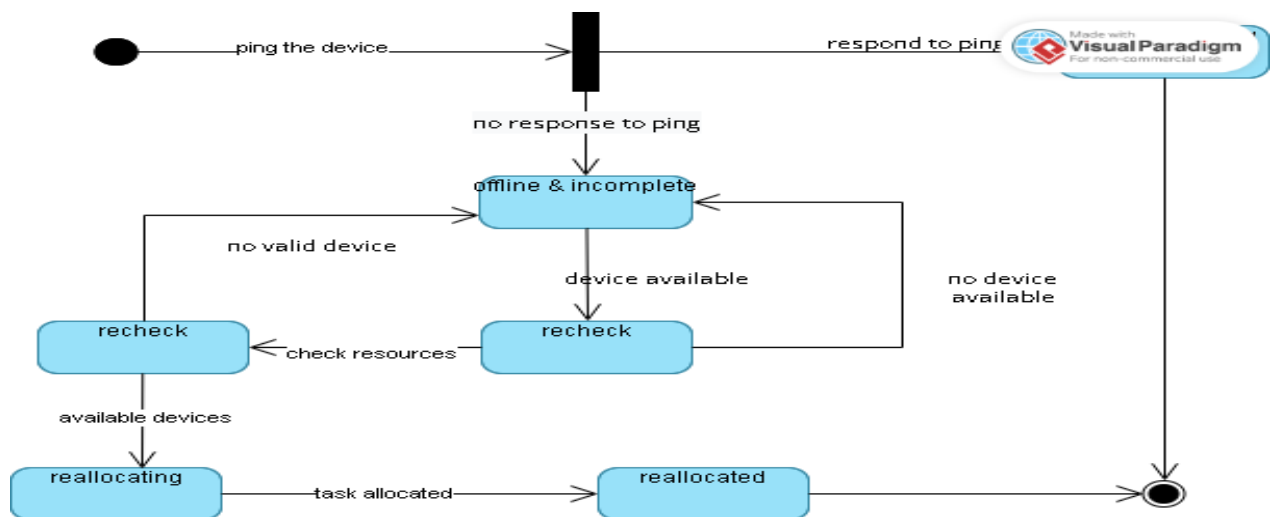


Figure 41- ER10 State machine diagram

ER-11 Activity diagram:

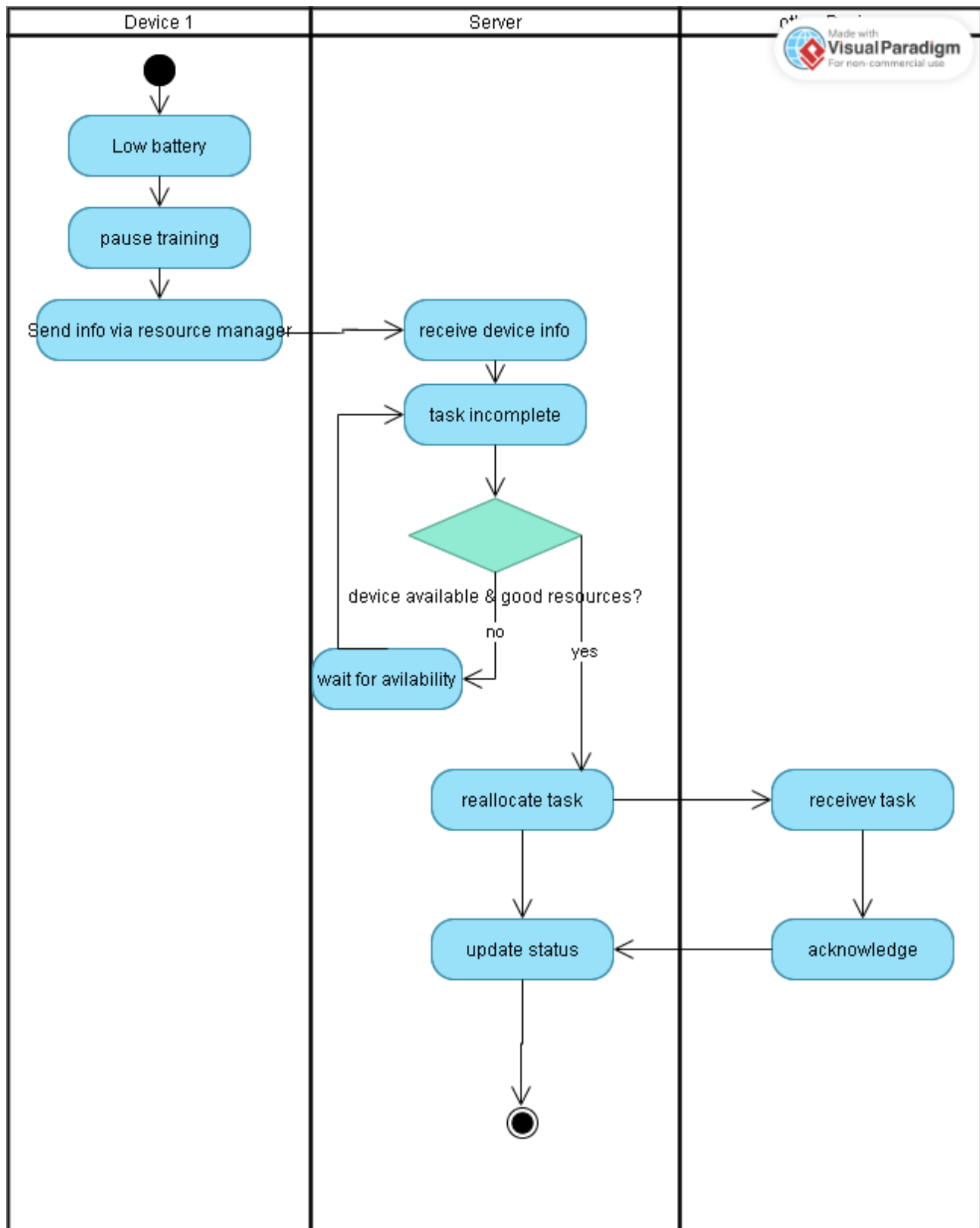


Figure 42- ER11 Activity diagram

ER-11 Sequence diagram:

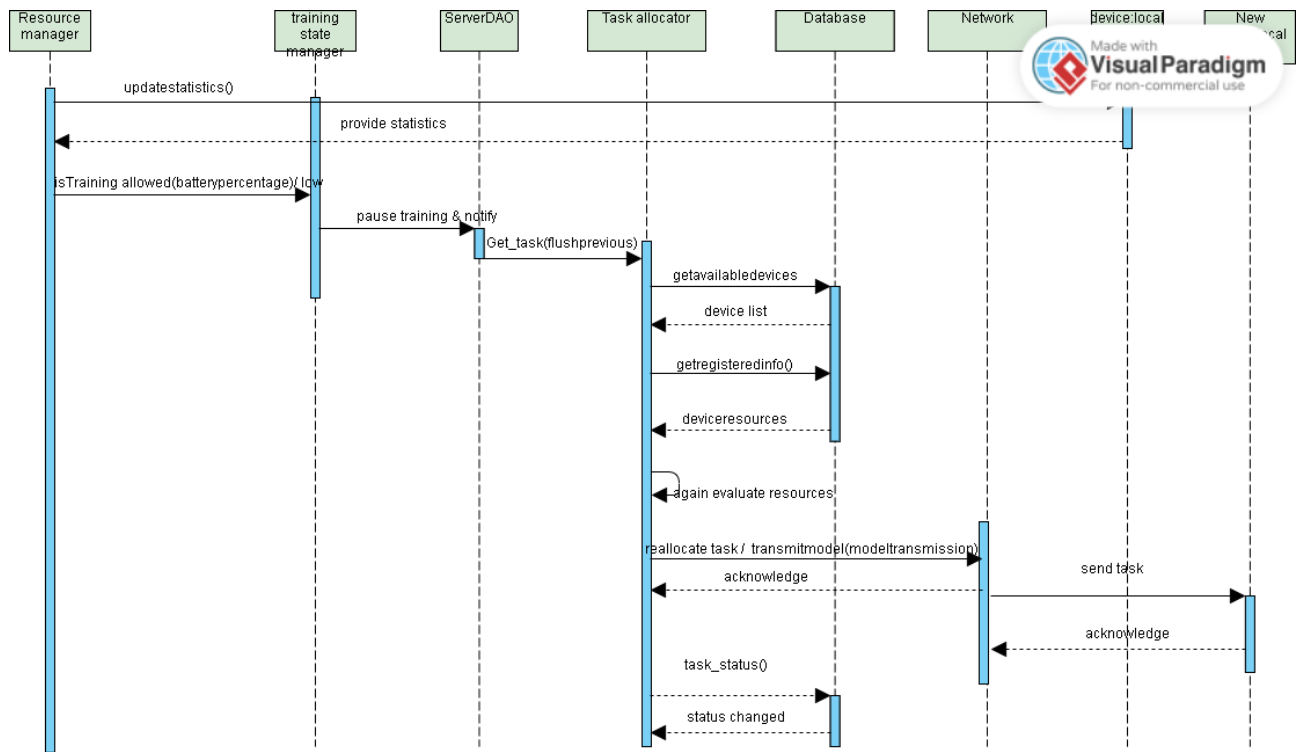
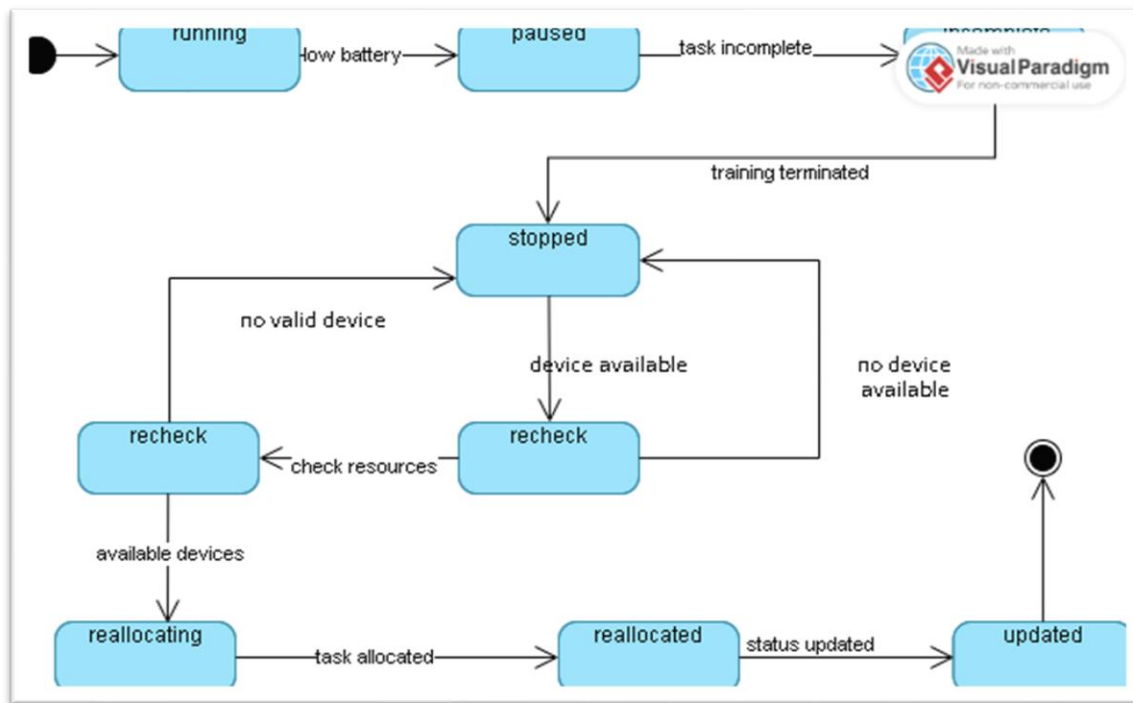


Figure 43- ER11 Sequence diagram

ER-11 State machine diagram:



4.3 Data Design

The information domain of the Fractal system is transformed into data structures into four major entities User, task, Allocation record, Unregister DTO. User registration is stored in User collection. User un-registration complaint and description request data from Unregister DTO collection. When tasks are assigned to user, task assignment information is stored in task allocation collection. Training tasks are stored by type and date. Database used is Firebase Fire store NoSQL

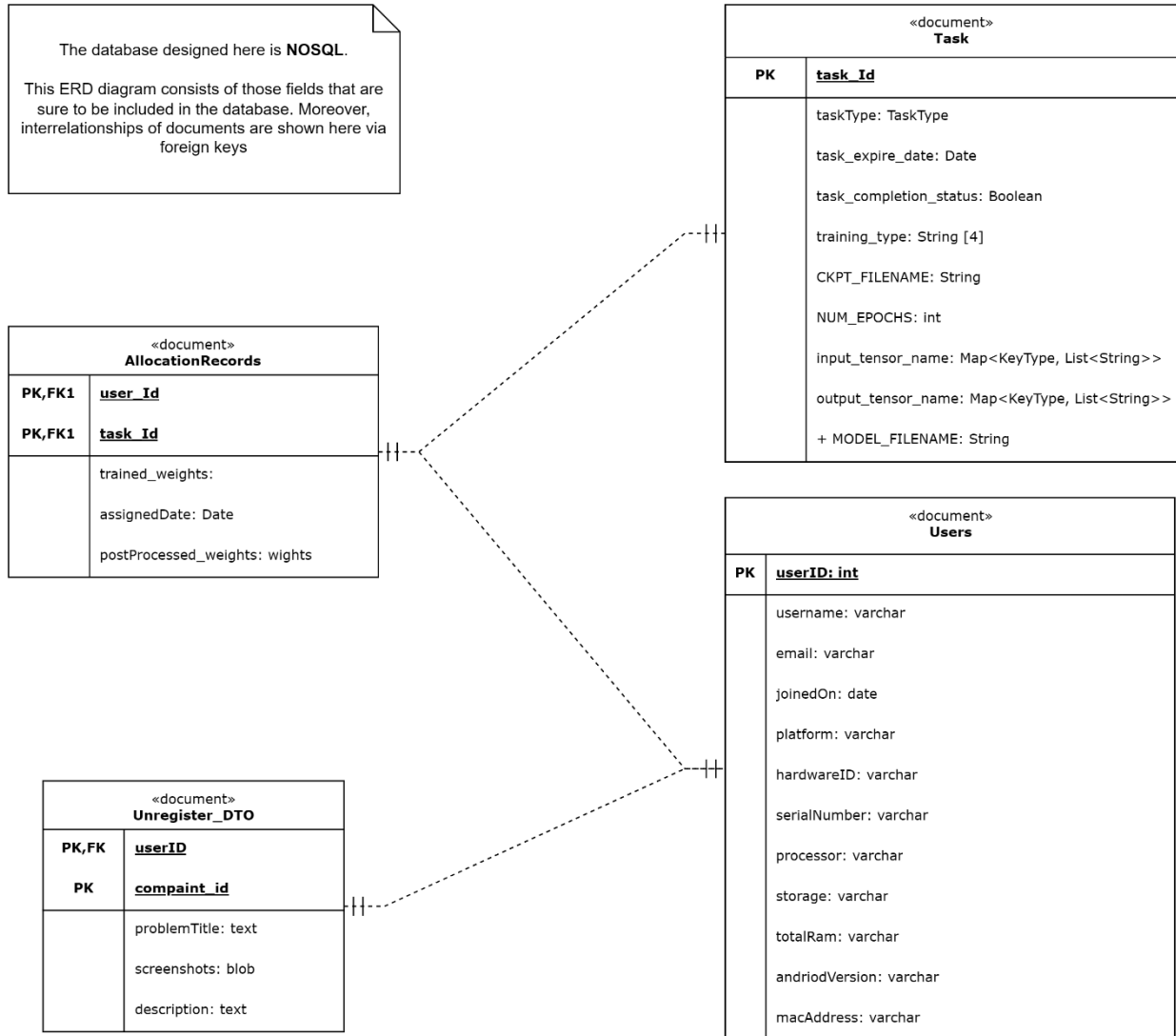


Figure 45- ERD diagram

4.3.1 Data Dictionary

Data dictionary list system entities with its attributes, methods and method parameters. All are provided below:

1. Allocation Records:

Attributes:

- User ID
- Task ID
- Trained weight

- Assigned weight
- Post processed weight

2. Task

- Task type
- Task id
- Tasks expire date
- Task completion status
- Training type
- Input tensor name
- Output tensor name
- CKPT-filename

3. User

- User ID
- Username
- Email
- Joined on
- Platform
- Hardware ID
- processor
- Storage
- Total RAM
- android version
- macAddress

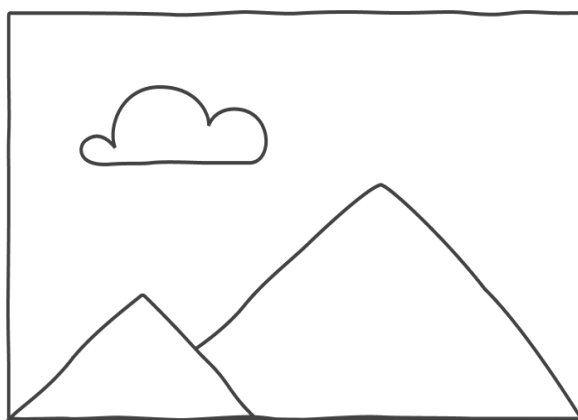
4. Unregister-DTO

- Complaint ID
- User ID
- Problem title
- Screen shot
- Description

4.4 Human Interface Design

Fractal (a decentralized distributed computation harvesting to provide computation services) is a system that allows users to join the computation harvesting network by installing the application, then by sharing the idle resources of their devices (android mobile devices as contributor's only) for tasks such as model training. Below provided screen images are the step-by-step guide of using system from user's perspectives

4.4.4 Screen Images



company Name

what our company presents/ what our
company do/ the ratio of our app/ what
the services we present. basically/ about
the hosted task.

get started

Figure 46-Home

...



Settings

Training Preference

- On-WiFi Train Only ☒
- On-Data Train Only ☒
- Over-Night Utilization ☒
- Idle-Time Utilization ☒

Only train when charging is above: 34%

0%

100%

- On-Charging exclusive training ☒

Device

Home

Model

Figure 47-Setting

Figure 48-Active

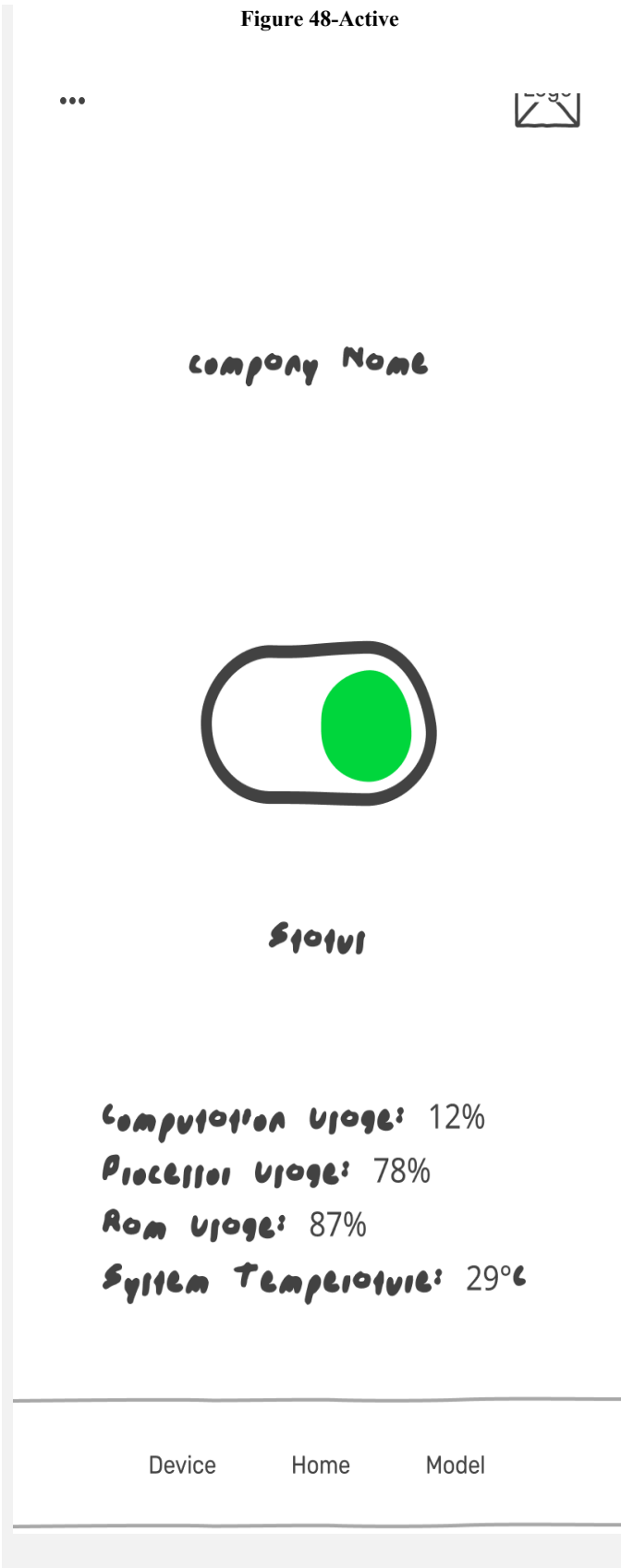


Figure 49 - Model training

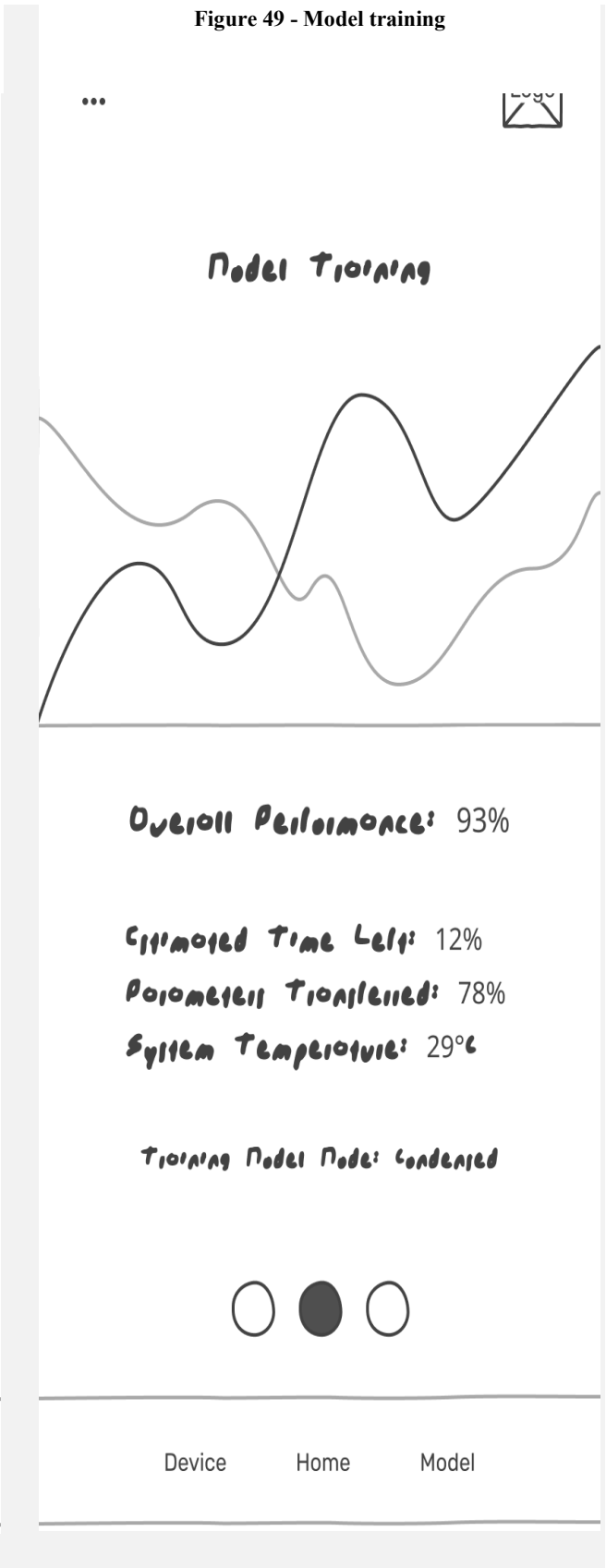


Figure 50-Device insight screen

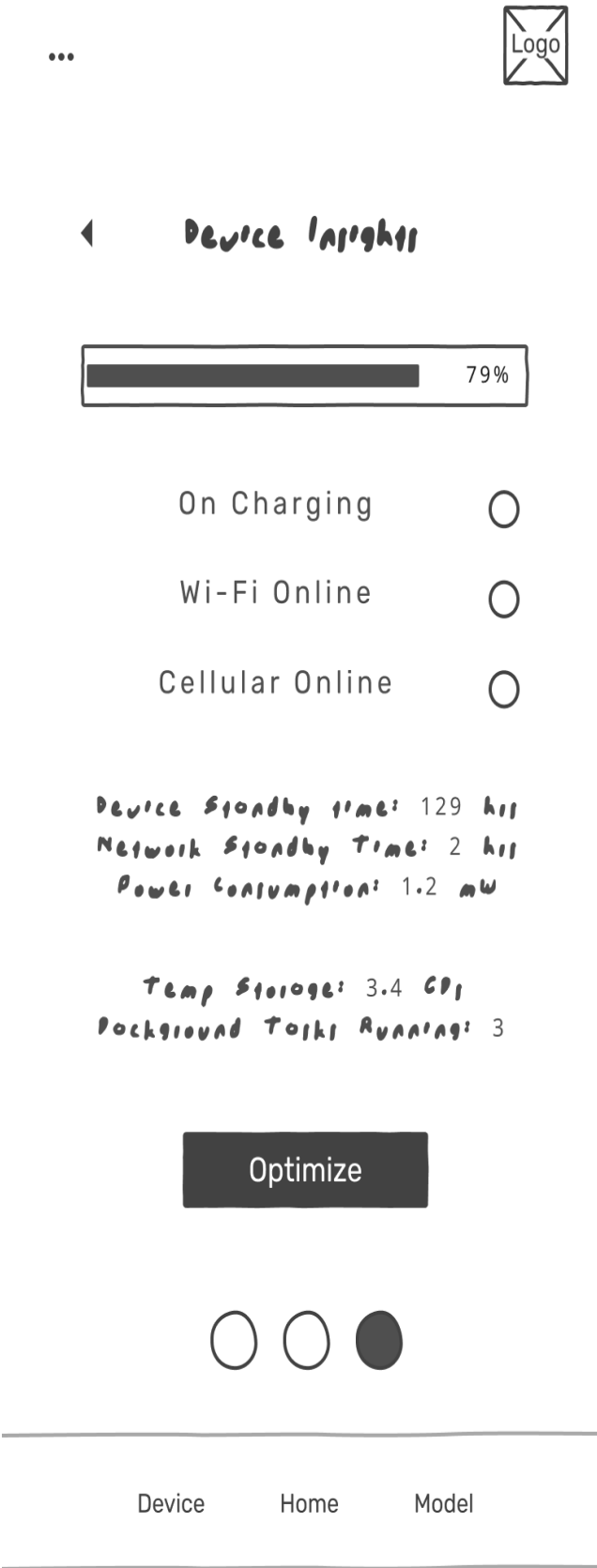


Figure 51 - Usage insight screen

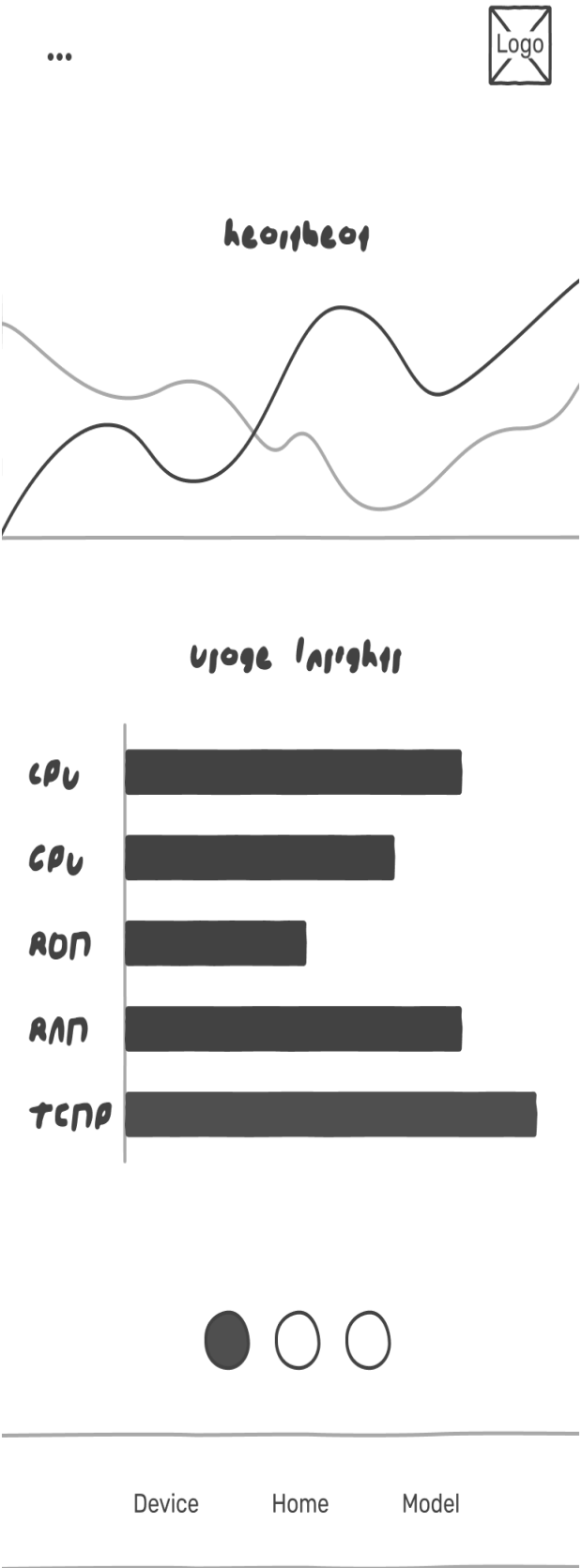




Figure 52 - Status screen

Registered Info

User

Username: whitekadow69

Email: b1ed4389@gmail.com

Joined On: 23rd Feb, 2024

Platform

Android

Hardware Info

Hardware ID: 203-2930

Serial Number: 203-2930

Processor: Snapdragon 4.2

Storage: 64GB

Total RAM: 4GB

Software Info

Android Version: 7.1 One

Network Info

MAC Address: 04-2D-91-98-32-0A

Device

Home

Model

Figure 53 - Registration screen

4.4.1 Screen Objects and Actions

4.4.2 Objects

- Company Name
- Button
- Checkboxes
- Slider
- Bottom navigation
- Toggle switch (app)
- Usage insight graph
- Training insight graph

4.4.3 Actions

- Company names display the “Fractal” with additional information below
- Setting checkboxes allows user to select preferences according to choice
- Sliders allow users to select minimum battery level for training
- Optimize button triggers to improve device performance
- Get started button work as a join now link
- Toggle switch is used to active/inactive for training
- Bottom navigation bar navigates to selected group of screens
- Second toggle designed navigation bar navigates to the screen of device insight and usage insight
- Training insight graph displays the progress of the model training over the time
- Usage insight graph displays the resource consumption

5 Implementation

This section lists the user interface and all the algorithms that combine to make the system work. These algorithms are provided in the structured pseudocode from

5.1 Algorithm

Below are the core functional modules of this class described with clear structured pseudocode

Algorithm 1 Model initialization
Input: Android context
Output: Initialize tensor flow lite interpreter
1: FUNCTION InitializeModel(context) 2: TRY 3: Open asset file descriptor for "model.tflite" 4: Map model file to read-only ByteBuffer (modelBuffer) using FileChannel 5: interpreter ← new Interpreter(modelBuffer) 6: LOG "Model loaded successfully" 7: LOG "Available signatures: " + interpreter.signableKeys 8: CATCH exception 9: LOG "ERROR: Model loading failed - " + exception.message 10: END TRY 11: END FUNCTION
Algorithm 2 On device training loop
Input: Training callback for progress update
Output: Trained model
1: FUNCTION TrainModel(callback) 2: IF interpreter IS NULL THEN 3: LOG "ERROR: Interpreter not initialized" 4: RETURN 5: END IF 6: imgFile ← File(internalStorage, "train_images_server.bin") 7: lblFile ← File(internalStorage, "train_labels_server.bin") 8: IF NOT (imgFile.exists AND lblFile.exists) THEN 9: LOG "ERROR: Training data files missing" 10: callback.onLog("Data files not found") 11: RETURN 12: END IF 13: totalBatches ← NUM_EPOCHS × (NUM_TRAININGS / BATCH_SIZE) 14: currentBatch ← 0 15: merge ← true 16: WHILE merge == true DO 17: merge ← false 18: FOR epoch FROM 0 TO NUM_EPOCHS-1 DO 19: LOG "Starting Epoch " + (epoch + 1) 20: FOR batchIndex FROM 0 TO (NUM_TRAININGS / BATCH_SIZE - 1) DO 21: batchData ← LoadBatchFromFile(batchIndex, imgFile, lblFile) 22: IF batchData IS NULL THEN 23: CONTINUE 24: END IF 25: (imageBatch, labelBatch) ← batchData 26: // Prepare batched inputs 27: batchImages ← AllocateDirectBuffer(BATCH_SIZE × IMG_SIZE × 4).asFloatBuffer() 28: batchLabels ← AllocateDirectBuffer(BATCH_SIZE × NUM_CLASSES × 4).asFloatBuffer() 29: batchImages.put(imageBatch.asFloatBuffer()) 30: batchLabels.put(labelBatch.asFloatBuffer()) 31: batchImages.rewind() 32: batchLabels.rewind() 33: inputs ← Map("x" → batchImages, "y" → batchLabels)

<pre> 34: lossBuffer ← AllocateDirectBuffer(4).asFloatBuffer() 35: outputs ← Map("loss" → lossBuffer) 36: interpreter.runSignature(inputs, outputs, "train") 37: loss ← lossBuffer.get(0) 38: currentBatch ← currentBatch + 1 39: progress ← (currentBatch × 100) / totalBatches 40: IF currentBatch MOD 10 == 0 OR currentBatch == totalBatches THEN 41: LOG "Batch " + currentBatch + " Loss: " + loss 42: callback.onProgress(progress) 43: END IF 44: END FOR 45: END FOR 46: LOG "Training completed successfully" 47: END WHILE 48: END FUNCTION </pre>	
Algorithm 3 Batch data loading	Algorithm 4 save trainable weights
Input: image file	Input: None
Output: Pair of byte buffers(images)	Output: Boolean true success
<pre> 1: FUNCTION LoadBatchFromFile(batchIndex, imgFile, lblFile) 2: TRY 3: imgStream ← FileInputStream(imgFile) 4: lblStream ← FileInputStream(lblFile) 5: imageOffset ← batchIndex × BATCH_SIZE × IMG_SIZE × 4 6: labelOffset ← batchIndex × BATCH_SIZE × NUM_CLASSES × 4 7: imgStream.channel.position(imageOffset) 8: lblStream.channel.position(labelOffset) 9: imageBytes ← ByteArray(BATCH_SIZE × IMG_SIZE × 4) 10: labelBytes ← ByteArray(BATCH_SIZE × NUM_CLASSES × 4) 11: imgStream.read(imageBytes) 12: lblStream.read(labelBytes) 13: imgStream.close() 14: lblStream.close() 15: imageBuffer ← ByteBuffer.wrap(imageBytes).order(nativeOrder) 16: labelBuffer ← ByteBuffer.wrap(labelBytes).order(nativeOrder) 17: RETURN (imageBuffer, labelBuffer) 18: CATCH exception 19: LOG "ERROR loading batch " + batchIndex + ": " + exception.message 20: RETURN NULL 21: END TRY 22: END FUNCTION </pre>	<pre> 1: FUNCTION SaveWeights() 2: checkpointPath ← internalStorage + "/checkpoint.ckpt" 3: TRY 4: inputs ← Map("checkpoint_path" → checkpointPath) 5: interpreter.runSignature(inputs, emptyMap, "save") 6: LOG "Weights saved successfully" 7: RETURN true 8: CATCH exception 9: LOG "Save failed: " + exception.message 10: RETURN false 11: END TRY 12: END FUNCTION </pre>
Algorithm 5 Restore trainable weight	
Input: none	
Output: Boolean	
<pre> 1: FUNCTION RestoreWeights() 2: checkpointFile ← File(internalStorage, "checkpoint.ckpt") 3: IF NOT checkpointFile.exists THEN 4: LOG "No checkpoint found" 5: RETURN false 6: END IF 7: TRY 8: inputs ← Map("checkpoint_path" → checkpointFile.absolutePath) 9: interpreter.runSignature(inputs, emptyMap, "restore") 10: LOG "Weights restored successfully" 11: RETURN true 12: CATCH exception 13: LOG "Restore failed: " + exception.message 14: RETURN false 15: END TRY 16: END FUNCTION </pre>	

Table 29 Algorithm

5.2 User Interface

Fractal user interface is Mobile App (Android) which is designed for decentralized distributed computation harvesting on mobile devices. The user interface screen available in both Black/white for design clarity. The details of the screens are provided below:

- **About us:** About us, also named as home screen is an information screen that explains the applications hypothesis with a registration link
- **Device Insights:** Device insights provide relevant information such as battery percentage, running tasks, network information, power consumption etc. with a button name “Optimize” that cleans up the irrelevant running tasks
- **Device registration:** Entry point for the user to link their devices. Take username, email address and password, then on click “Register” redirect user to Home Screen
- **Forget Password:** Took user email then send OTP via email and as user enter new password click “Register”, then redirect to Login Screen
- **Home Active:** Displays device real time information about resources in case of Inactive (White), user click on White then it turns to black button that means model training is active
- **Home Inactive:** Displays device real time information about resources in case of active (Black), user click on “Black” then it turns to white button, means model training is inactive
- **Model training:** Displays the system’s performance with few metrics. Model training starts after Model Active. Get relevant information about models with the help of Ping
- **Register Info:** Displays users’ device relevant information gathered after the registration process
- **Setting:** Setting screen enables customization of preferences according to user choice

About Us

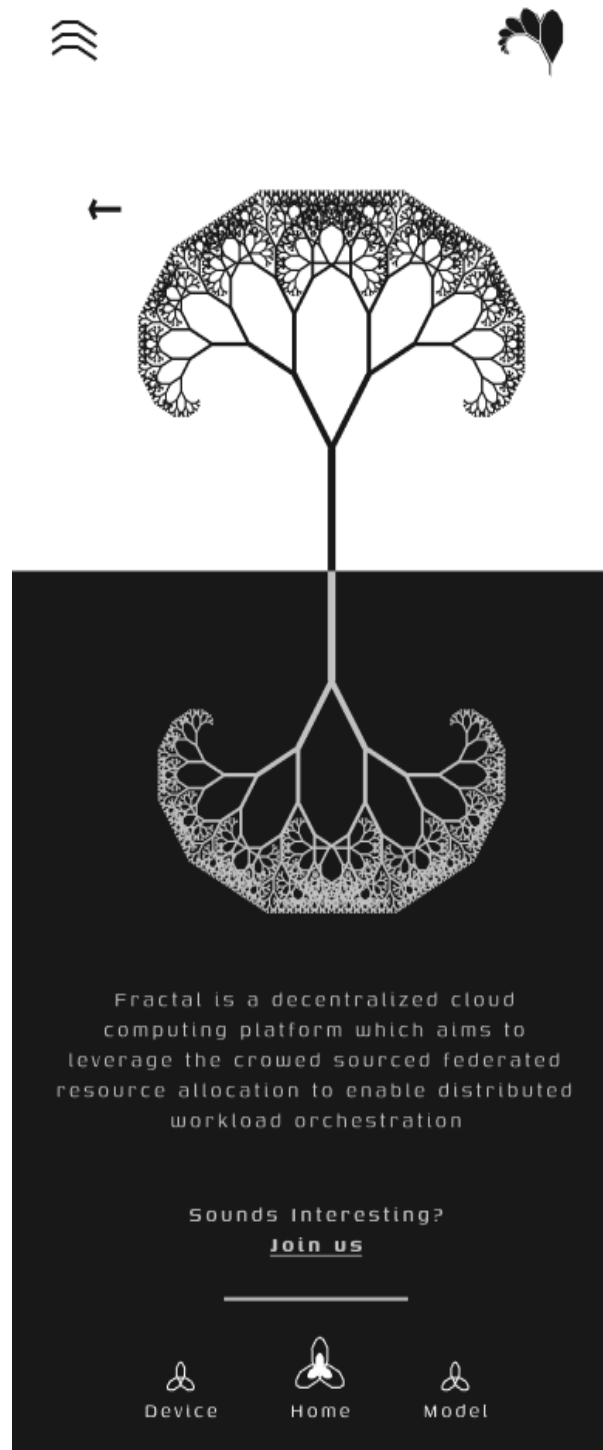


Figure 54- About US UI

Device Insights

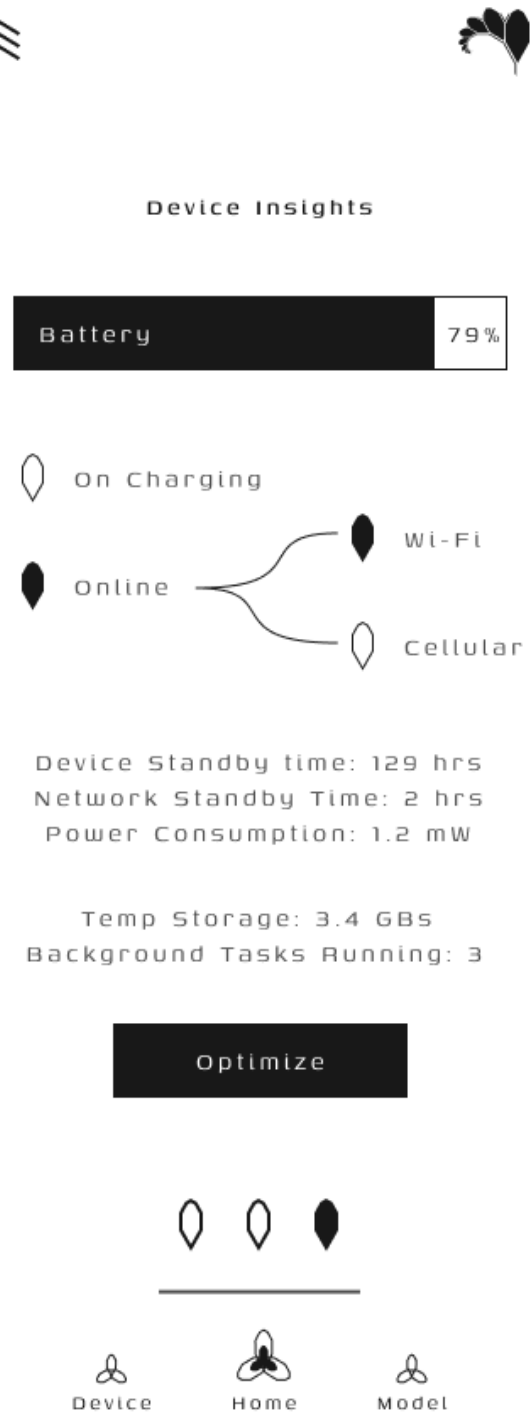


Figure 55 - Device Insight UI

Device registration



 Device Registration

Username

Email

Password



[Forget Password](#)

0

I Agree To [Terms And Conditions](#)

Register

Figure 56 - Device registration UI

Forget Password



Forget Password

Email

Confirmation code

New Password



Register

DeviceHomeModel

Figure 57 - Forget password UI

Home Active



FRACTAL



69%

Computation Usage: 12%
Processor Usage: 78%
Ram Usage: 87%
System Temperature: 29°C



Figure 58 - Active UI

Home Inactive



FRACTAL



inactive

Computation Usage: 12%
Processor Usage: 78%
Ram Usage: 87%
System Temperature: 29°C



Figure 59 - Inactive UI

Model Training

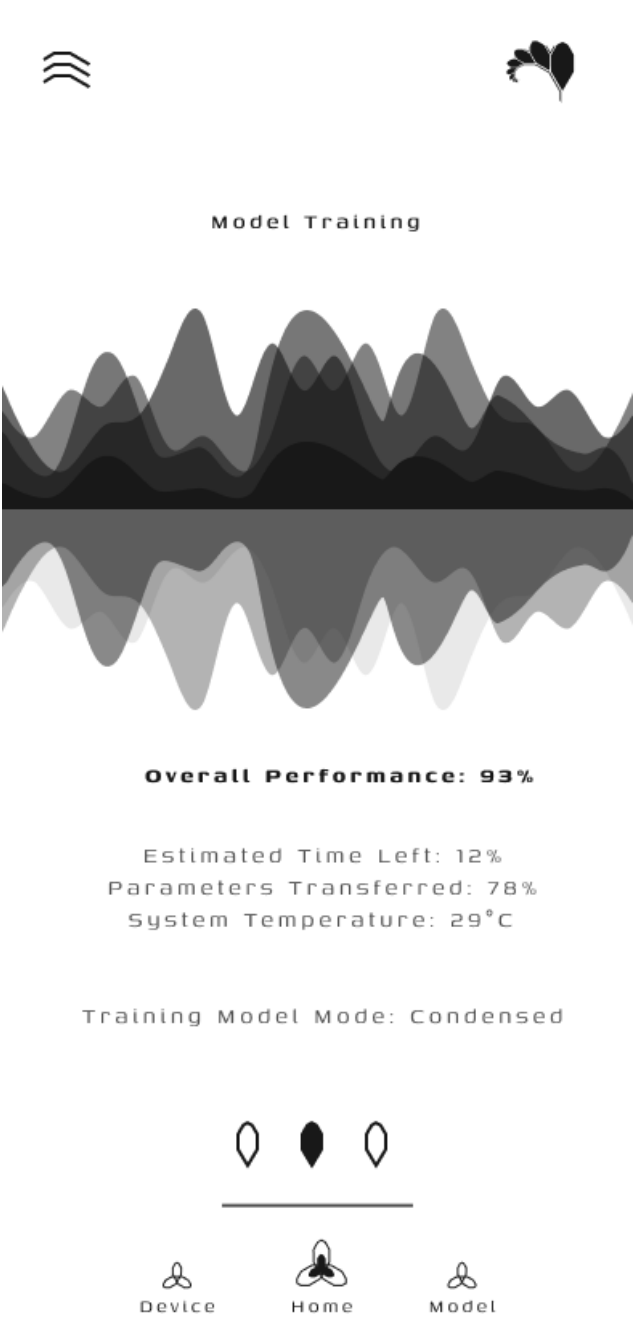


Figure 60 - Model training UI

Registered Info



Figure 61 - Registered Info UI

Setting Screen

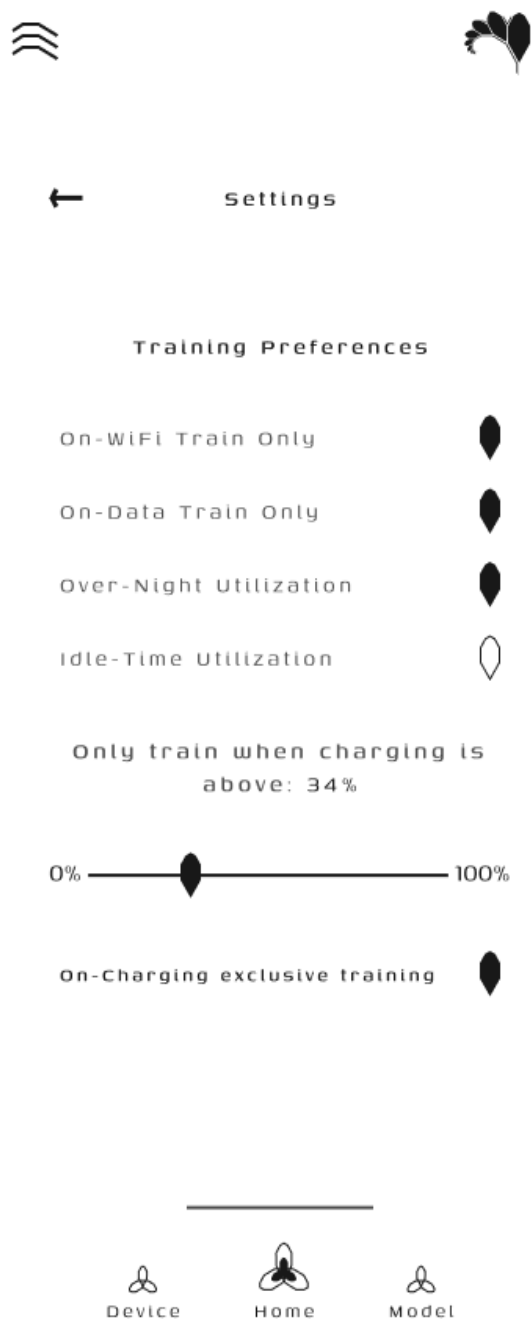


Figure 63 - Setting UI

Unregister Screen



Figure 62 - Unregister UI

Heartbeat

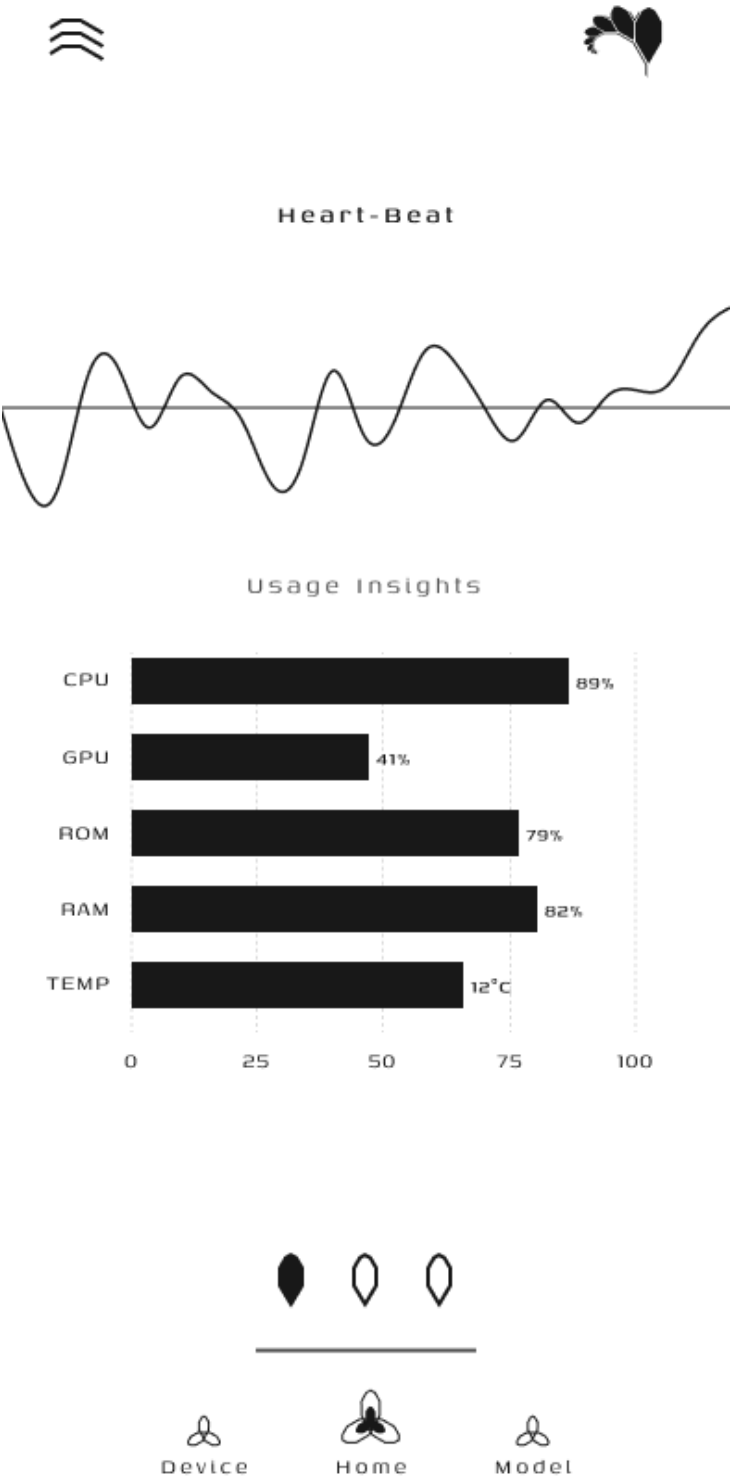


Figure 64 - Heartbeat UI

5.3 Server-side dashboard

The server-side dashboard displays the whether server is online or offline, number of clients reported, system metrics, number of rounds, pipeline of task and which is currently on going (initialization, data dispatch, on device training etc.)

The server dashboard is provided below:



Figure 65 – Server-side dashboard

6 Testing and Evaluation

6.1 Unit Testing

Unit Testing 1: About us components

Testing Objective: To ensure that the all components on click on about us it displays the join us, underline it with different color and on click shift to deice authentication

No.	Test case/Test script	Attribute and value	Expected result	Result
1	About us screen displays join us link	Launch About us	Display the detail with underline link	Pass
2	Check the underline Join_us	Observe About us screen	Displayed Join_us with underline link	Pass
3	Validate the join us link	Click join us	Navigates to device auth screen	Pass

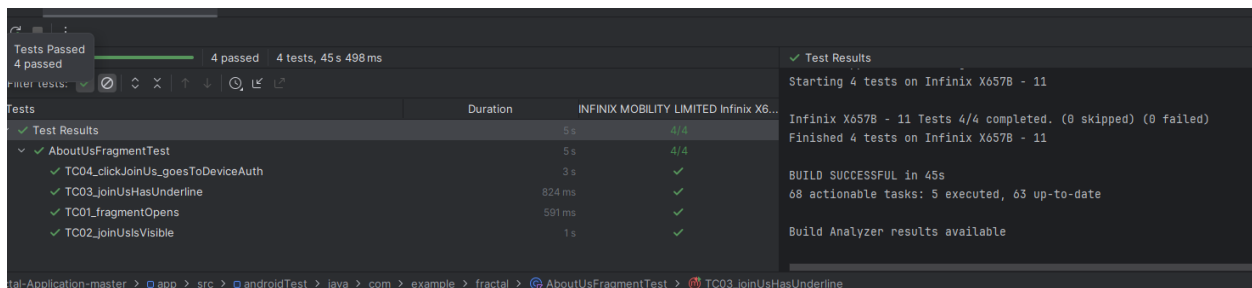


Figure 66 About us test

Unit Testing 2: Get started components

Testing Objective: To ensure working of components that the on click on about us it displays the join us, underline it with different color and on click shift to deice authentication

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Get started screen displays join us link	Launch Get started	Display the detail with link	Pass
2	Check the visible Get started	Observe Get started screen	Displayed Get started with Get started link	Pass
3	Validate the Get started link	Click Get started	Navigates to device Home screen	Pass

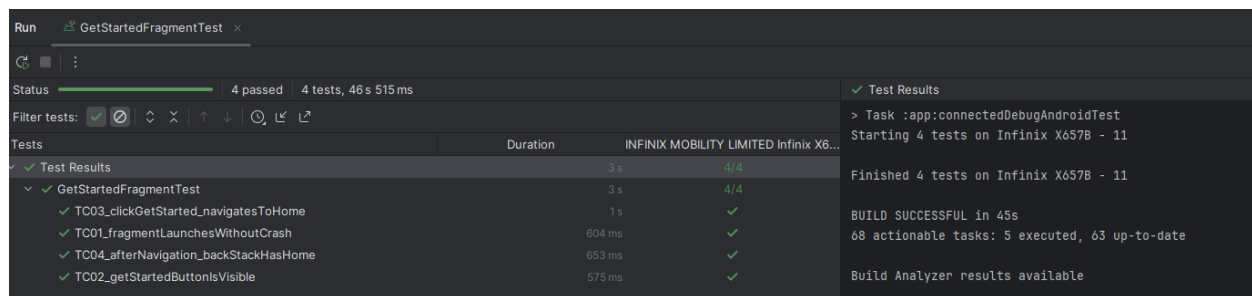


Figure 67 Get started test

Unit Testing 3: Device authorization components

Testing Objective: To ensure that the components as user login or register efficiently considering all conditions and input fields

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Validates launch of device auth and visibility of all input fields	No input	Displayed all input fields and button	Pass
2	Check leaving all or either one input field empty	Email, username, password = "Empty", Email = "Empty" username = "Empty", password = "Empty",	Please "Fill all the fields" Please "Fill all the fields" Please "Fill all the fields" Please "Fill all the fields"	Pass

3	Validate the work of “forget password”	Click the “forget password”	Navigates to forget password screen	Pass
4	Check terms & conditions checkbox	Checked Double click check	Enabled to login/register unchecked	Pass

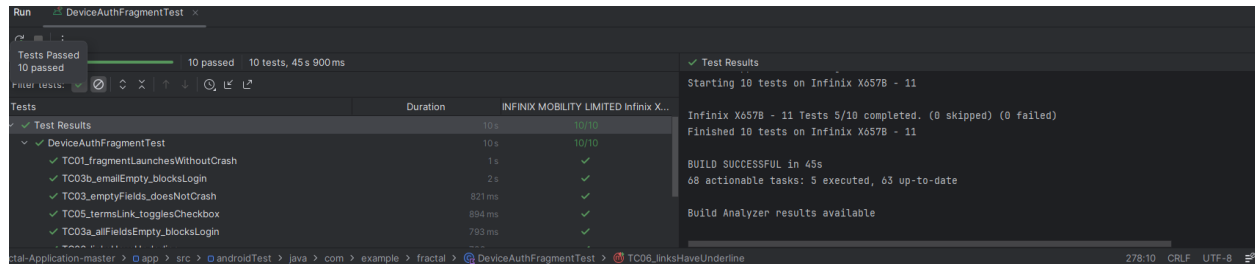


Figure 68 Device authorization test

Unit Testing 4: Setting screen components

Testing Objective: To ensure that the default components setting works, toggle WIFI, update the charging limit and it work accordingly

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Verify default configuration value	No input	OnWIFI=true, minchargelimit=20, onData=false etc.	Pass
2	Check Wi-Fi Toggle by state change	WIFI “on” data “on” WIFI “on” data “off”	Data still active on WIFI on WIFI on stay active	Pass
3	Update charge limit and validate range	Value= 50 Value= 0 Value= 100	Set the limit when to charge 50, 0 and 100	Pass
4	Update charge limit and validate range	Value= -5 Value= 110	Se reject	Pass

5	Validate toggle setting overnight, on charge	Call one Call twice	Return to original	Pass
---	--	----------------------------	--------------------	------

```

> Test Results 1 sec 414 ms ✓ 12 tests passed 12 tests total, 1 sec 414 ms
Executing tasks: [:app:testDebugUnitTest, --tests, com.example.fractal.SettingsViewModelTest] in project C:\Users\ashra\Down
> Task :app:checkKotlinGradlePluginConfigurationErrors SKIPPED
> Task :app:preBuild UP-TO-DATE
> Task :app:preDebugBuild UP-TO-DATE
> Task :app:dataBindingMergeDependencyArtifactsDebug UP-TO-DATE
> Task :app:generateDebugResValues UP-TO-DATE
> Task :app:generateDebugResources UP-TO-DATE
> Task :app:processDebugGoogleServices UP-TO-DATE

```

Figure 69 setting screen unit test

Unit Testing 5: Home compnents

Testing Objective: To ensure that the components user is able to active, inactive and pause the training and it displays the resources of the relevant device

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Check whether all resources info and diamond button visible	Click “Get started”	Diamond toggle button and resources displayed	Pass
2	Check the impact of first click on toggle button	Status “Inactive” click	Status change !=inactive	Pass
3	Check another impact on click	Status != server offline Click	Status change pause or cancel training	Pass
4	Check another impact on click	Status != waiting for data click	Status change pause or cancel training	Pass
5	Verify progress from diamond toggle button	Click diamond button	Within seconds diamond stats to change color to black 1,10,50,70, 100	Pass

6	Validate the extend value than range for diamond toggle	diamond.setProgress(-10) diamond.setProgress(110)	Toggle button didn't stop or crash	Pass
---	---	--	------------------------------------	------

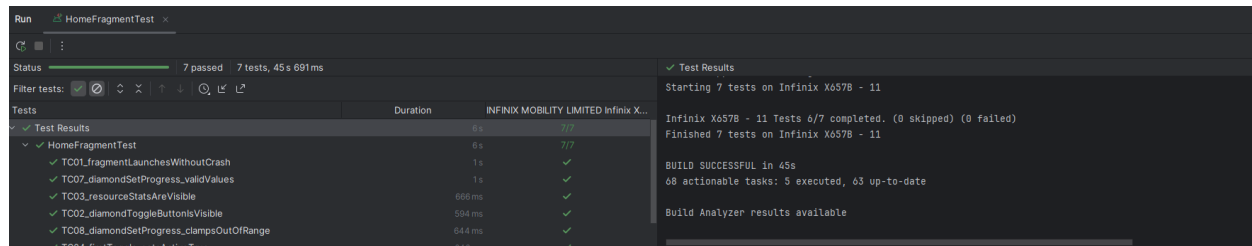


Figure 70 Home testing

Unit Testing 6: Unregister components

Testing Objective: To ensure that the component user be able to unregister the device and submit any problem

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Check the impact of leaving fields or any single one empty	Title = "" & decryption = "", Title = "" & decryption = "aaaaa", Title = "aliali" & decryption = "",	Unable to unregister/submit	Pass
2	Check the term uncheck result	Uncheck terms & conditions	unregister/submit	Pass
3	Validate the work double check the term check box and "back arrow"	Click the "back" Click the checkbox twice	Navigates to auth screen Unchecked to unchecked	Pass

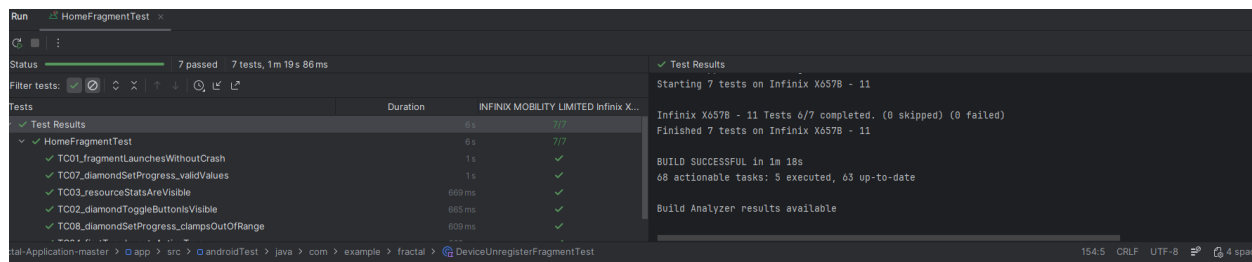


Figure 71 unregister

Unit Testing 7: Usage insight components

Testing Objective: To ensure that the component usage insight such as heartbeat and resource chart visible with their updates

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Check resource chart set values	CPU=0.8, GPU=0.5, ROM=0.7, RAM=0.6, TEMP=0.4	Values updated	Pass
2	Check resource chart set values to boundary values	CPU=0.0, GPU=1.0	Value updated silently	Pass
3	Verify heartbeat active	setActive(true) setActive(false)	View active inactive state	Pass
4	Verify heartbeat update with dummy data	updateStats(map)	Accept the input and didn't crash	Pass

✓ Test Results	0 s	7/7	> Task :app:checkDebugAndroidTestDuplicateClasses UP-TO-DATE
✓ HomeFragmentTest	0 s	7/7	> Task :app:desugarDebugAndroidTestFileDependencies UP-TO-DATE
✓ TC01_fragmentLaunchesWithoutCrash	1 s	✓	> Task :app:mergeExtDexDebugAndroidTest UP-TO-DATE
✓ TC07_diamondSetProgress_validValues	1 s	✓	> Task :app:mergeLibDexDebugAndroidTest UP-TO-DATE
✓ TC03_resourceStatsAreVisible	672 ms	✓	> Task :app:mergeDebugAndroidTestJniLibFolders UP-TO-DATE
✓ TC02_diamondToggleButtonsVisible	543 ms	✓	> Task :app:mergeDebugAndroidTestNativeLibs NO-SOURCE
✓ TC08_diamondSetProgress_clampsOutOfRange	641 ms	✓	> Task :app:stripDebugAndroidTestDebugSymbols NO-SOURCE
✓ TC04_firstToggle_setsActiveTrue	593 ms	✓	> Task :app:validateSigningDebugAndroidTest UP-TO-DATE
✓ TC05_firstClick_startsTraining	772 ms	✓	> Task :app:writeDebugAndroidTestSigningConfigVersions UP-TO-DATE
			> Task :app:compileDebugAndroidTestKotlin
			> Task :app:compileDebugAndroidTestJavaWithJavac UP-TO-DATE
			> Task :app:processDebugAndroidTestJavaRes UP-TO-DATE
			> Task :app:mergeDebugAndroidTestJavaResource UP-TO-DATE
			> Task :app:dexBuilderDebugAndroidTest

Figure 72 usage insight testing

Unit Testing 8: Model Initialization and Data Loading

Testing Objective: To ensure the model loading and batch data loading functions work correctly with valid and invalid inputs.

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Check model loading with valid TFLite file	Model file: "model.tflite" (valid asset)	Model loads successfully, interpreter is not null	Pass
2	Check model loading with invalid file name	Model file: "invalid_model.tflite"	Throws IOException, logs error message	Pass

Unit Testing 9: Batch Loading with Valid/Invalid Batch Index

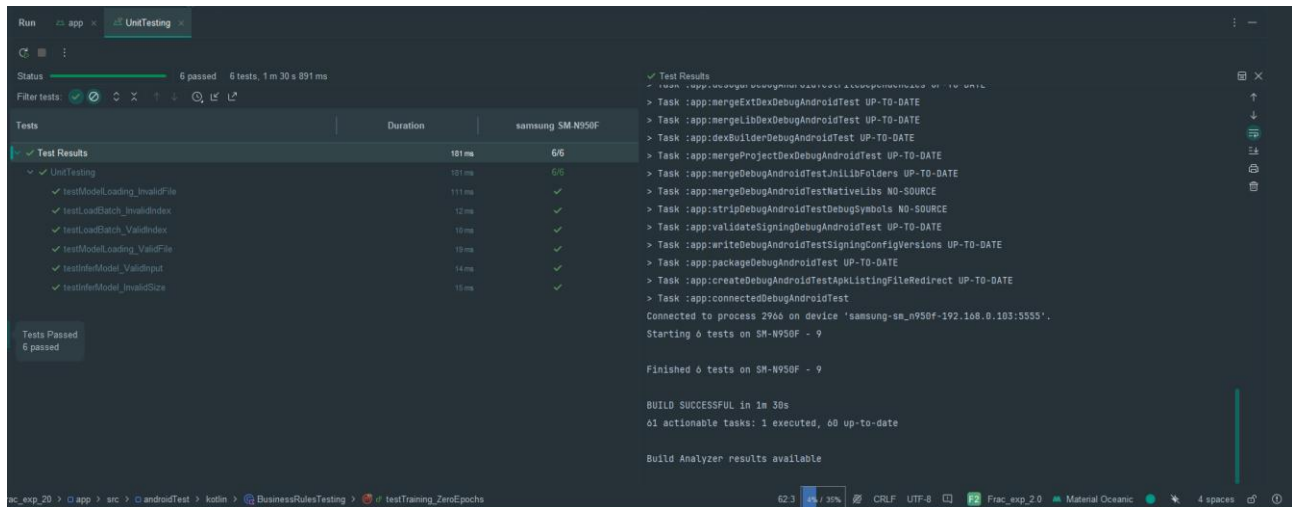
Testing Objective: To ensure batch loading handles correct skipping and reading of image/label data

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Load valid batch (e.g., batch 0)	Batch index: 0, Assets: "train_images.bin", "train_labels.bin"	Returns Pair of FloatBuffers with size BATCH_SIZE * IMG_HEIGHT * IMG_WIDTH and BATCH_SIZE * NUM_CLASSES	Pass
2	Load invalid batch (out of range)	Batch index: NUM_BATCHES + 1	Returns null, logs error for incomplete read	Pass

Unit Testing 9: Inference with Valid/Invalid Input

Testing Objective: To ensure inference processes correct-sized inputs and handles errors

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Infer with valid image array	testImage: FloatArray(784) { 0.5f }	Logs predicted class and probability, no errors	Pass
2	Infer with invalid size array	testImage: FloatArray(100)	Logs error for size mismatch (expected 784), returns early	Pass



6.2 Functional Testing

Functional Testing 1: Get started screen

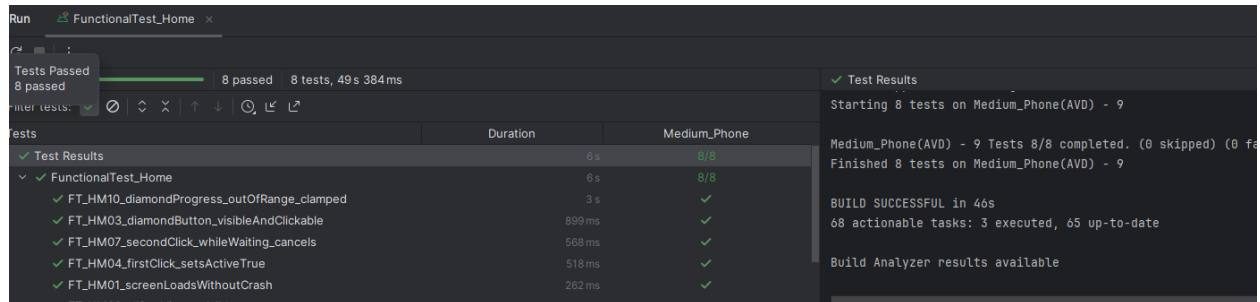
No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Get started with button visible	Open app	Visibility = true Isenabled = true	Starting screen visible with get started button at bottom	Pass
2	Get_started button clickable	On get started screen	Clickable =true	Button is clickable	Pass
3	Tap “Get started” button	On get started screen	Navigation_controller= navigation_home	Navigate to the home screen	Pass



Figure 73 FT get started

Functional Testing 2: Home screen

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Get resource stats visible	On home screen	All resource_visible = true	home screen visible resources	Pass
2	Diamond button visible and clickable	On home screen	Clickable =true	Button is clickable	Pass
3	First tap diamond button	isActive = false	Status!=inactive	Status updated to active, server offline, wait to be idle	Pass
4	second tap diamond button	inactive = false	Status!=active or server offline	Status paused	Pass
5	Diamond progress	Set progress (50)	Progress 50	No exception for valid	Pass



Functional Testing 3: Insight screen

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Get indicators visible	On default screen	All indicators_visible = true	Indicator model, device and home screen visible	Pass
2	Default page 1 and total 3	On device insights screen	Currentitem=1	Device isnight	Pass
3	Get right side indicator Heartbeat view	Click= right_indicator	SetActive(true)	Active the heartbeat view	Pass
4	Resource chart view	Click= left_indicator	SetActive(true)	Displays CPU, GPU RAM etc	Pass
5	Diamond progress	Set progress (50)	Progress 50	No exception for valid	Pass

Functional Testing 4: Full Training Cycle

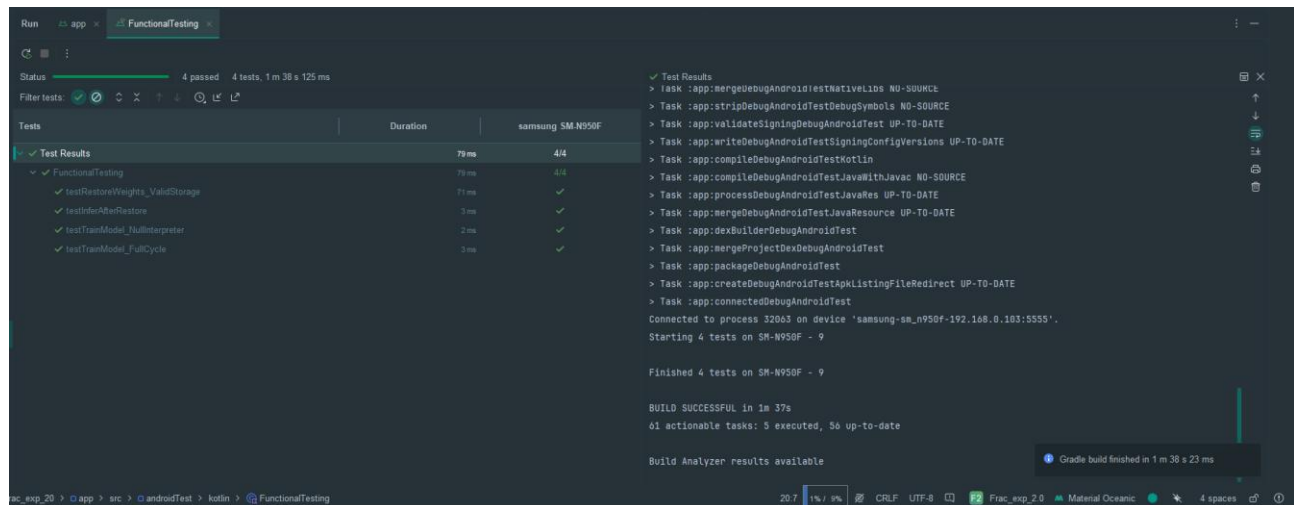
Objective: To ensure the training process completes epochs with batch loading and loss calculation without crashes.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Run full trainModel() with default params	NUM_EPOCHS=2, NUM_TRAININGS=6000, BATCH_SIZE=100	Completes all epochs, logs losses, processes all samples	Training completed, losses logged	Pass
2	Run trainModel() with null interpreter	interpreter = null	Logs "Interpreter not initialized" and returns early	Error logged, no training attempted	Pass

Functional Testing 5: Weight Restoration and Inference

Objective: To ensure weights can be saved/restored and inference works post-restoration.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Call restoreWeights() with valid storage	Checkpoint path: Documents/checkpoint.ckpt	Saves/restores weights, logs success, test inference runs	Weights restored, test inference successful	Pass
2	Call inferModel() after restore	testImage: Dummy zero array (784 floats)	Predicts class with probability, logs result	Predicted class 0, probability ~1.0	Pass

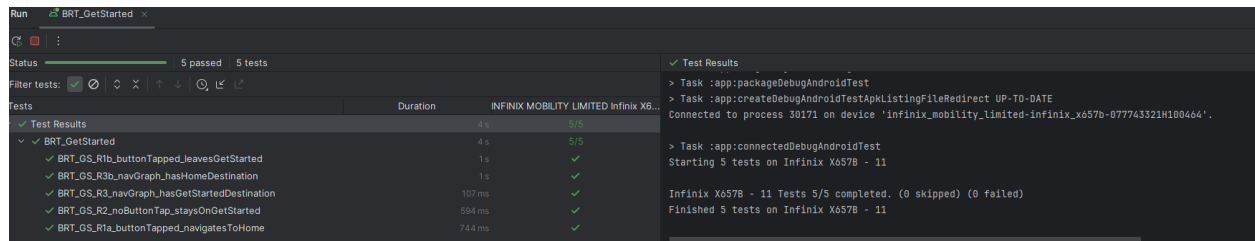


6.3 Business Rules Testing

Business Rules Testing 1: Getting started navigation

Objective: Validate business rules for navigating to home screen

Conditions/Rules	Rule 1: Button tapped	Rule 2: Button not tapped	Rule 3: Back pressed
btn_get_started clicked	Y	N	N
Back pressed	N	N	Y
Nav graph actions	Y	Y	Y
Actions/output			
Navigate to home	Y	N	N
Stay on nav get started	N	Y	N
Back stack	N	N	Y
Expected Output	Home screen loaded	Stays	App exit



Business Rules Testing 2: Device auth

Objective: Validate business rules for all input fields before auth attempted

Conditions/Rules	Rule 1: All valid	Rule 2: invalid email	Rule 3: invalid username	Rule 4: term not accepted	Rule 4: invalid password
Username	Y	Y	N	Y	Y
email	Y	N	Y	Y	Y
Password	Y	Y	Y	Y	N
Term and condition checked	Y	Y	Y	N	Y
Actions/output					
Et_email: error	N	Y	N	N	N
Et_username: error	N	N	Y	N	N
Et_password: error	N	N	N	N	Y
Please accept terms	N	N	N	Y	N
Btn_text= auth	Y	N	N	N	N
Proceed to server	Y	N	N	N	N
Expected Output	Auth proceed	Email field error	username field error	terms field error	password field error

Business Rules Testing 3: Device unregister

Objective: Validate business rules for all input fields before auth attempted

Conditions/Rules	Rule 1: feedback	Rule 2: feedback unregister	Rule 3: invalid/empty title	Rule 4: term not checked
problem	Y	Y	N	Y
Description	Y	Y	Y	Y
Term and condition checked	Y	Y	Y	N
Unregister checked	N	Y	N	N
Actions/output				
Feedback submitted	Y	Y	N	N
Device unregistered	N	Y	N	N
Error: empty	N	N	Y	N
Please accept terms	N	N	N	Y
Btn_submit	-	-	Y	Y
Expected Output	Feedback sent	Feedback sent + unregister	field error	terms field error

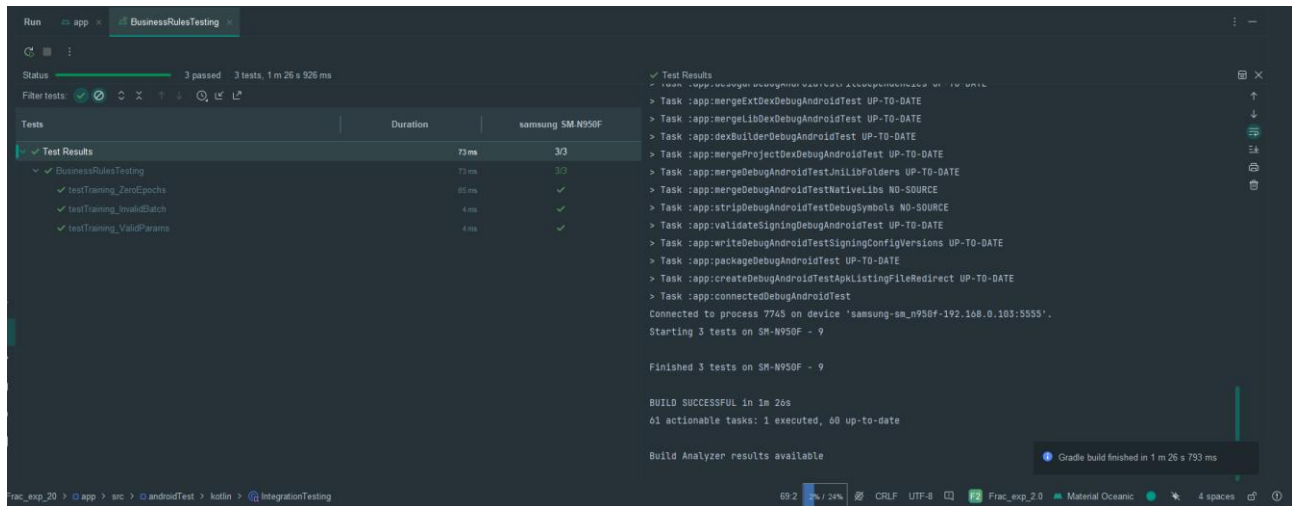


```
Run BRT_DeviceUnregister
Status: 5 passed, 5 tests
Filter tests: [Icons]
Tests:
  ✓ Test Results
  ✓ BRT_DeviceUnregister (0.5s)
    ✓ BRT_DU_R3a_emptyTitle_blocksSubmission (1s)
    ✓ BRT_DU_R2_validWithUnregister_bothProcessed (2s)
    ✓ BRT_DU_R1_validFeedbackOnly_submits (863 ms)
    ✓ BRT_DU_R3b_emptyDescription_blocksSubmission (867 ms)
    ✓ BRT_DU_R4_termsUnchecked_blocksSubmission (877 ms)
  ✓ Test Results
    > Task :app:packageDebugAndroidTest
    > Task :app:createDebugAndroidTestApkListingFileRedirect UP-TO-DATE
    Connected to process 31520 on device 'infinix_mobility_limited-infinix_x657b-077743321H100464'.
    > Task :app:connectedDebugAndroidTest
    Starting 5 tests on Infinix X657B - 11
    Infinix X657B - 11 Tests 5/5 completed. (0 skipped) (0 failed)
    Finished 5 tests on Infinix X657B - 11
```

Business Rules Testing 4: Training Parameter Validation

Objective: Validate business rules for training parameters (e.g., batch size divisibility, epoch limits).

Conditions/Rules	Rule 1: Valid Params	Rule 2: Invalid Batch	Rule 3: Zero Epochs	Actions/Outputs
NUM_EPOCHS > 0	Y	Y	N	Proceed to training
NUM_TRAININGS % BATCH_SIZE == 0	Y	N	Y	Full batches processed
BATCH_SIZE > 0	Y	Y	Y	Batches loaded correctly
Expected Output	Training completes	Logs warning for incomplete batch	Skips training loop	-



6.4 Integration Testing

Integration Testing 1: End-to-End Training and Inference

Testing Objective: To ensure integration between data streams, model interpreter, training, and inference modules works seamlessly.

No.	Test	Attribute and value	Expected result	Actual result	Result
-----	------	---------------------	-----------------	---------------	--------

	case/Test script				
1	Train then infer	Call trainModel(), then inferModel(FloatArray(784))	Training completes, inference predicts on trained model	Loss decreases over epochs, valid prediction logged	Pass
2	Restore then train	Call restoreWeights(), then trainModel()	Weights load correctly, training resumes without errors	Restoration successful, training proceeds	Pass

Integration Testing 2: Resource Cleanup

Testing Objective: To ensure cleanup integrates with interpreter and streams without leaks.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1	Train, infer, then cleanup	Full cycle + cleanup()	Interpreter and streams closed, no exceptions	Resources freed, no leaks in logs	Pass
2	Multiple cleanups	Call cleanup() twice	Second call safe, no crashes	Ignores second call gracefully	Pass

```

Run - app - IntegrationTesting -
Status 4 passed 4 tests, 1 m 35 s 18 ms
Filter tests: [Icons]

Tests
  ✓ Test Results 231 ms 4/4
    ✓ IntegrationTesting 231 ms 4/4
      ✓ testSessionThenTrain 190 ms ✓
      ✓ testTrainThenInfer 14 ms ✓
      ✓ testFullCycleWithCleanup 16 ms ✓
      ✓ testMultipleCleanups 11 ms ✓

  Tests Passed 4 passed

  Test Results
  ✓ Task :app:mergeExtDexDebugAndroidTest UP-TO-DATE
  > Task :app:mergeLibDexDebugAndroidTest UP-TO-DATE
  > Task :app:dexBuilderDebugAndroidTest UP-TO-DATE
  > Task :app:mergeProjectDexDebugAndroidTest UP-TO-DATE
  > Task :app:mergeDebugAndroidTestNativeLibs NO-SOURCE
  > Task :app:stripDebugAndroidTestDebugSymbols NO-SOURCE
  > Task :app:validateSigningDebugAndroidTest UP-TO-DATE
  > Task :app:writeDebugAndroidTestSigningConfigVersions UP-TO-DATE
  > Task :app:packageDebugAndroidTest UP-TO-DATE
  > Task :app:createDebugAndroidTestApkListingFileRedirect UP-TO-DATE
  > Task :app:connectedDebugAndroidTest
  Connected to process 18438 on device 'samsung-sm-n950f-192.168.0.103:5555'.
  Starting 4 tests on SM-N950F - 9

  Finished 4 tests on SM-N950F - 9

  BUILD SUCCESSFUL in 1m 34s
  41 actionable tasks: 1 executed, 60 up-to-date

  Build Analyzer results available

  Gradle build finished in 1 m 35 s 65 ms
  
```

7 Conclusion and Future Work

This conclusion and future work section demonstrates the successfully implemented decentralized distributed computation harvesting platform and the future enhancements going to happen in the platform

7.1 Conclusion

Fractal a decentralized distributed computation harvesting system demonstrates the utilization of computational power of mobile devices for model training. In this platform, computationally intensive task was divided into smaller chunks and distributed among the active idle mobile phone devices. Each device performs the assigned task locally for a fixed duration. After the duration, the processed data from the devices were be retrieved, aggregated, and federated into a single unit. The system demonstrated the feasibility of utilizing idle devices making it efficient, low-cost, and energy-efficient approach to providing resources than traditional ones. This project highlights the innovation of using idle mobile devices for training purposes.

Overall, the Fractal prove that mobile based computation harvesting is feasible, ecofriendly and especially for small AI/ML companies or organizations, independent researchers can be cost effective

7.2 Future Work

The current functionality of FRACTAL demonstrates the core functionality of the system, several features can be enhanced and added in the system like:

- The fractal is currently a computation harvesting platform for android devices only but in future we extend the devices including IOS and IOT
- To engage the user, we will implement the reward-based system just like Microsoft edge (daily login points, search points mean usage points combine to redeem the vouchers such as Shopen etc.). In this system, on daily login and usage of the application like

activation of the access to train the model, the user gets points through which user redeems the vouchers

- Enhance the security of model transmission so that devices remain safe from malicious updates
- Enhance the scalability of the system to make it efficient in real world circumstances

8. References

- [1] Adnan Mehonic, A. k. (2021). Brain-inspired computing: We need a master plan. *researchgate*. Retrieved from https://www.researchgate.net/publication/351221765_Brain-inspired_computing_We_need_a_master_plan
- [2] Anderson., D. D. (n.d.). Retrieved from BONIC: <https://boinc.berkeley.edu/>
- [3] Aslan, M. (2023). The Views of EFL Preparatory Students on the Use of Mobile Devices in Language Learning. *Researchgate*. Retrieved from https://www.researchgate.net/publication/371491269_The_Views_of_EFL_Preparatory_Students_on_the_Use_of_Mobile_Devices_in_Language_Learning
- [4] Bai, W., Jia, J., Deng, T., & Dong, M. (2024). Distributed Machine Learning with Electric Vehicles in Parking Lots. *2024 International Joint Conference on Neural Networks (IJCNN)*. Yokohama: IEEE. Retrieved from <https://ieeexplore.ieee.org/document/10651414>
- [5] Jane Theirs, F. e. (2025, January 9). *California wildfires raise alarm on water-guzzling AI like ChatGPT*. Retrieved from Fortune: <https://fortune.com/article/how-much-water-does-ai-use/>
- [6] Katherine Saltzman, J. m. (2025, July). How data centers have come to matter: Governing the spatial and environmental footprint of the “digital gateway to Europe”. *researchgate*. Retrieved from https://www.researchgate.net/publication/382094232_How_data_centers_have_come_to_matter_Governing_the_spatial_and_environmental_footprint_of_the_digital_gateway_to_Europe
- [7] Marin, D. (n.d.). Retrieved from NEXUS: <https://nexus.xyz/network>
- [8] Thomas Gabor, F. r. (2020). The Holy Grail of Quantum Artificial Intelligence: Major Challenges in Accelerating the Machine Learning Pipeline. *researchgate*. Retrieved from https://www.researchgate.net/publication/341039795_The_Holy_Grail_of_Quantum_Artificial_Intelligence_Major_Challenges_in_Accelerating_the_Machine_Learning_Pipeline